Cours ASPD
Principaux Algorithmes nRépartis
Telecom Nancy 2A IL

Dominique Méry
Telecom Nancy
Université de Lorraine

Année universitaire 2025-2026
3 mars 2026(9:42pm)

Sommaire

1. Election
   - Problème de l'élection
2. Election
   - Problème de l'élection
   - Election dans un graphe acyclique
3. Auto-stabilisation
   - Généralités
   - Définition
   - Réseaux en anneau
   - Algorithme de Dijkstra
   - Algorithme de coloriage
4. Calcul de l'état global
   - Introduction du problème
   - Etat global consistant
   - Algorithme de Chandy et Lamport

Section Courante

1. Election
   - Problème de l'élection
2. Election
   - Problème de l'élection
   - Election dans un graphe acyclique
3. Auto-stabilisation
   - Généralités
   - Définition
   - Réseaux en anneau
   - Algorithme de Dijkstra
   - Algorithme de coloriage
4. Calcul de l'état global
   - Introduction du problème
   - Etat global consistant
   - Algorithme de Chandy et Lamport

]

Etat courant

1. Election
   - Problème de l'élection
2. Election
   - Problème de l'élection
   - Election dans un graphe acyclique
3. Auto-stabilisation
   - Généralités
   - Définition
   - Réseaux en anneau
   - Algorithme de Dijkstra
   - Algorithme de coloriage
4. Calcul de l'état global
   - Introduction du problème
   - Etat global consistant
   - Algorithme de Chandy et Lamport

Election du leader

- Un algorithme d'élection dun leader satisfait les propriétés suivantes :
  - Chaque processus ou site du réseau exécute le même algorithme.
  - L'algorithme est décentralisé : le calcul peut être initialisé par un sous-ensemble arbitraire de processus
  - L'algorithme atteint une configuration terminale pour chaque calcul et dans toute configuration terminale accessible, il n'y a qu'un seul processus dans l'état de savoir qu'il est leader et tous les autres savent qu'ils ne sont pas leader c'est-à-dire perdus.
- Algorithmes considérés
  - Algorithme de LeLann et Chang et Roberts (anneau)
  - Algorithme IEEE 1394 (arborescence)

Problème de l'élection d'un leader

- Les sites d'un réseau n'ont pas conscience des autres sites non connectés à eux.
- Chaque site a une connaissance locale
- Organiser des calculs dans un réseau nécessite de coordiner les calculs : besoin d'un leader.
- Calculs répartis par ondelettes

Algorithme de LeLann

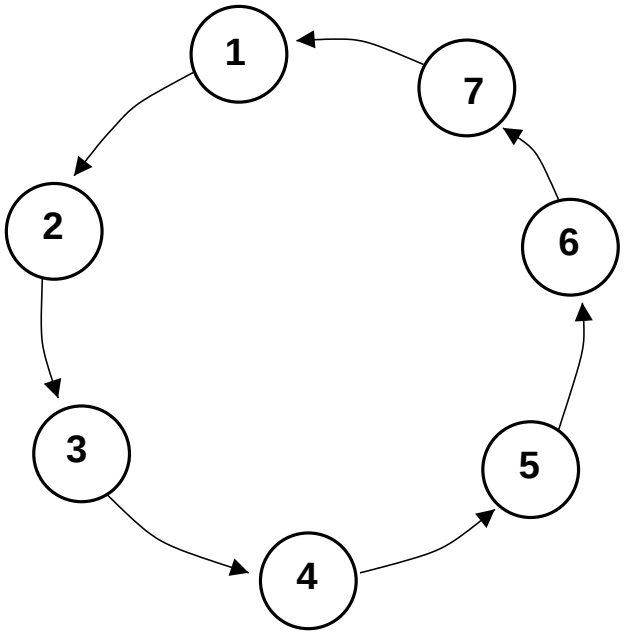
- Un ensemble de nœuds connectés en anneau
- Un sous-ensemble des nœuds est appelé l'ensemble des **candidats** à l'élection
- Chaque candidat maintient un ensemble de valeurs reçues des autres nœuds.
- Si un nœud reçoit le jeton d'un autre nœud et qu'il n'est pas candidat, il ne peut plus être candidat et passe le jeton à son voisin.
- Quand un nœud a reçu son jeton envoyé, il regarde s'il est le jeton de numéro le plus petit et il est élu.

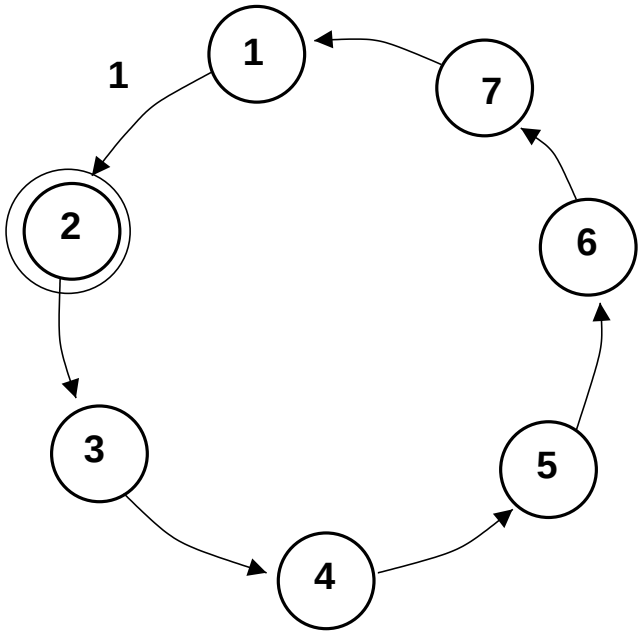
Algorithme de Chang et Roberts

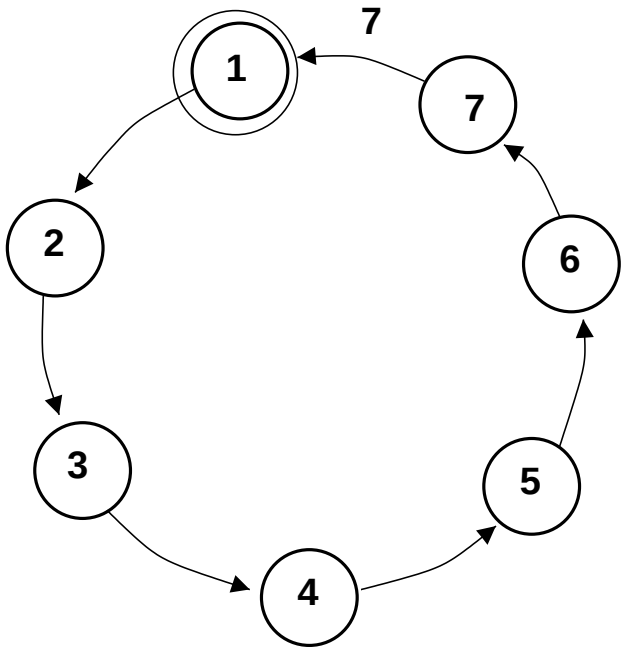
- Un ensemble de nœuds connectés en anneau
- Un sous-ensemble des nœuds est appelé l'ensemble des candidats à l'élection
- Si un nœud candidat reçoit un jeton plus petit que son jeton, il passe le jeton et se déclare pedu, sinon il garde le jeton.
- Si un nœud reçoit le jeton d'un autre nœud et qu'il n'est pas candidat, il ne peut plus être candidat et passe le jeton à son voisin.
- Quand un nœud a reçu son jeton envoyé, il est élu.

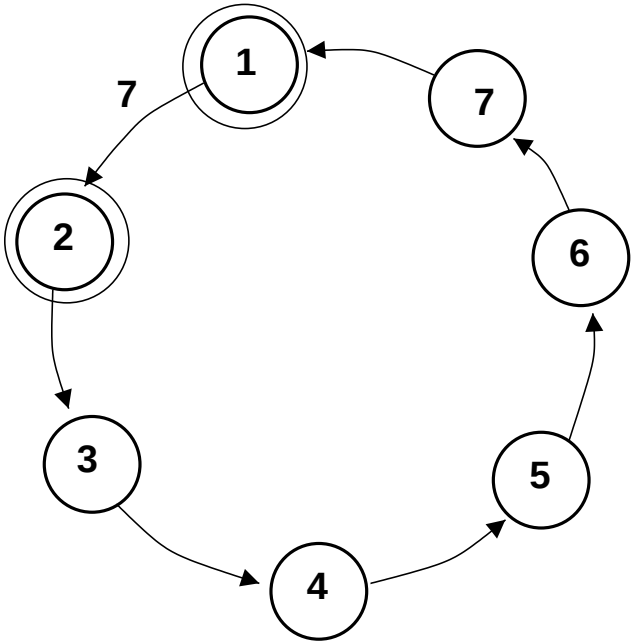
Algorithme LRC Chang et Roberts

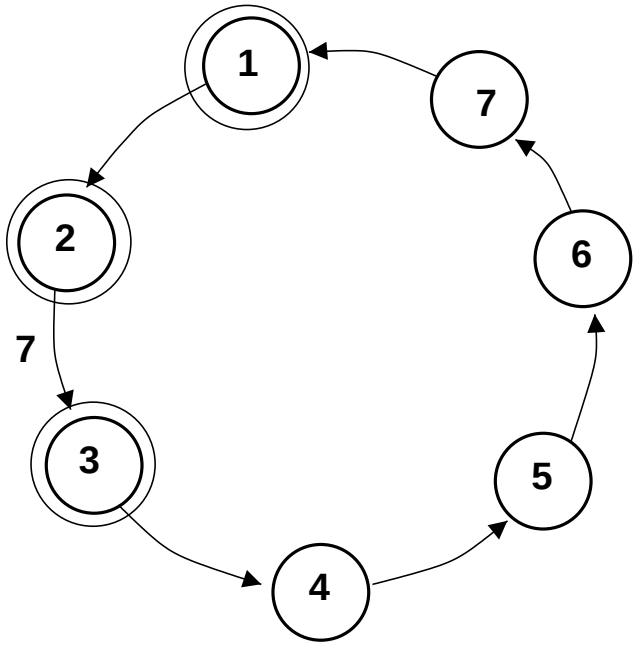
- Un réseau en anneau comporte des nœuds ayant un numéro différent.
- Un des nœuds ets le maximum du réseau
- Le processus d'élection transmet un jeton avec une valeur mise à jour à chaque nœud.

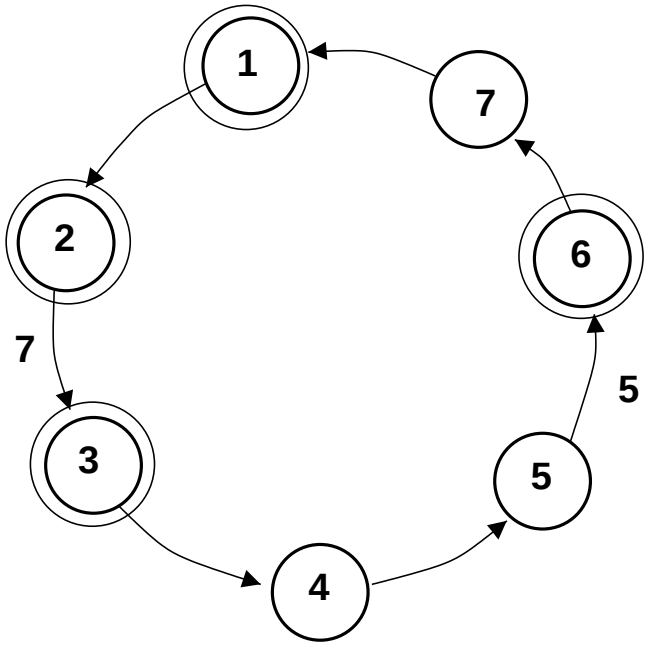

1
2
3
4
5
6
7

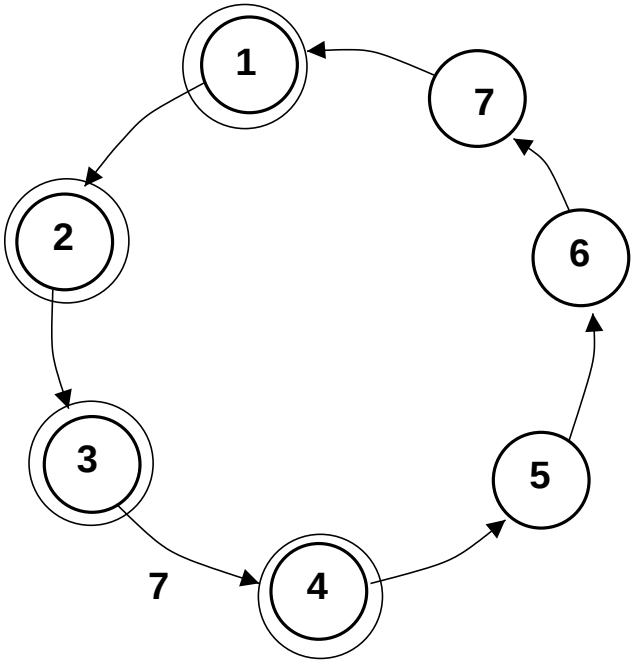

1
1
2
3
4
5
6
7

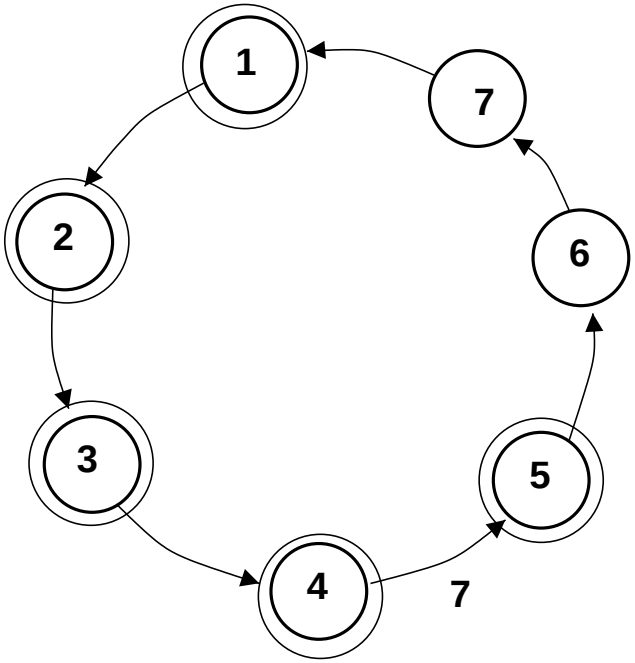

7
1
7
2
6
3
5
4


1
7
2
3
4
5
6
7


1
2
3
4
5
6
7
7


1
7
2
6
7
5
3
5
4


1
2
3
4
5
6
7
7


1
2
3
4
5
6
7
7

Algorithme LCR

- Seul le nœud de numéro maximal sera élu leader
- Les nœuds de numéro minimal restent dans l'état inconnu

Section Courante

1. Election
   - Problème de l'élection
2. Election
   - Problème de l'élection
   - Election dans un graphe acyclique
3. Auto-stabilisation
   - Généralités
   - Définition
   - Réseaux en anneau
   - Algorithme de Dijkstra
   - Algorithme de coloriage
4. Calcul de l'état global
   - Introduction du problème
   - Etat global consistant
   - Algorithme de Chandy et Lamport

]

Etat courant

1. Election
   - Problème de l'élection
2. Election
   - Problème de l'élection
   - Election dans un graphe acyclique
3. Auto-stabilisation
   - Généralités
   - Définition
   - Réseaux en anneau
   - Algorithme de Dijkstra
   - Algorithme de coloriage
4. Calcul de l'état global
   - Introduction du problème
   - Etat global consistant
   - Algorithme de Chandy et Lamport

Election du leader

- Un algorithme d'élection dun leader satisfait les propriétés suivantes :
  - Chaque processus ou site du réseau exécute le même algorithme.
  - L'algorithme est décentralisé : le calcul peut être initialisé par un sous-ensemble arbitraire de processus
  - L'algorithme atteint une configuration terminale pour chaque calcul et dans toute configuration terminale accessible, il n'y a qu'un seul processus dans l'état de savoir qu'il est leader et tous les autres savent qu'ils ne sont pas leader c'est-à-dire perdus.
- Algorithmes considérés
  - Algorithme de LeLann et Chang et Roberts (anneau)
  - Algorithme IEEE 1394 (arborescence)

Problème de l'élection d'un leader

- Les sites d'un réseau n'ont pas conscience des autres sites non connectés à eux.
- Chaque site a une connaissance locale
- Organiser des calculs dans un réseau nécessite de coordiner les calculs : besoin d'un leader.
- Calculs répartis par ondelettes

Algorithme de LeLann

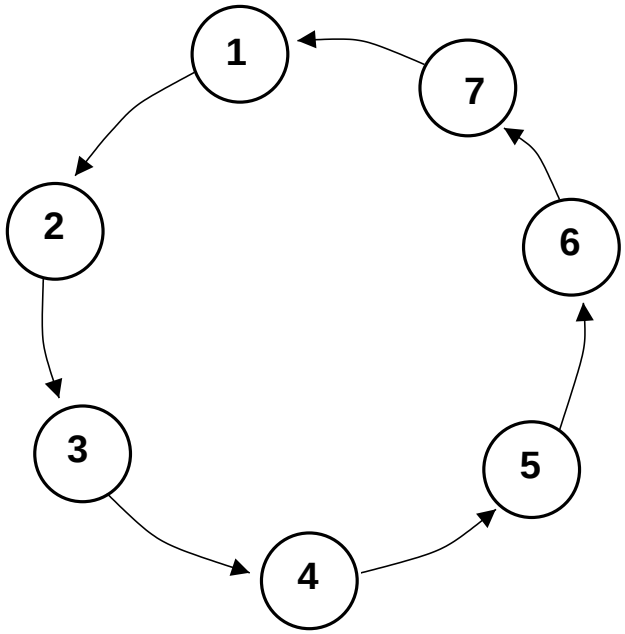
- Un ensemble de nœuds connectés en anneau
- Un sous-ensemble des nœuds est appelé l'ensemble des **candidats** à l'élection
- Chaque candidat maintient un ensemble de valeurs reçues des autres nœuds.
- Si un nœud reçoit le jeton d'un autre nœud et qu'il n'est pas candidat, il ne peut plus être candidat et passe le jeton à son voisin.
- Quand un nœud a reçu son jeton envoyé, il regarde s'il est le jeton de numéro le plus petit et il est élu.

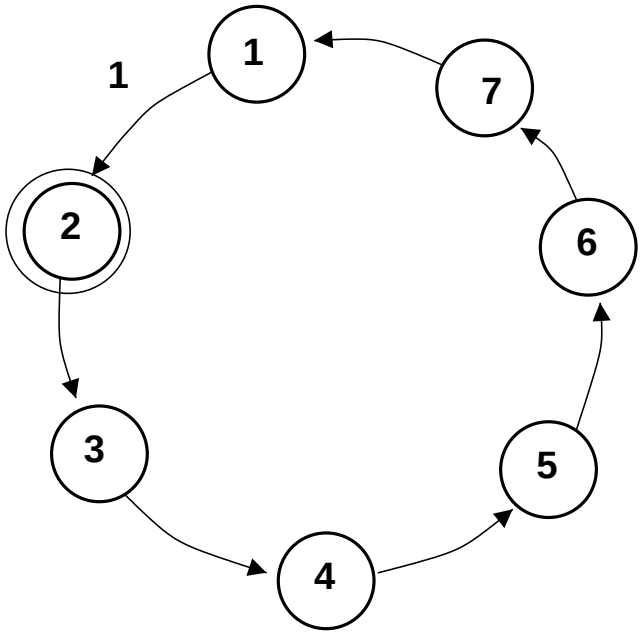
Algorithme de Chang et Roberts

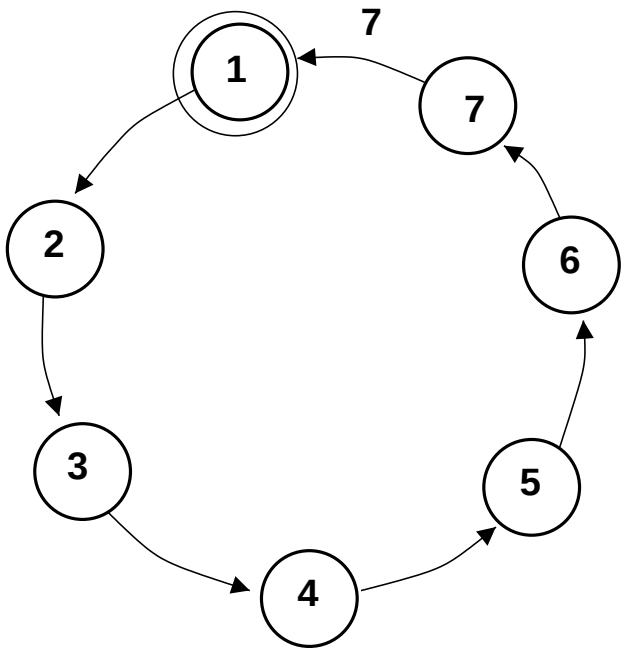
- Un ensemble de nœuds connectés en anneau
- Un sous-ensemble des nœuds est appelé l'ensemble des candidats à l'élection
- Si un nœud candidat reçoit un jeton plus petit que son jeton, il passe le jeton et se déclare pedu, sinon il garde le jeton.
- Si un nœud reçoit le jeton d'un autre nœud et qu'il n'est pas candidat, il ne peut plus être candidat et passe le jeton à son voisin.
- Quand un nœud a reçu son jeton envoyé, il est élu.

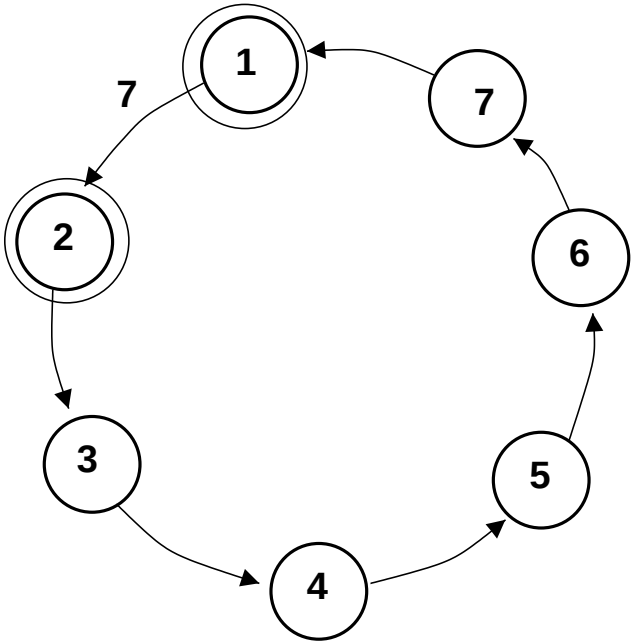
Algorithme LRC Chang et Roberts

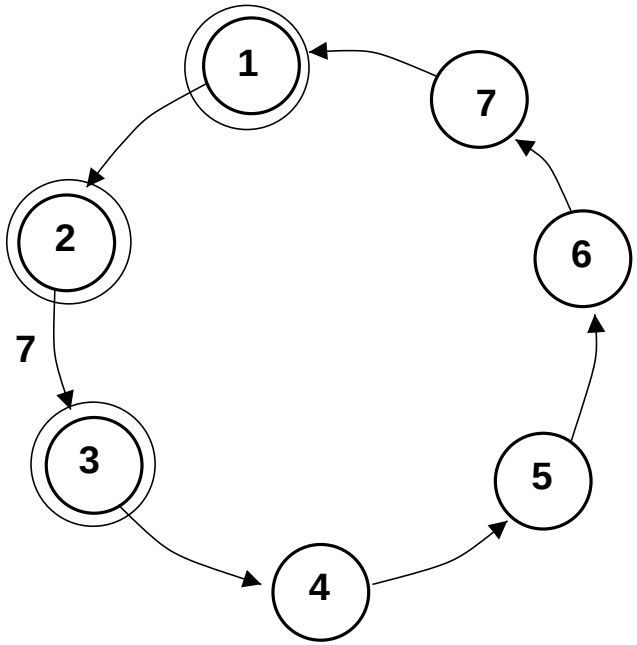
- Un réseau en anneau comporte des nœuds ayant un numéro différent.
- Un des nœuds ets le maximum du réseau
- Le processus d'élection transmet un jeton avec une valeur mise à jour à chaque nœud.

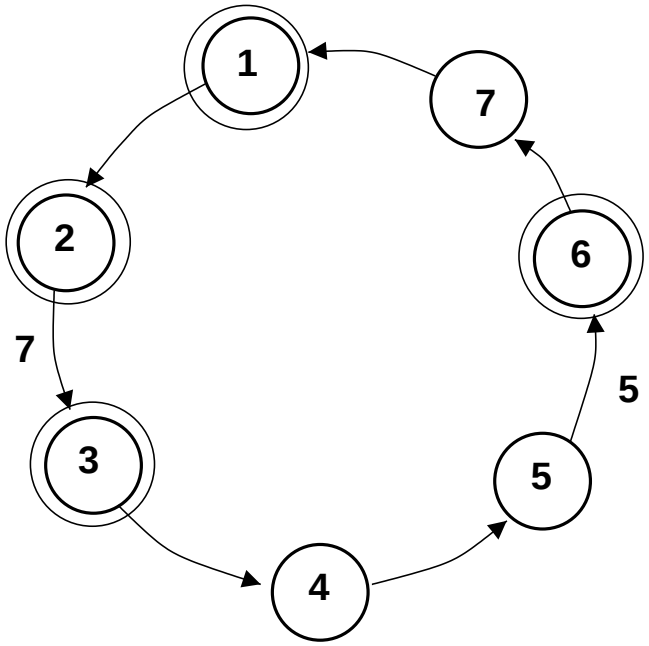

1
2
3
4
5
6
7

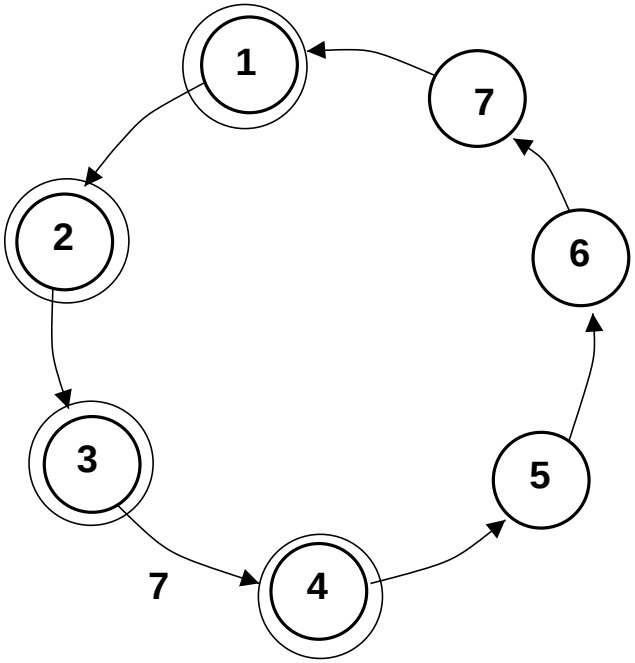

1
1
2
3
4
5
6
7

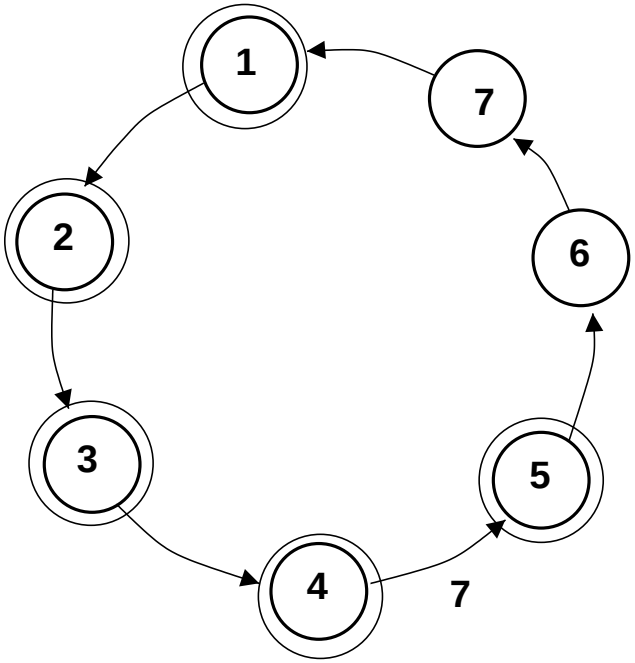

7
1
7
2
6
3
5
4


1
7
2
3
4
5
6
7


1
2
3
4
5
6
7
7


1
7
2
6
7
5
3
5
4


1
2
3
4
5
6
7
7


1
2
3
4
5
6
7
7

Algorithme LCR

- Seul le nœud de numéro maximal sera élu leader
- Les nœuds de numéro minimal restent dans l'état inconnu

Etat courant

1. Election
   - Problème de l'élection
2. Election
   - Problème de l'élection
   - Election dans un graphe acyclique
3. Auto-stabilisation
   - Généralités
   - Définition
   - Réseaux en anneau
   - Algorithme de Dijkstra
   - Algorithme de coloriage
4. Calcul de l'état global
   - Introduction du problème
   - Etat global consistant
   - Algorithme de Chandy et Lamport

(FireWire)

- standard international
- Intérêts commerciaux pour sa correction
- Sun, Apple, Philips, Microsoft, Sony, etc impliqués dans son développment
- Trois couches (physique, liaison, transaction)
- Le protocole d'étude est le Tree Identify Protocol
- Localisé dans la phase Bus Reset de la couche physique

Le Problème (1)

- Le bus est utilisé pour acheminer des siganux audio et vidéo digitalisés.
- Il est "hot-pluggable"
- Unités et périphériques peuvent être ajoutés ou retirés à tout instant.
- De tels changements sont suivis d'un bus reset
- L'élection du leader a lieu après un reset dans le réseau.
- Un leader doit être choisi pour agir en tant que manager du bus

Le Problème (2)

- Après un reset du bus tous les nœuds ont même statut.
- Tout nœud sait à quels nœuds il est directement connecté.
- Le réseau est connexe
- Le réseau est acyclique

Références (1)

BASE

- IEEE. *IEEE Standard for a High Performance Serial Bus. Std 1394-1995*. 1995
- IEEE. *IEEE Standard for a High Performance Serial Bus (supplement). Std 1394a-2000*. 2000

Références (2)

GENERAL

- N. Lynch. *Distributed Algorithms*. Morgan Kaufmann. 1996
- R. G. Gallager et al. *A Distributed Algorithm for Minimum Weight Spanning Trees*. IEEE Trans. on Prog. Lang. and Systems. 1983.

Références (3)

MODEL CHECKING

- D.P.L. Simons et al. *Mechanical Verification of the IEE 1394a Root Contention Protocol using Uppaal2* Springer International Journal of Software Tools for Technology Transfer. 2001
- H. Toetenel et al. *Parametric verification of the IEEE 1394a Root Contention Protocol using LPMC* Proceedings of the 7th International Conference on Real-time Computing Systems and Applications. IEEE Computer Society Press. 2000

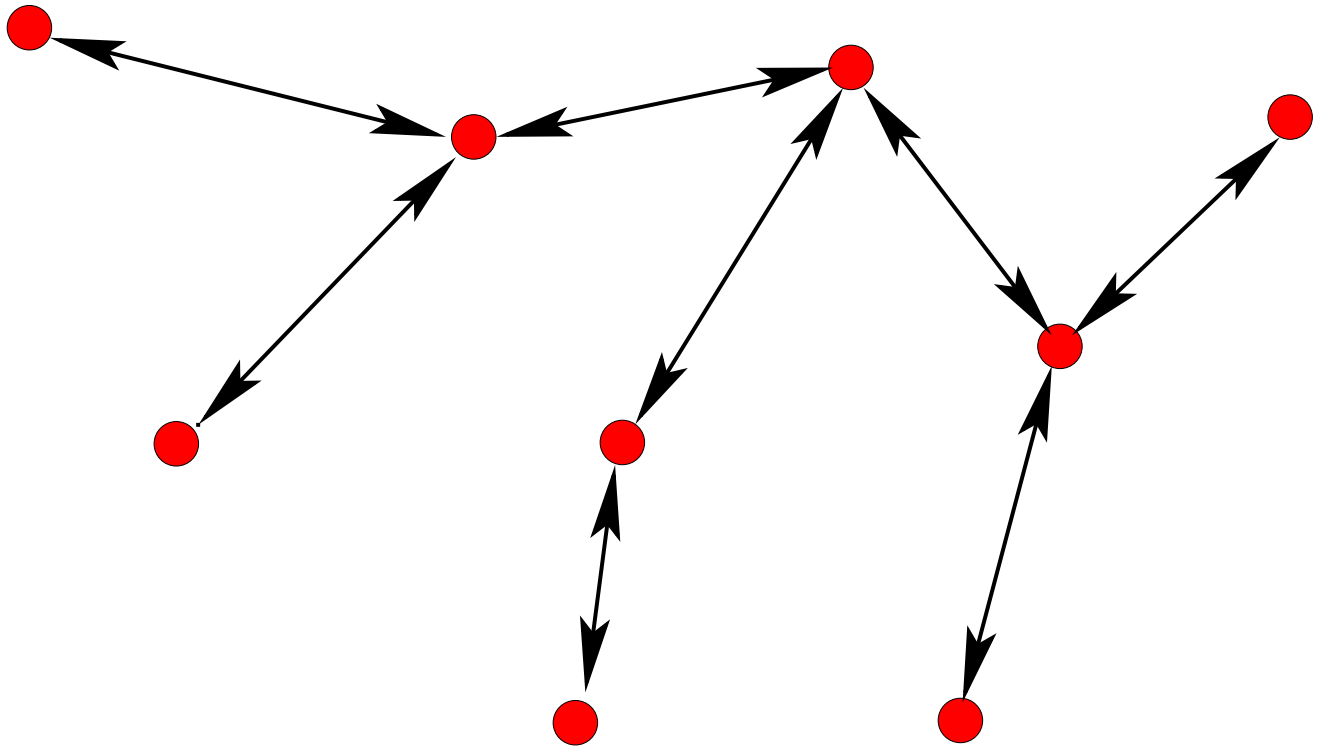
Références (4)

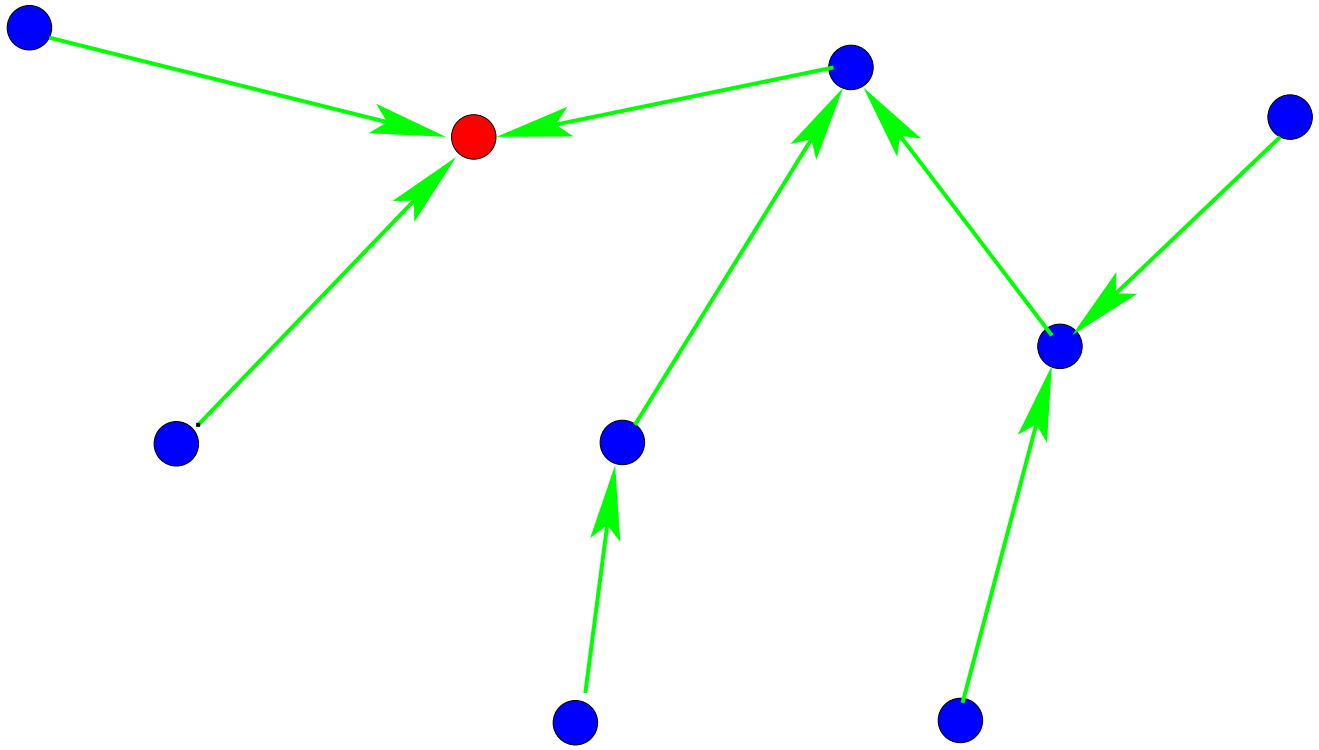
THEOREM PROVING

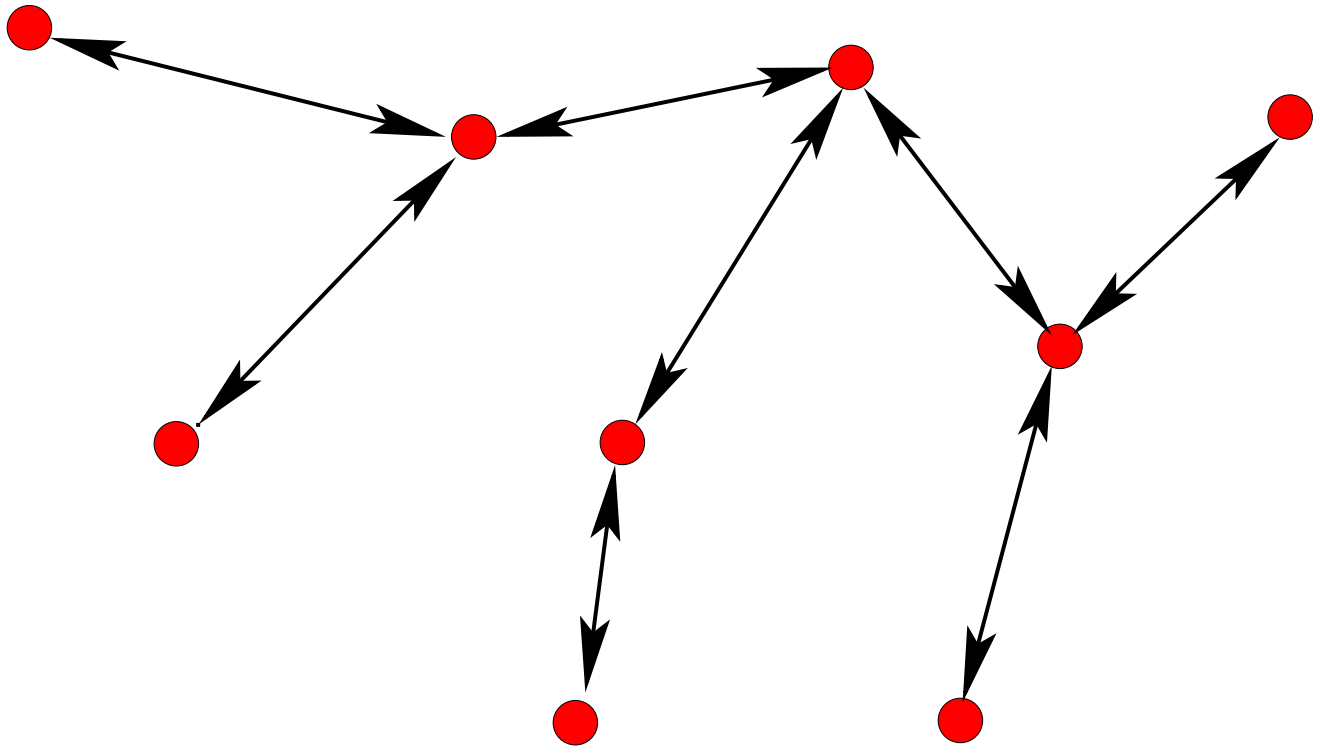
- M. Devillers et al. *Verification of the Leader Election : Formal Method Applied to IEEE 1394*. Formal Methods in System Design. 2000
- J.R. Abrial et al. *A Mechanically Proved and Incremental Development of IEEE 1394*. Formal Aspects of Computing, 2003.

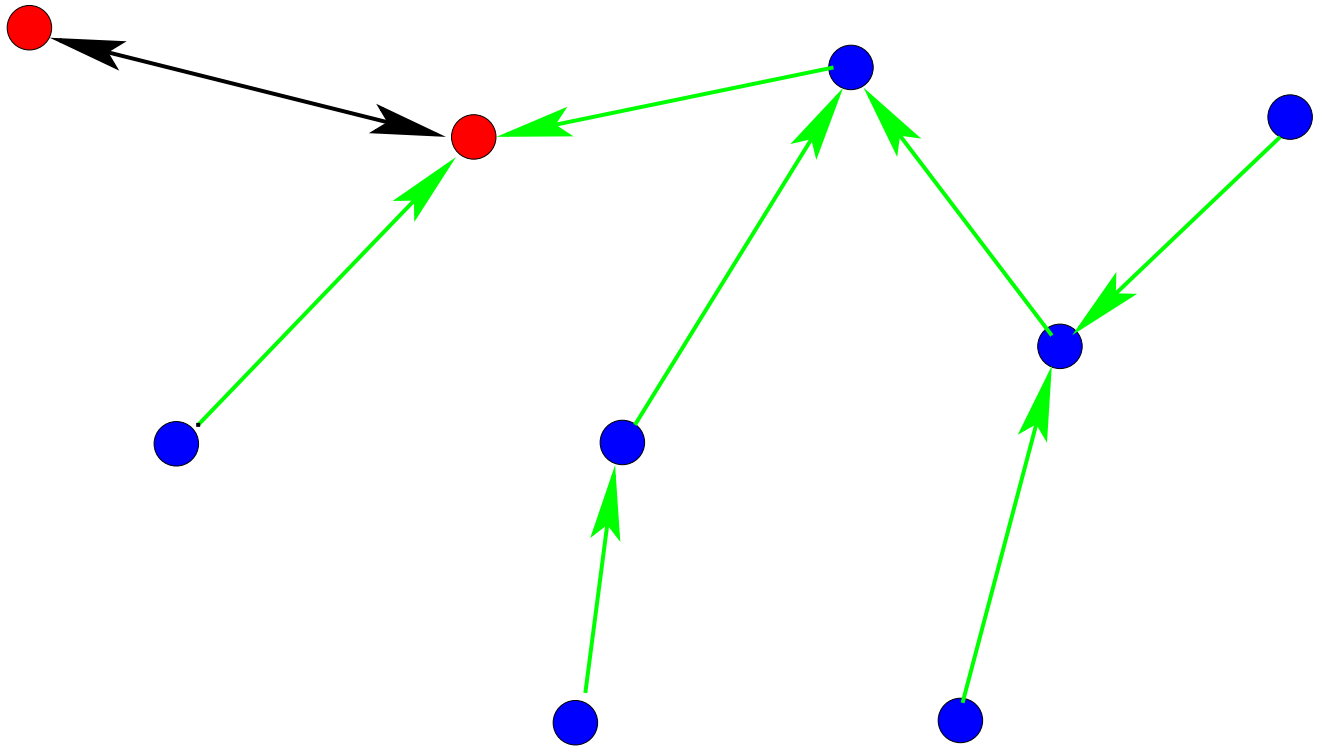
Propriétés informelles abstraites du protocole

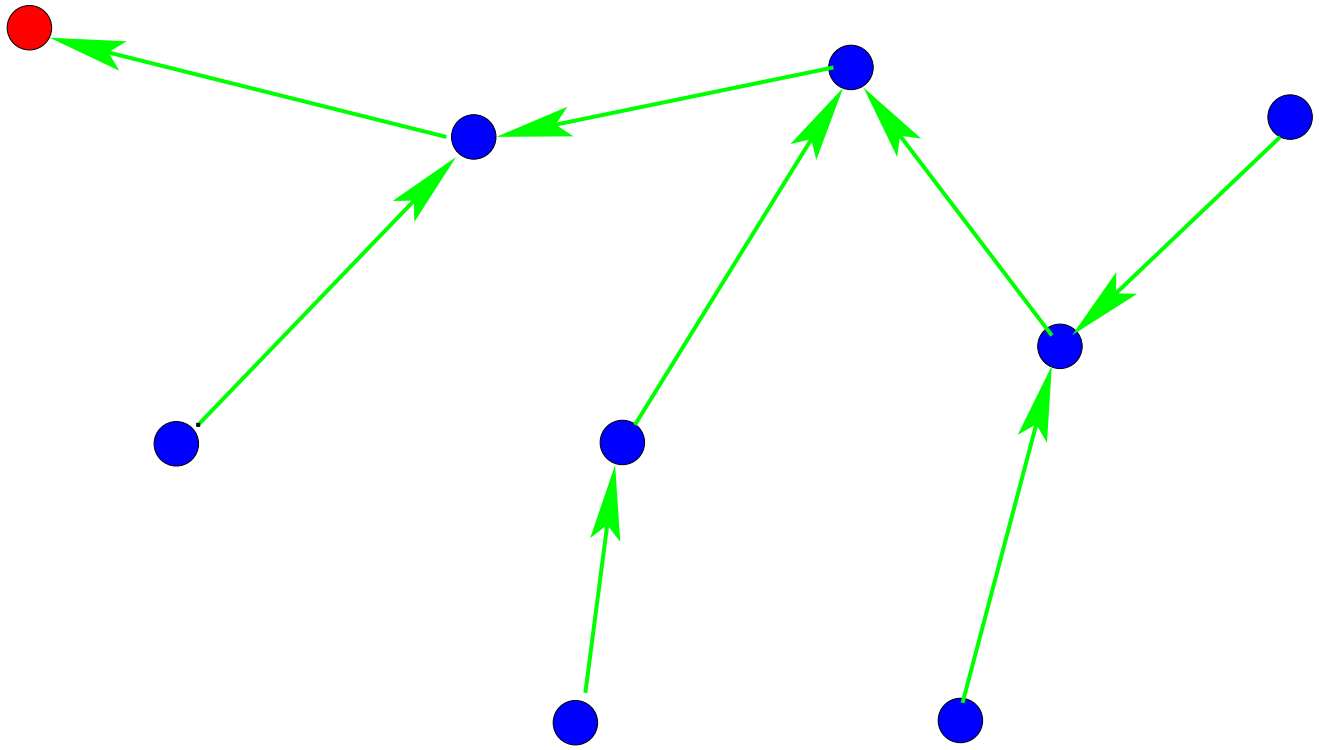
- Un réseau connexe et acyclique de nœuds.
- Nœuds reliés par des canaux bidirectionnels
- Election en un temps fini d'un leader.
- De manière répartie et non-déterministe.
- Un exemple d'animation du protocole.





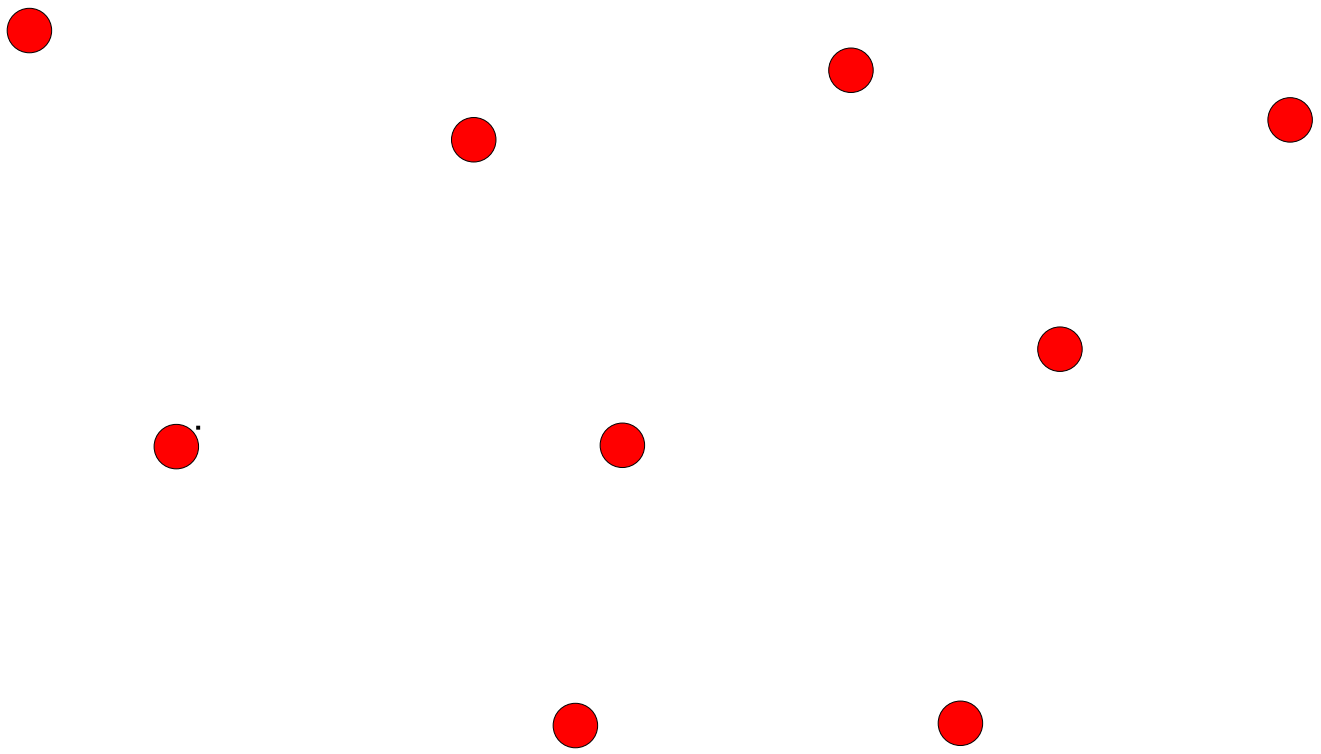




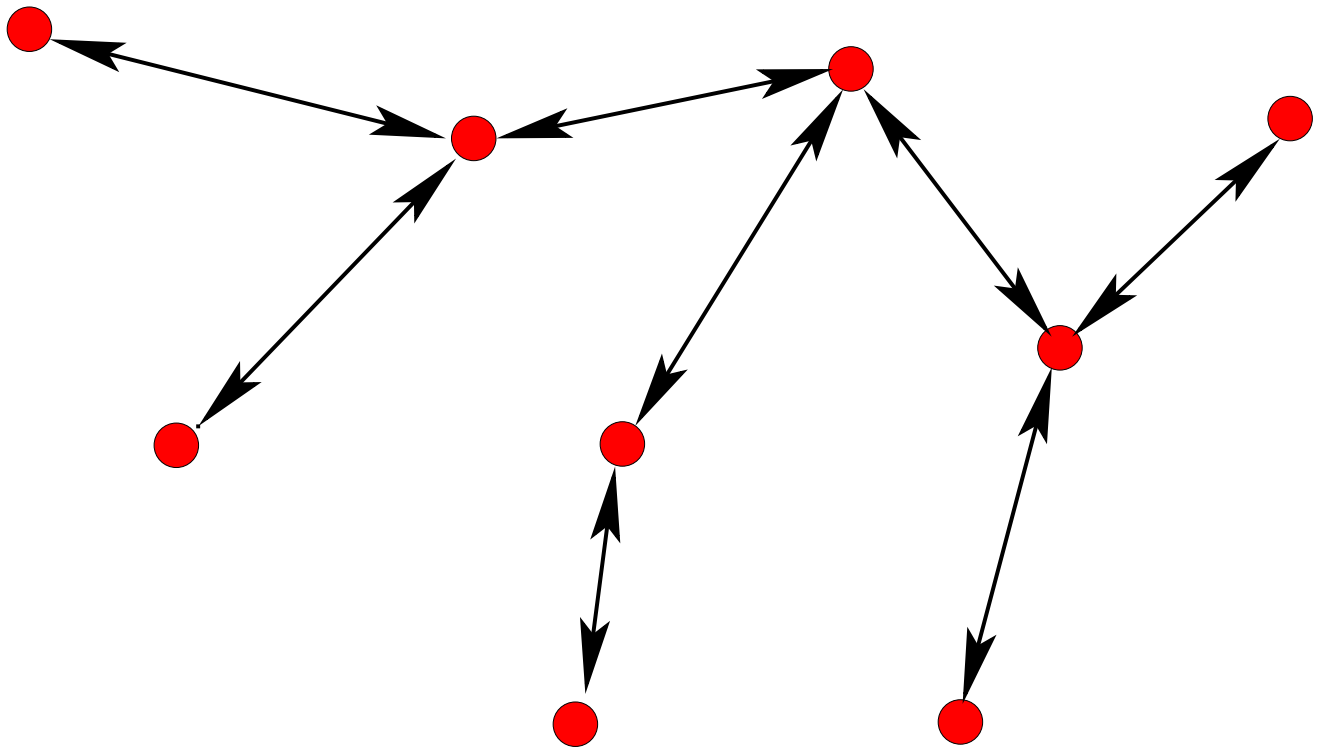


Le processus de développement

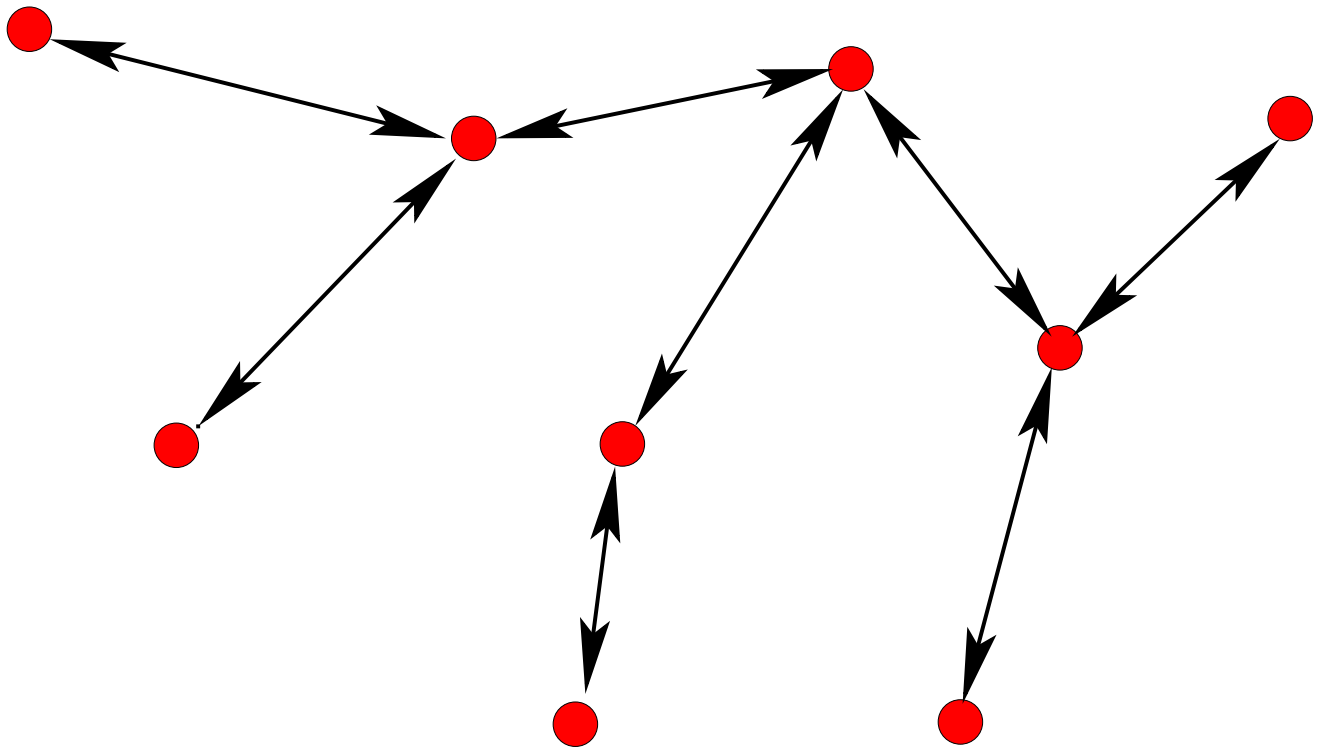
- Définition et propriétés du réseau dans un style formel
- Un modèle abstrait ¡¡one-shot¿¿ du protocole.
- Présentation d'une solution centralisée mais encore abstraite !
- Introduction du mécanisme de message passing entre les nœuds et des délais
- Modification de la structure de donnée pour répartir le protocole



ND un ensemble de noeuds (au moins 2 noeuds)



gr un graphe construit et défini sur ND



gr est un graphe symétrique et non réflexif

gr : un graphe construit sur ND $\qquad$ $gr \subseteq ND \times ND$

gr : un graphe construit sur ND $\qquad$ $gr \subseteq ND \times ND$

gr est défini sur ND $\qquad$ $\mathsf{dom}\,(gr) = ND$

gr : un graphe construit sur ND $\quad gr \subseteq ND \times ND$

gr est défini sur ND $\quad \text{dom}\,(gr) = ND$

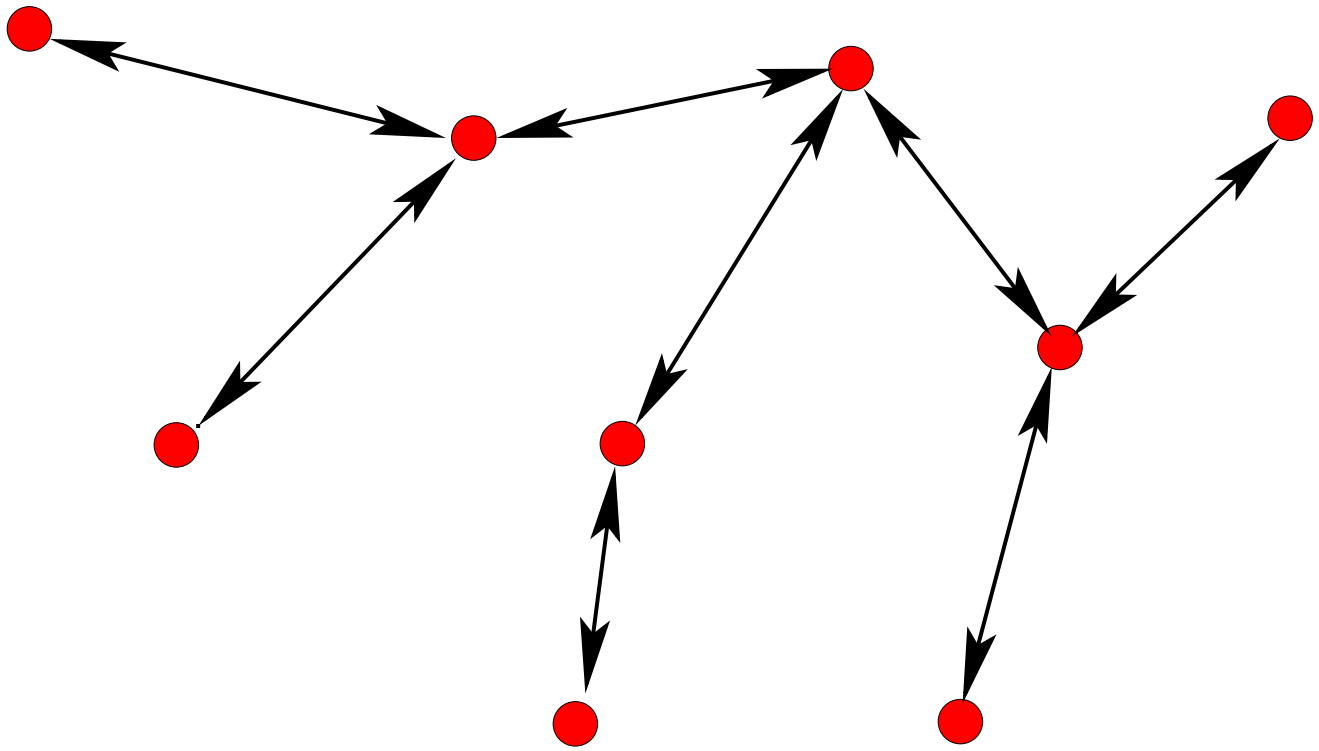
gr est symétrique $\quad gr = gr^{-1}$

gr : un graphe construit sur ND $\quad gr \subseteq ND \times ND$

gr est défini sur ND $\quad \mathsf{dom}\,(gr) = ND$

gr est symétrique $\quad gr = gr^{-1}$

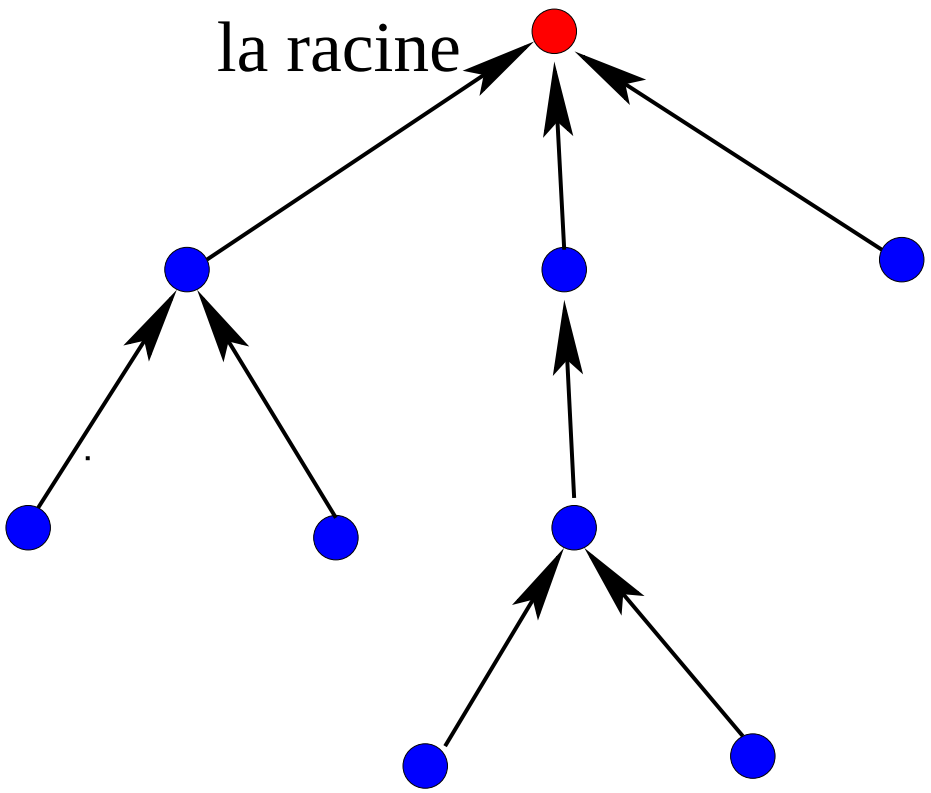
gr est non réflexif $\quad \mathsf{id}\,(ND) \cap gr =$



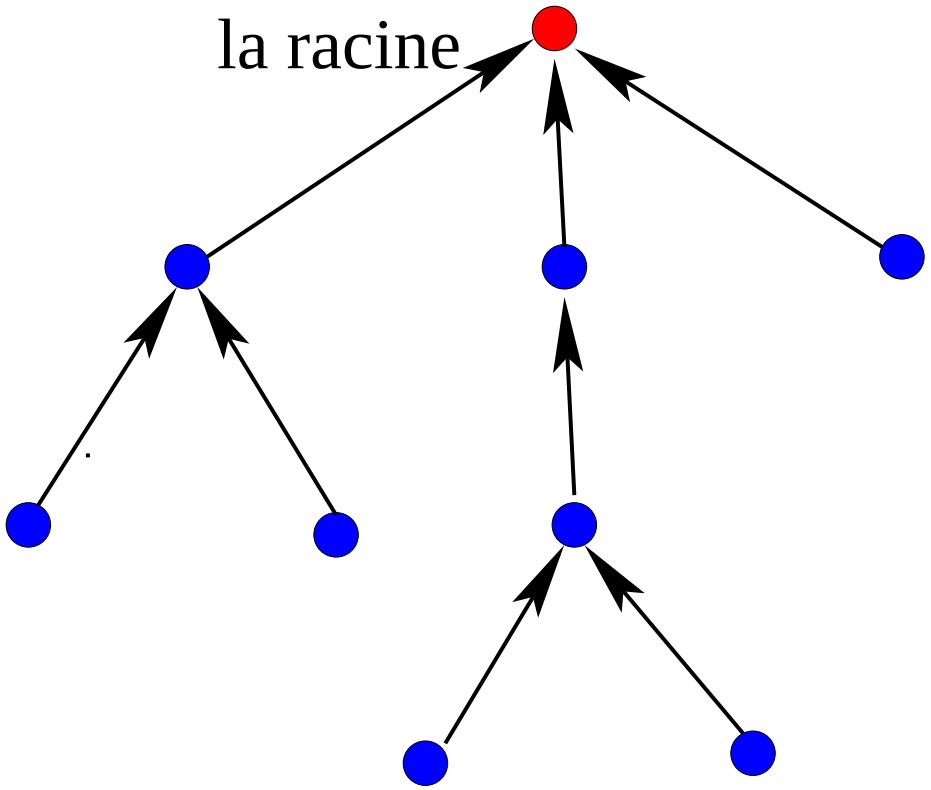
gr est connexe et acyclique

Un détour par les Arbres

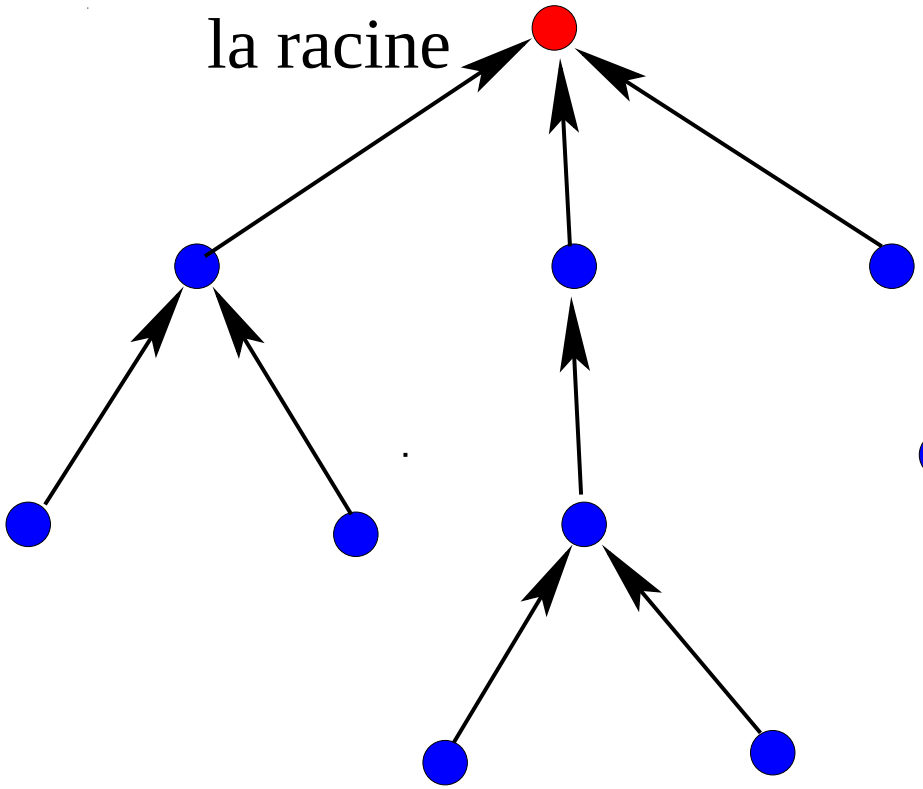
- Un arbre est un graphe particulier
- Un arbre a une racine
- Un arbre a une fonction parentale
- Un arbre est acyclique
- Un arbre est connexe à partir de la racine



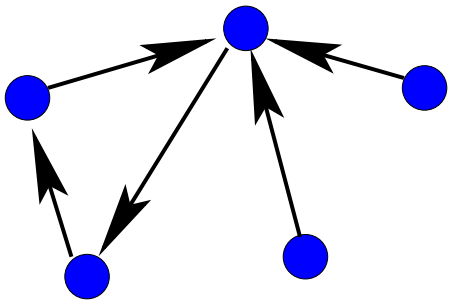
Un arbre t construit sur les noeuds



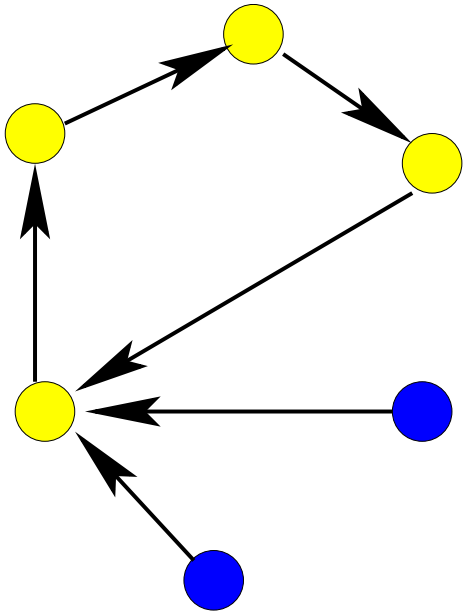
t est une fonction définie sur ND sauf pour la racine!



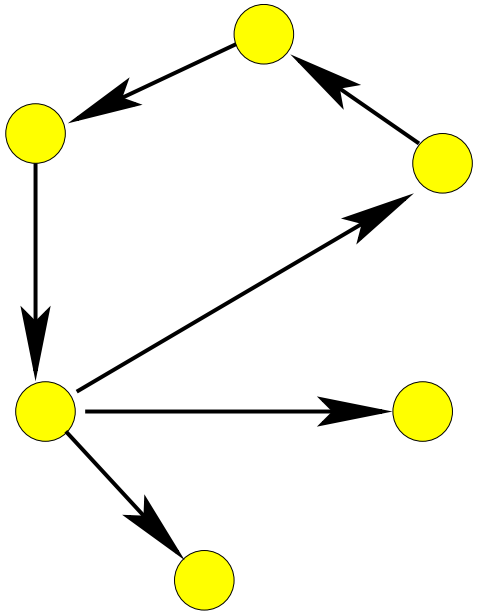
Eviter des cycles



Mauvais exemple!



Un cycle



Son image inverse

Les noeuds d'un cycle sont inclus
dans leur image inverse

Le prédicat tree (r, t)

Le prédicat tree (r, t)

r est un élément de ND $\quad r \in ND$

Le prédicat tree (r, t)

r est un élément de ND $\quad r \in ND$

t est une fonction $\quad t \in ND - \{r\} \rightarrow ND$

Le prédicat tree (r, t)

r est un élément de ND $\quad r \in ND$

t est une fonction $\quad t \in ND - \{r\} \rightarrow ND$

t est acyclique

$$\forall p \cdot \left(\begin{array}{l} p \subseteq ND \ \wedge \\ p \subseteq t^{-1}[p] \\ \Longrightarrow \\ p = \varnothing \end{array} \right)$$

t est acyclique : formulations équivalentes

$$\forall p \cdot \left(\begin{array}{l} p \subseteq ND \ \wedge \\ p \subseteq t^{-1}[p] \\ \Longrightarrow \\ p = \varnothing \end{array} \right) \quad \Leftrightarrow \quad \forall q \cdot \left(\begin{array}{l} q \subseteq ND \ \wedge \\ r \in q \ \wedge \\ t^{-1}[q] \subseteq q \\ \Longrightarrow \\ ND \subseteq q \end{array} \right.$$

On obtient une Règle d'Induction

$$\forall q \cdot \left(\begin{array}{l} q \subseteq ND \ \wedge \\ r \in q \ \wedge \\ \forall x \cdot (\, x \in ND - \{r\} \ \wedge \ t(x) \in q \implies x \in q \,) \\ \implies \\ ND \subseteq q \end{array} \right)$$

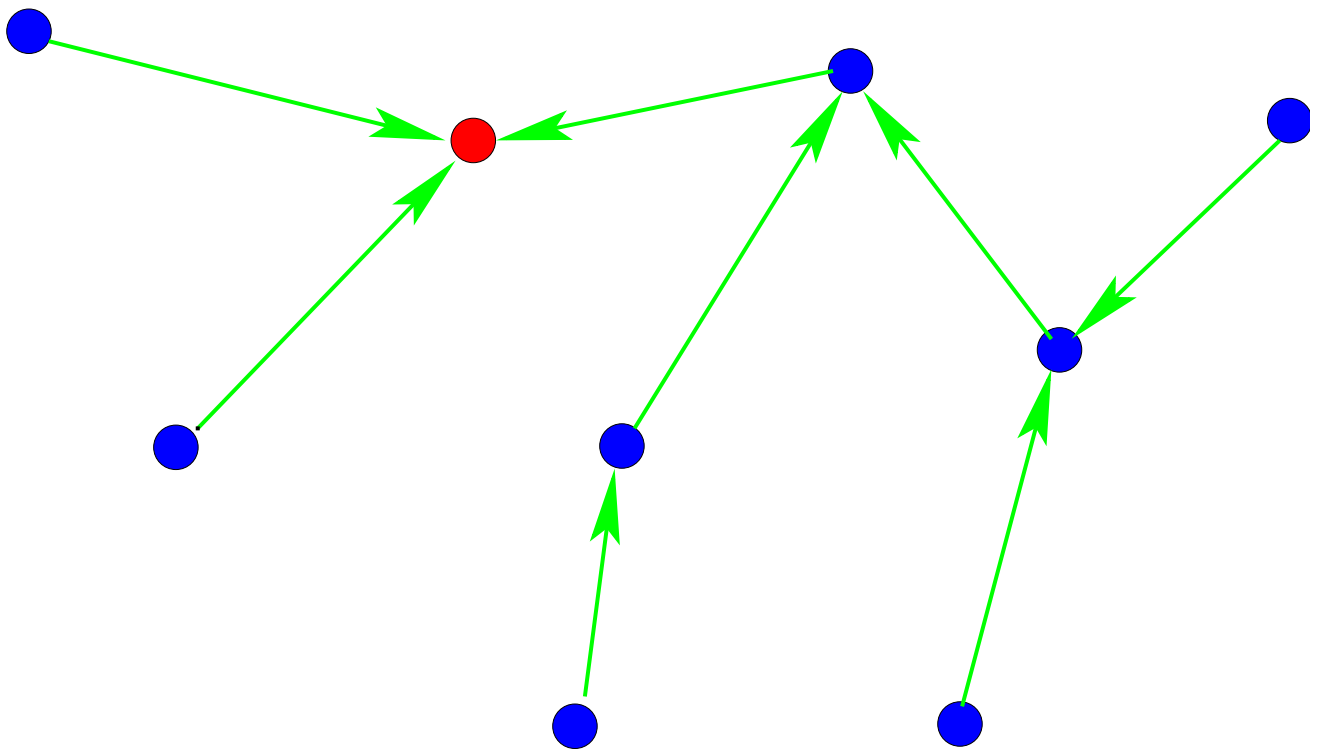
Le prédicat tree (r, t)

r est un élément de ND $\quad r \in ND$

t est une fonction $\quad t \in ND - \{r\} \rightarrow ND$

t est acyclique

$$\forall q \cdot \left(\begin{array}{l} q \subseteq ND \ \wedge \\ r \in q \ \wedge \\ t^{-1}[q] \subseteq q \\ \Longrightarrow \\ ND \subseteq q \end{array} \right)$$



Un arbre de recouvrement t de gr

Le prédicat spanning (r, t, gr)

r, t est un arbre	tree (r, t)
t est inclus dans gr	$t \subseteq gr$

Le graphe gr est connexe et acyclique (1)

- Définir une relation fn liant un noeud aux possibles spanning trees de gr ayant ce noeud comme racine :

$$fn \subseteq ND \times (ND \nrightarrow ND)$$

$$\forall (r,t) \cdot \left(\begin{array}{l} r \in ND \ \wedge \\ t \in ND \nrightarrow ND \\ \Longrightarrow \\ (r,t) \in fn \ \Leftrightarrow \ \text{spanning}\,(r,t,gr) \end{array} \right)$$

Le graphe gr est connexe et acyclique (2)

Totalité de la relation $fn \implies$ Connectivité of gr

Fonctionnalité d'une relation $fn \implies$ Acyclicité de gr

Point sur les constantes gr and fn

$$gr \subseteq ND \times ND$$
$$\mathsf{dom}\,(gr) = ND$$
$$gr = gr^{-1}$$
$$\mathsf{id}\,(ND) \;\cap\; gr \;=\; \varnothing$$

$$fn \in ND \rightarrow (ND \nrightarrow ND)$$
$$\forall (r,t) \cdot \left(\begin{array}{l} r \in ND \;\wedge \\ t \in ND \nrightarrow ND \\ \Longrightarrow \\ t = fn(r) \;\Leftrightarrow\; \mathsf{spanning}\,(r,t,gr) \end{array} \right)$$

Tree

- Variables rt et ts

$$rt \in ND$$
$$ts \in ND \leftrightarrow ND$$

$$\text{EVENT elect} \quad \widehat{=}$$
BEGIN
$$rt, ts : \text{spanning}\,(rt, ts, gr)$$
END

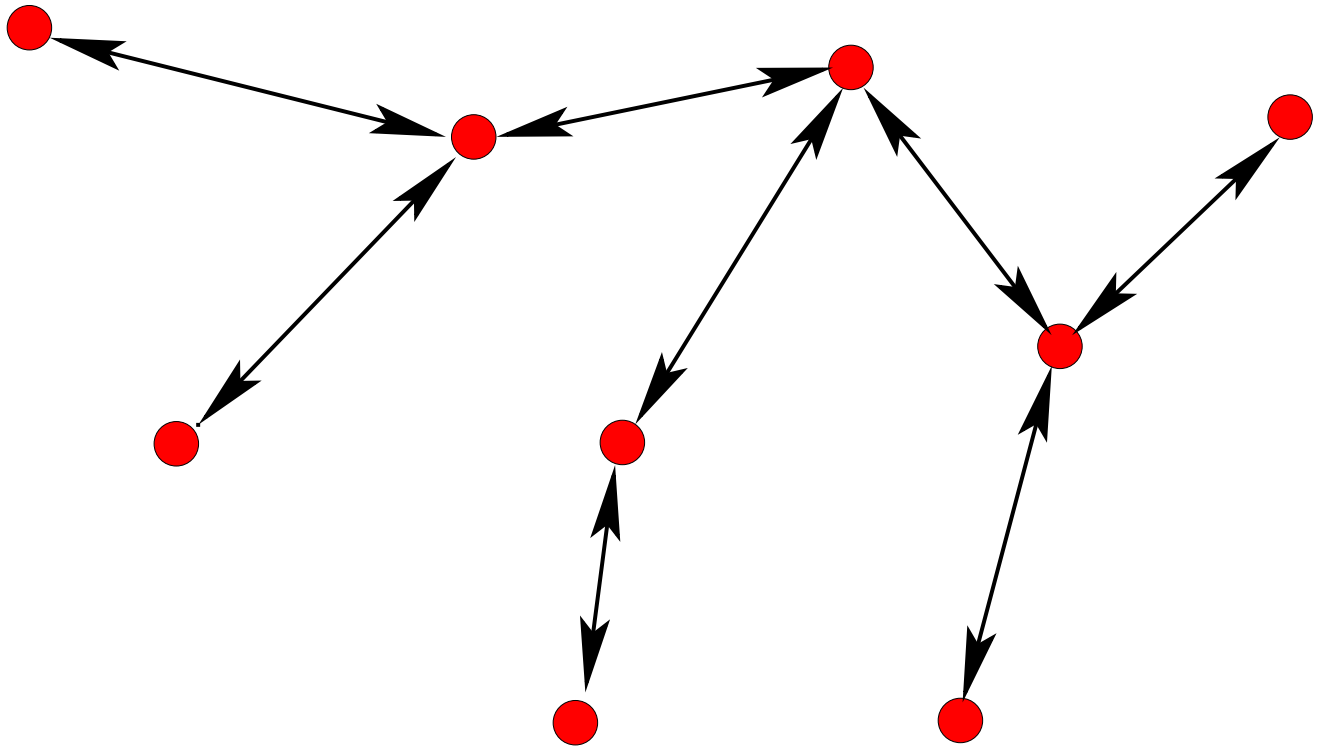
Premier raffinement (1)

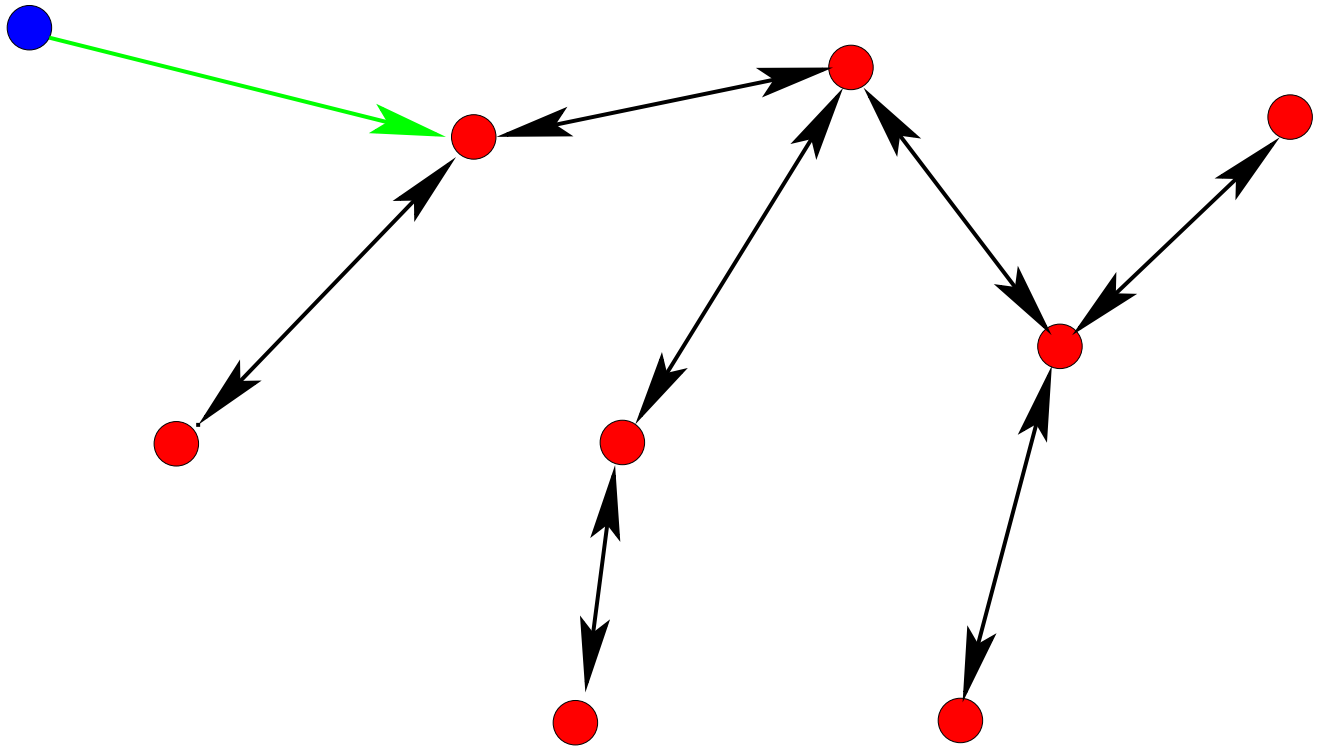
- Introduire une nouvelle variable, tr, correspondant à l'
  "arbre" en construction

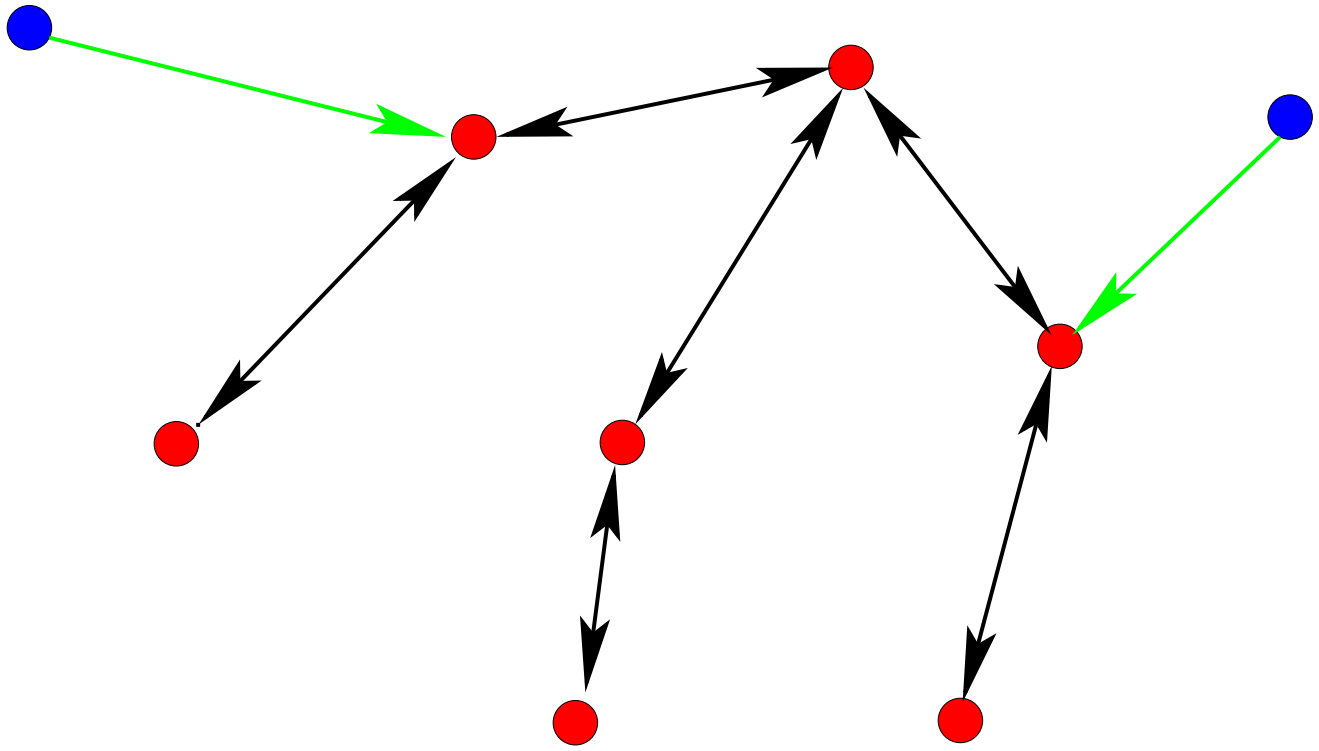
- Introduire un nouvel événement : progression

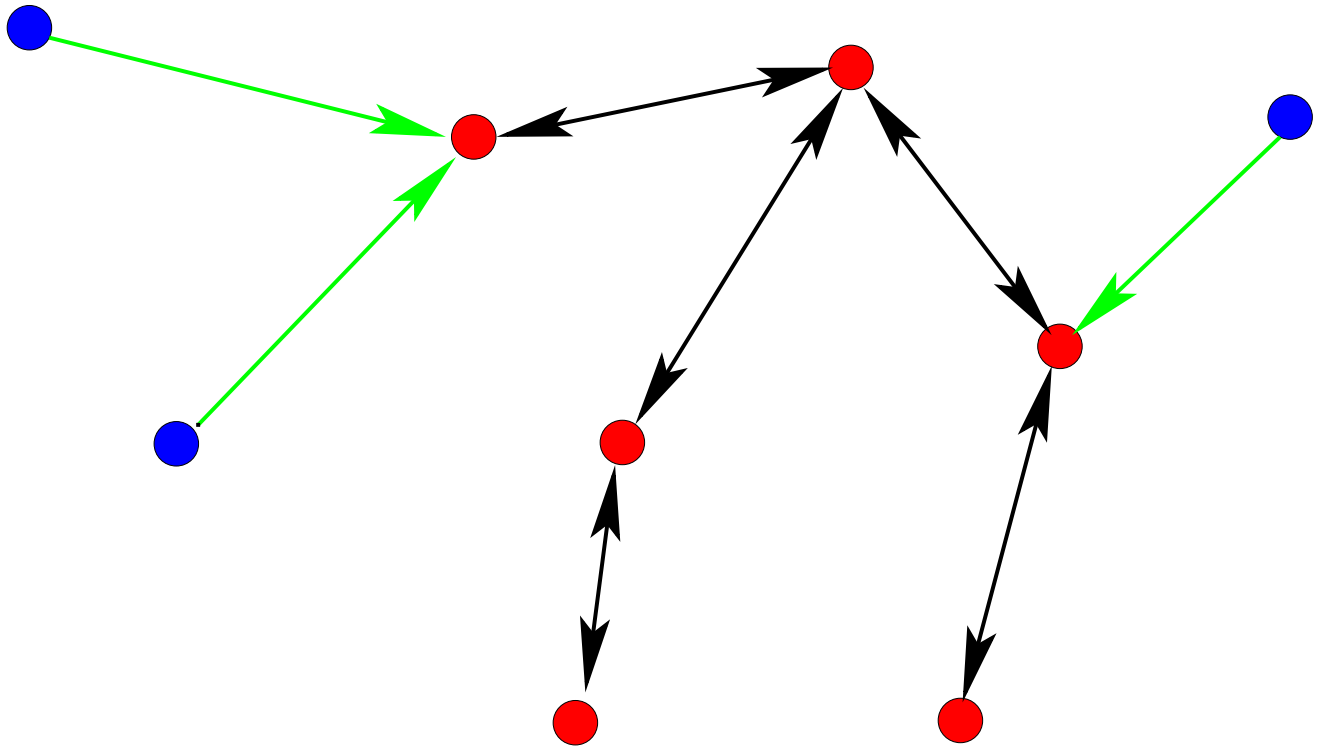
- Définir l'invariant

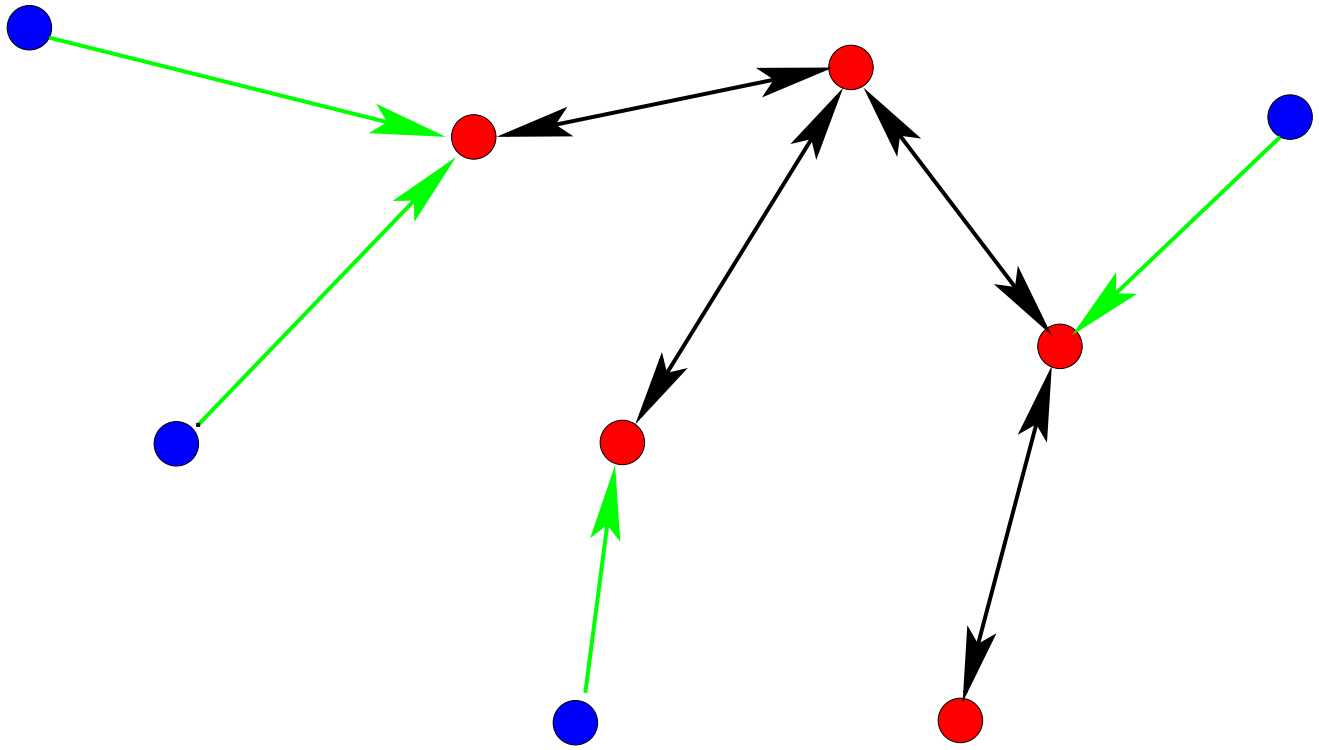
- Retour à l'animation : Observez la construction de l'arbre

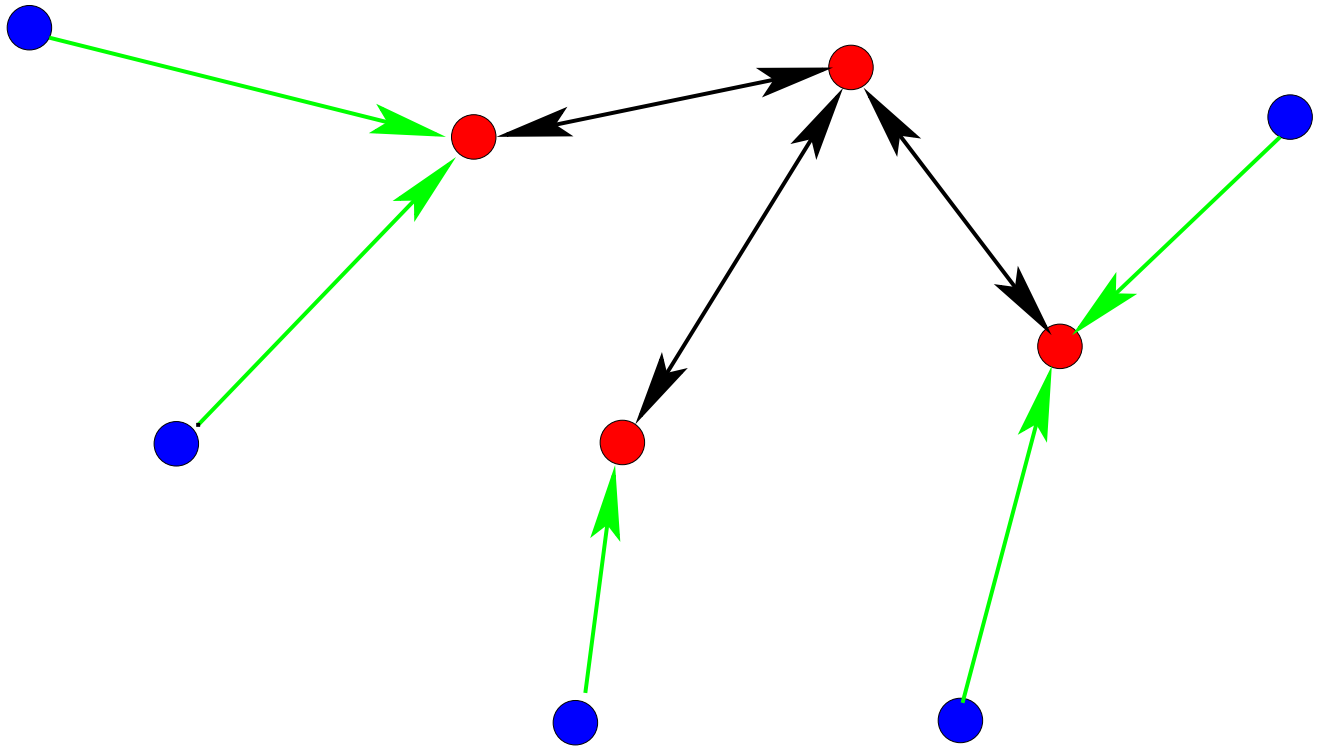


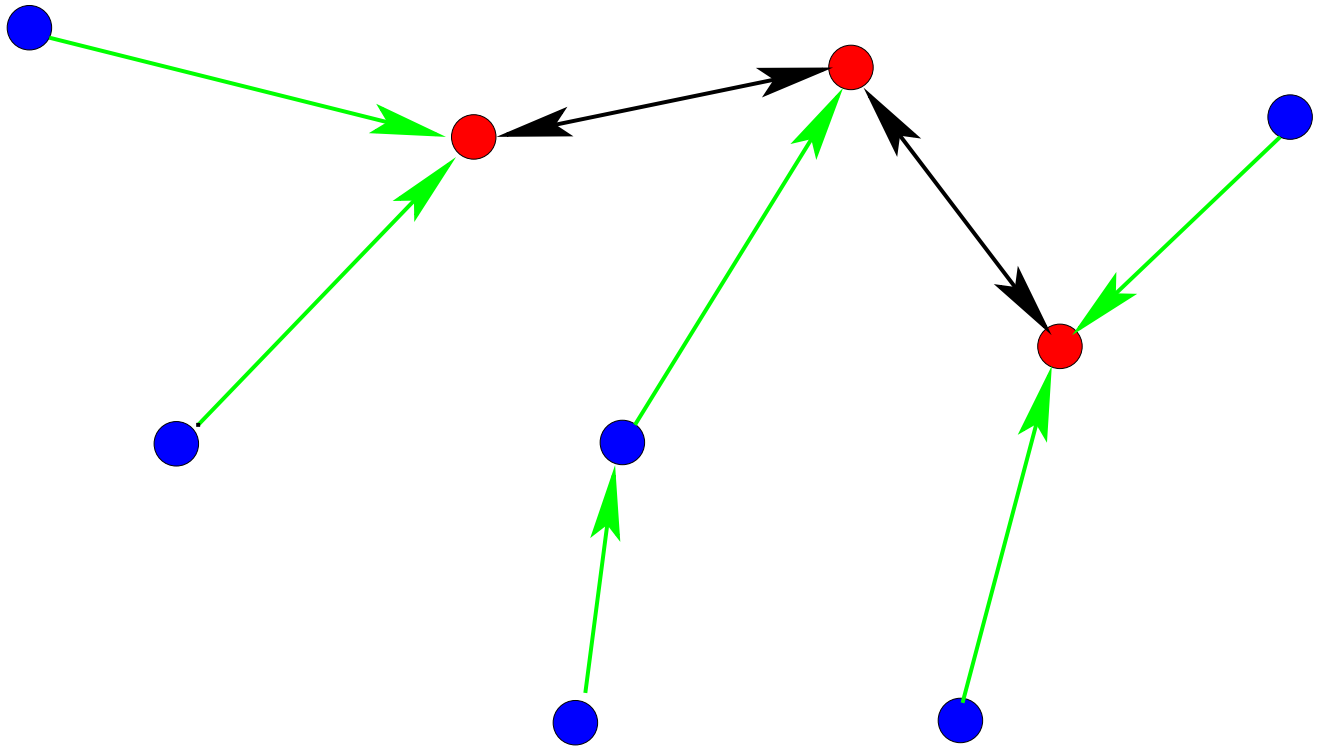


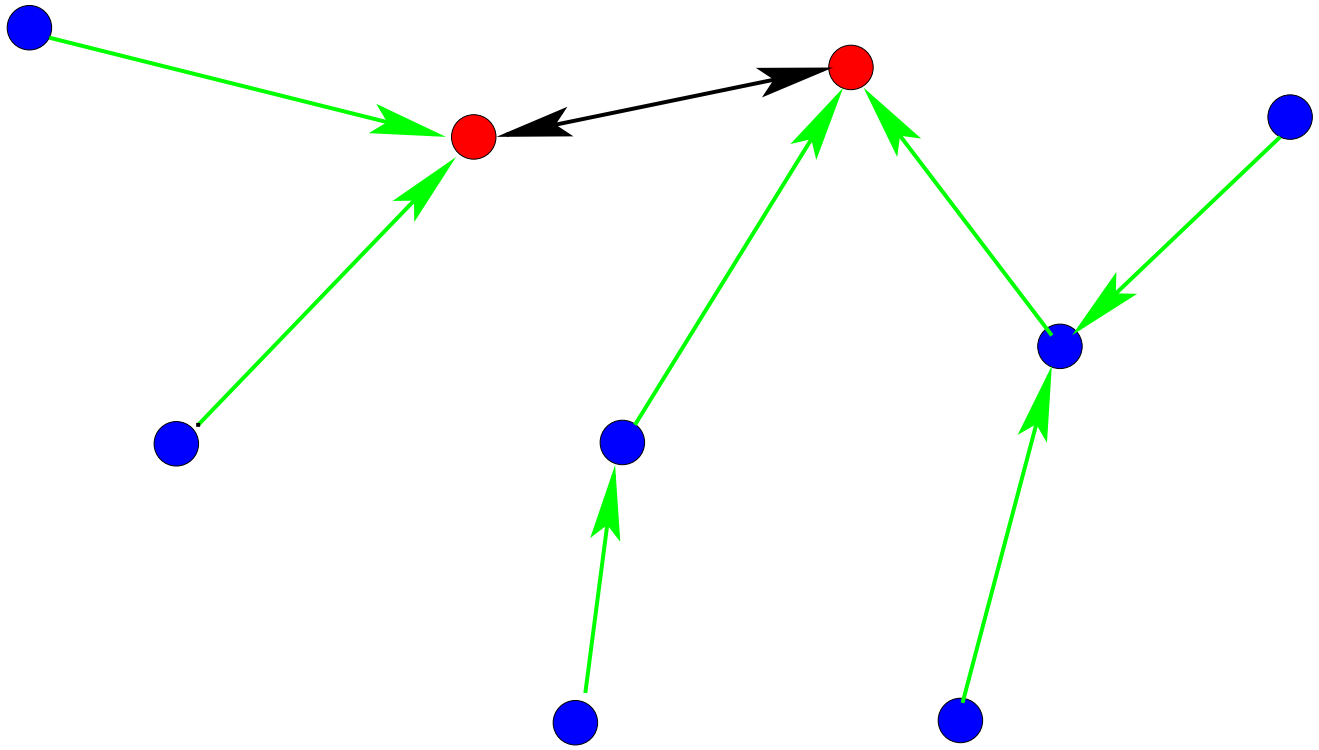


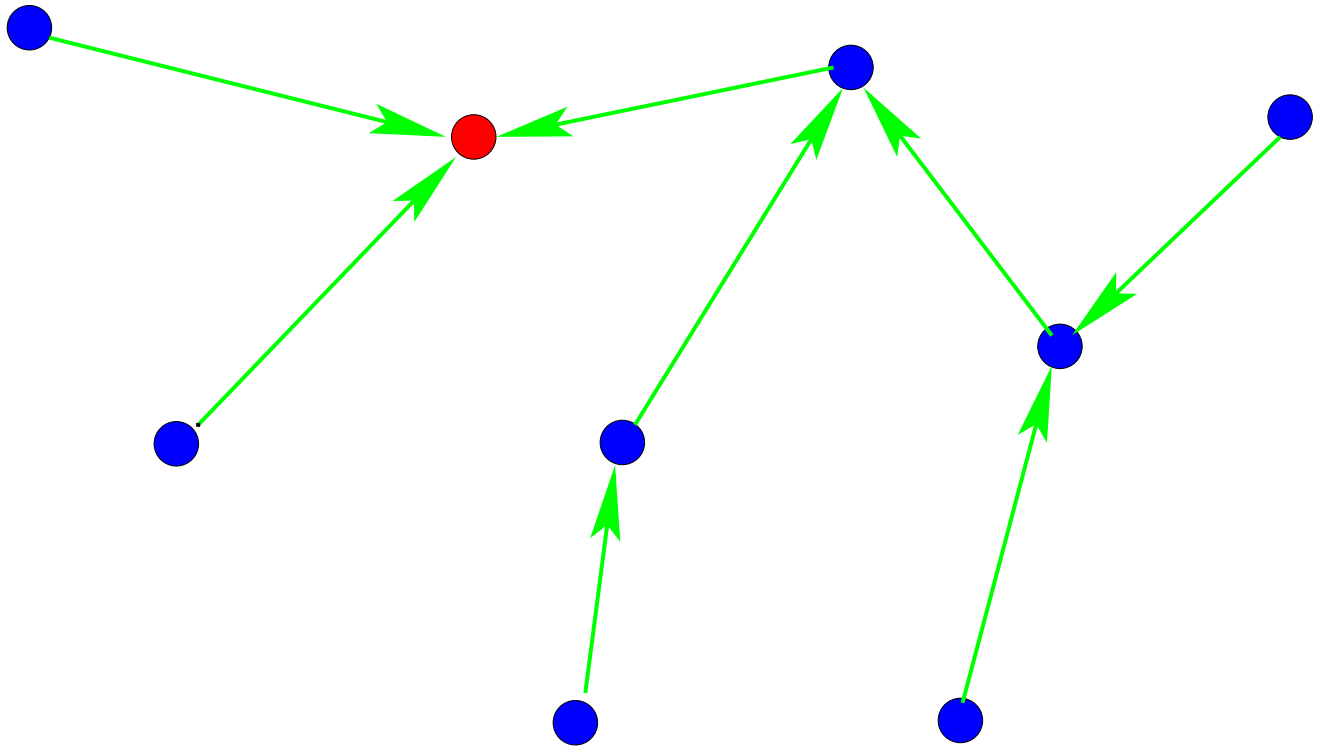












- Les flèches vertes correspondent à la fonction tr

- Les nœuds bleus sont le domaine de tr

- La fonction tr est une forêt sur les nœuds

- Les nœuds rouges sont les racines de ce arbres

Le prédicat invariant (tr)

$$tr \in ND \nrightarrow ND$$

Le prédicat invariant (tr)

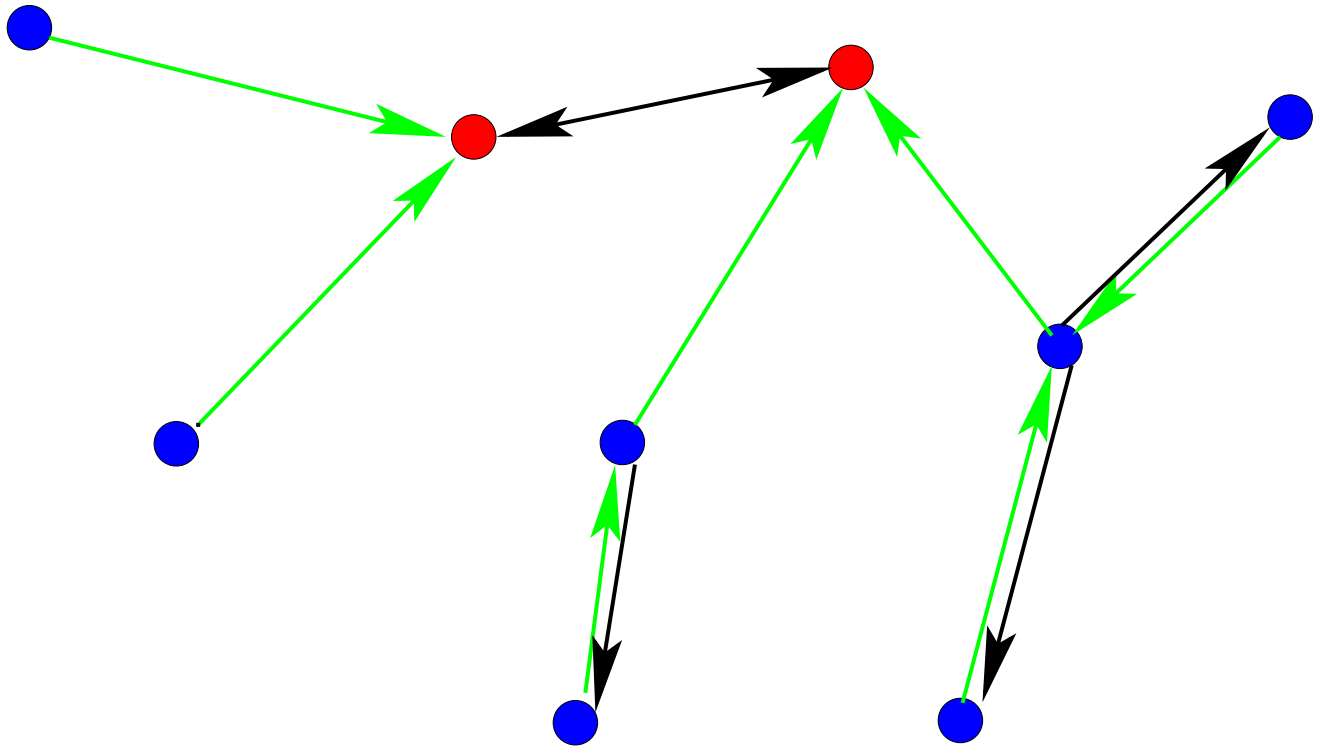
$$tr \in ND \nrightarrow ND$$

$$\forall p \cdot \left(\begin{array}{l} p \subseteq ND \quad \wedge \\ ND - \mathsf{dom}\,(tr) \subseteq p \quad \wedge \\ tr^{-1}\,[p] \subseteq p \\ \Longrightarrow \\ ND \subseteq p \end{array} \right)$$

Le prédicat invariant (tr)

$$tr \in ND \nrightarrow ND$$

$$\text{dom}\,(tr) \lhd (tr \cup tr^{-1}) = \text{dom}\,(tr) \lhd gr$$

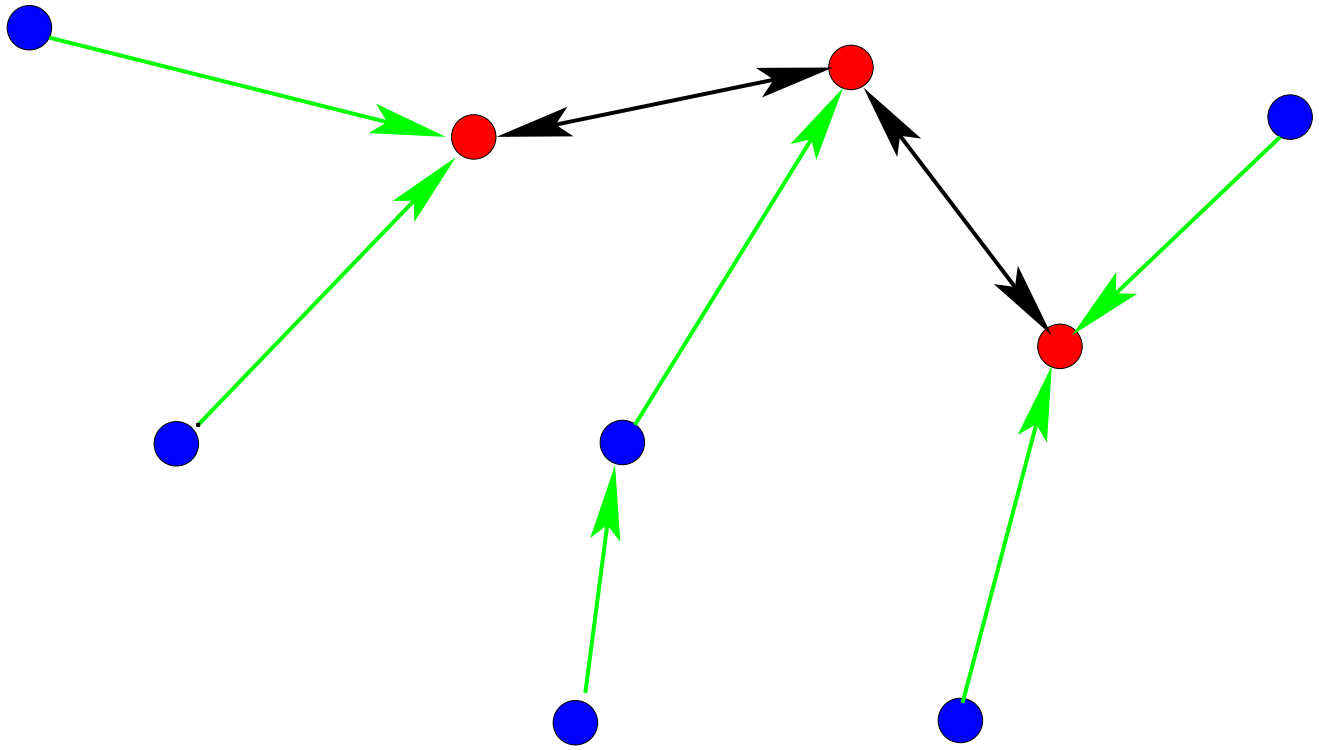


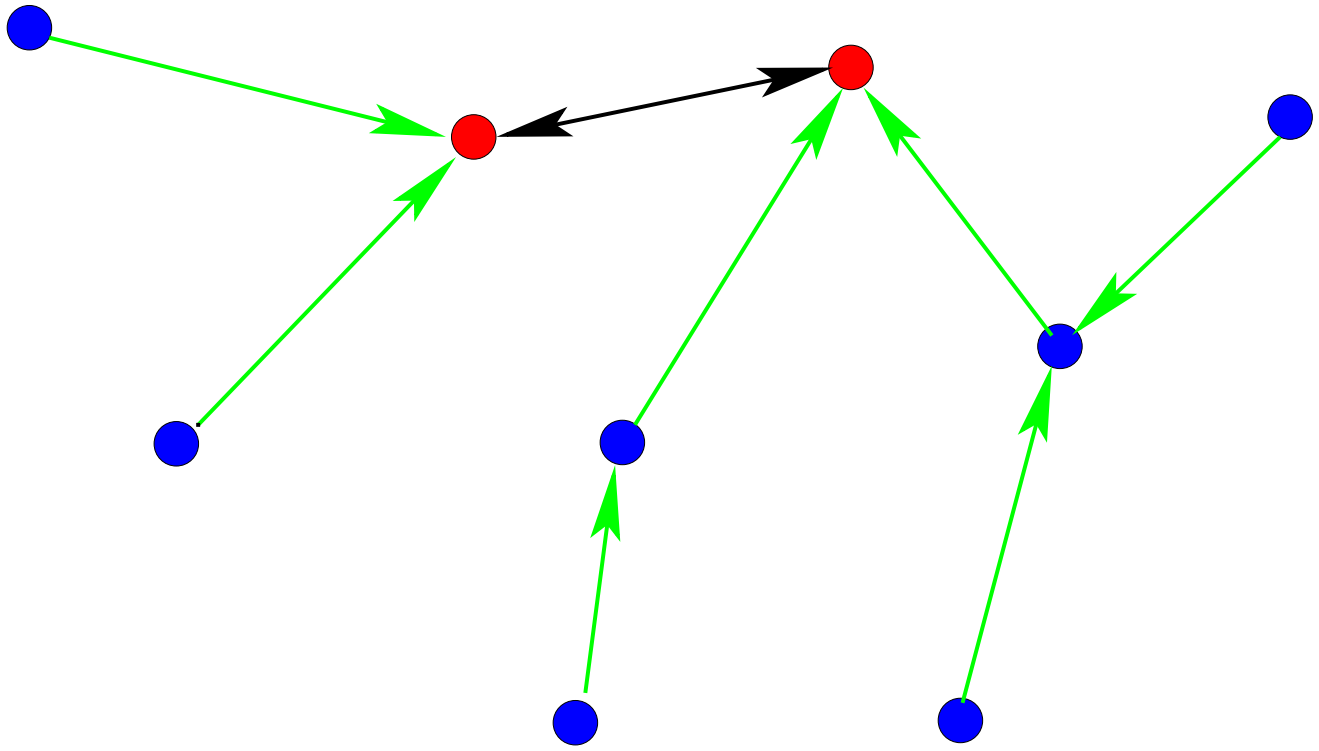
Premier raffinement (2)

- Introduire le nouvel événement "progress"

- Raffiner l'événement abstrait "elect"

- Retour à l'animation : Observez la "garde" de progress



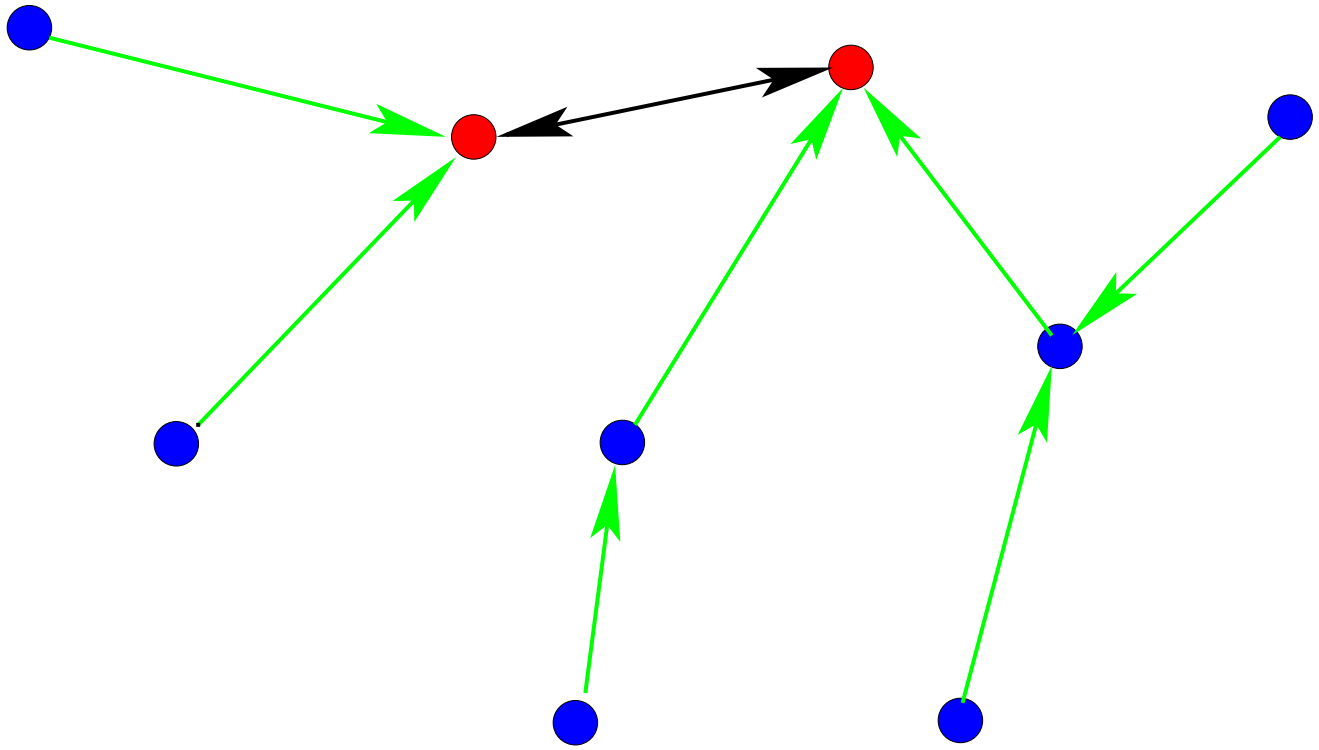


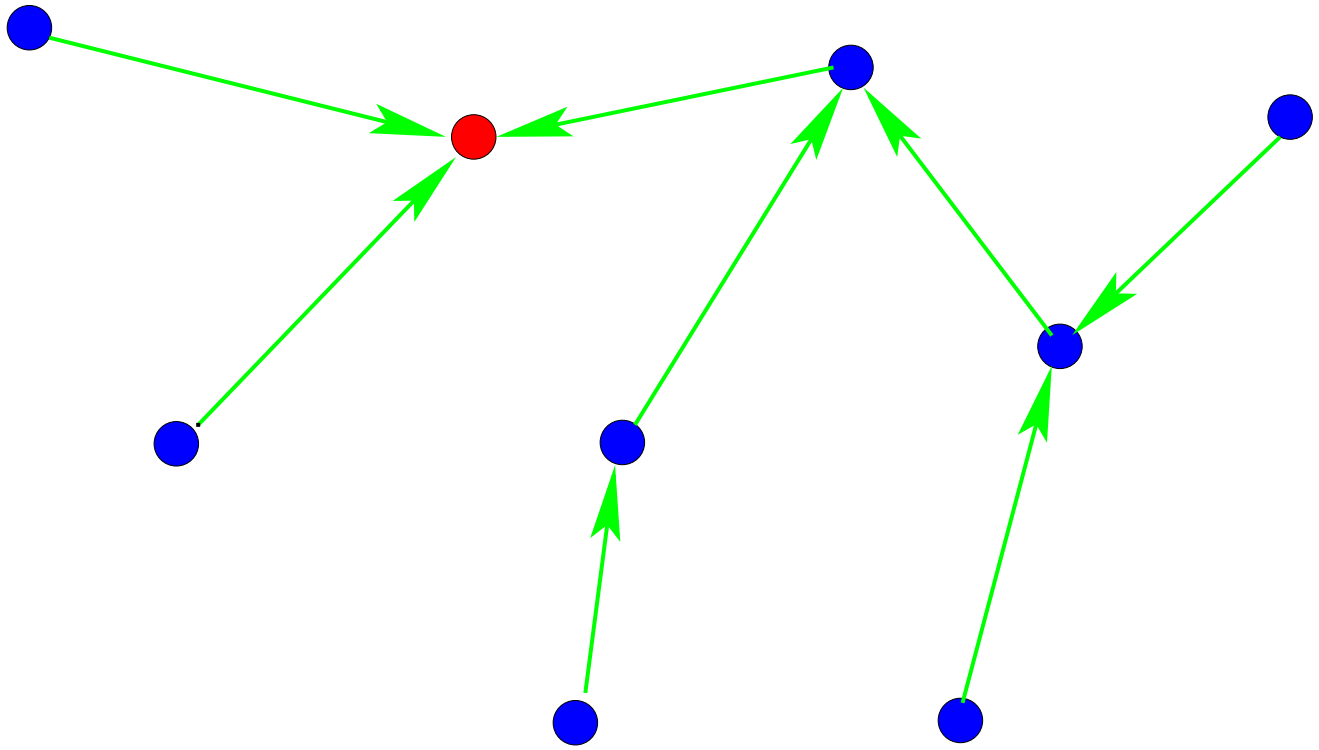
Quand un nœud rouge x est connecté à un autre nœud rouge y alors l'événement "progress" peut apparaître

$$
\begin{array}{l}
\text{EVENT progress} \;\widehat{=} \\
\quad \textbf{ANY } x, y \textbf{ WHERE} \\
\qquad x, y \in gr \;\wedge \\
\qquad x \notin \mathsf{dom}\,(tr) \;\wedge \\
\qquad y \notin \mathsf{dom}\,(tr) \;\wedge \\
\qquad gr[\{x\}] = tr^{-1}[\{x\}] \cup \{y\} \\
\quad \textbf{THEN} \\
\qquad tr := tr \cup \{x \mapsto y\} \\
\quad \textbf{END}
\end{array}
$$

Il faut prouver :

$$
\begin{array}{l}
\mathsf{invariant}(tr) \quad \wedge \\
x, y \in gr \quad \wedge \\
x \notin tr \quad \wedge \\
y \notin tr \quad \wedge \\
gr[\{x\}] = tr^{-1}[\{x\}] \cup \{y\} \\
\Longrightarrow \\
\mathsf{invariant}(tr \cup \{x \mapsto y\})
\end{array}
$$





Quand un nœud rouge x est SEULEMENT connecté aux nœuds bleus alors l'événement "elect" peut apparaître :

$$
\begin{array}{l}
\mathrm{EVENT}\ \text{elect} \ \widehat{=} \\
\quad \textbf{ANY}\ x\ \textbf{WHERE} \\
\qquad x \in ND \ \wedge \\
\qquad gr[\{x\}] = tr^{-1}[\{x\}] \\
\quad \textbf{THEN} \\
\qquad rt, ts := x, tr \\
\quad \textbf{END}
\end{array}
$$

$$\text{EVENT elect} \quad \widehat{=}$$

BEGIN

$$rt, ts : \text{spanning}\,(rt, ts, gr)$$

END

$$\text{EVENT elect} \quad \widehat{=}$$

ANY x **WHERE**

$$x \in ND \ \wedge$$

$$gr[\{x\}] = tr^{-1}[\{x\}]$$

THEN

$$rt, ts := x, tr$$

END

Il faut prouver :

$$\begin{array}{l} \mathsf{invariant}(tr) \quad \wedge \\ x \in ND \quad \wedge \\ gr[\{x\}] = tr^{-1}[\{x\}] \\ ts = tr \\ \Longrightarrow \\ \mathsf{spanning}(x, ts, gr) \end{array}$$

Un lemme

$$\begin{array}{l} \mathsf{invariant}(tr) \quad \wedge \\ x \in ND \quad \wedge \\ gr[\{x\}] = tr^{-1}[\{x\}] \\ \Longrightarrow \\ tr = fn(x) \end{array}$$

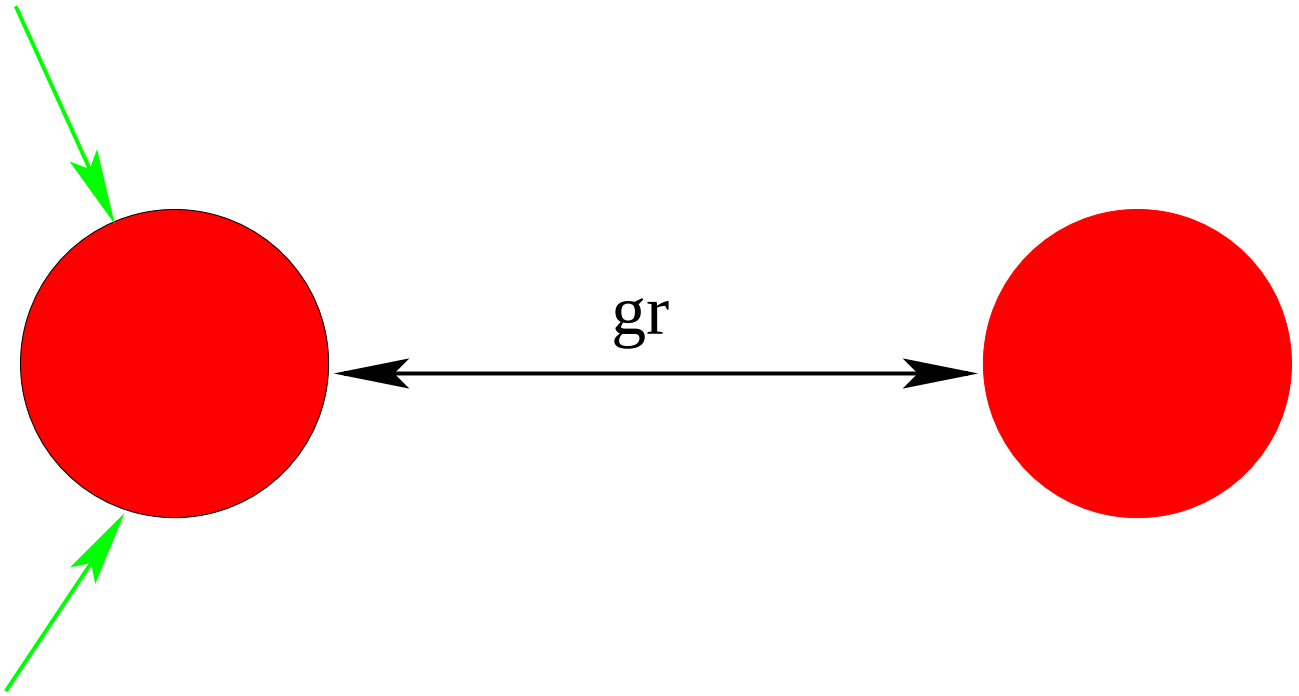
Récapitulatif du premier raffinement

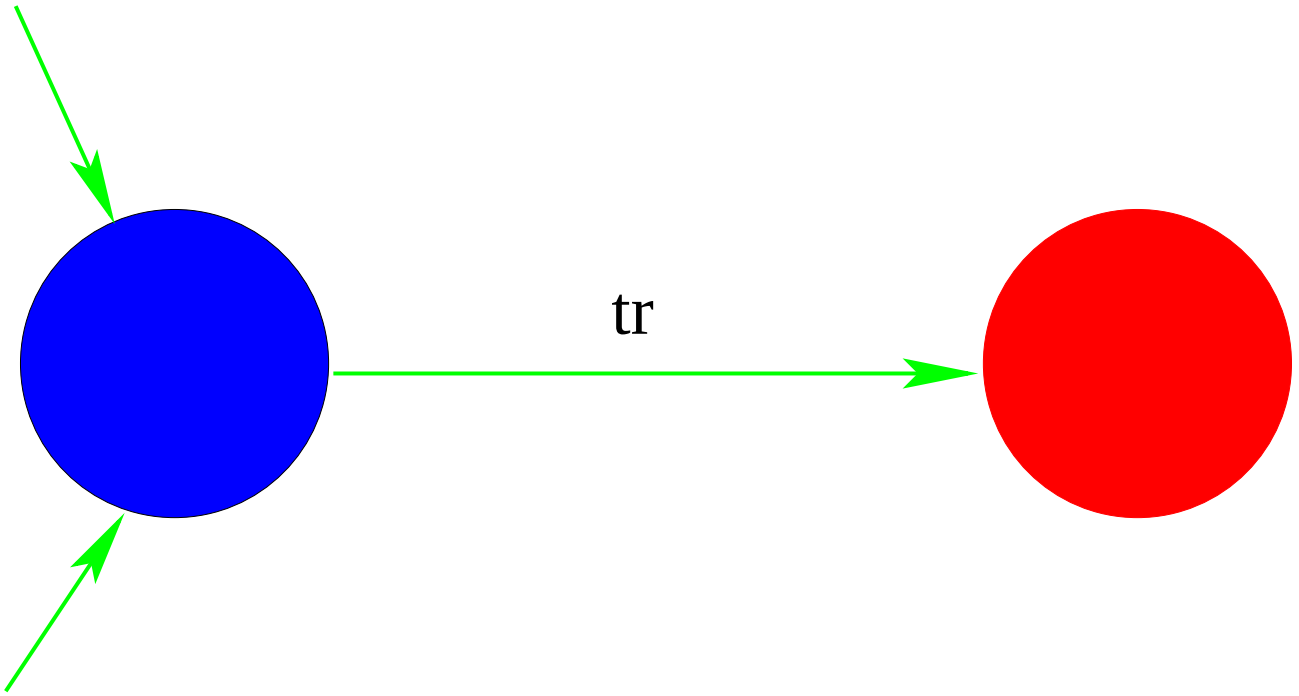
- 12 preuves

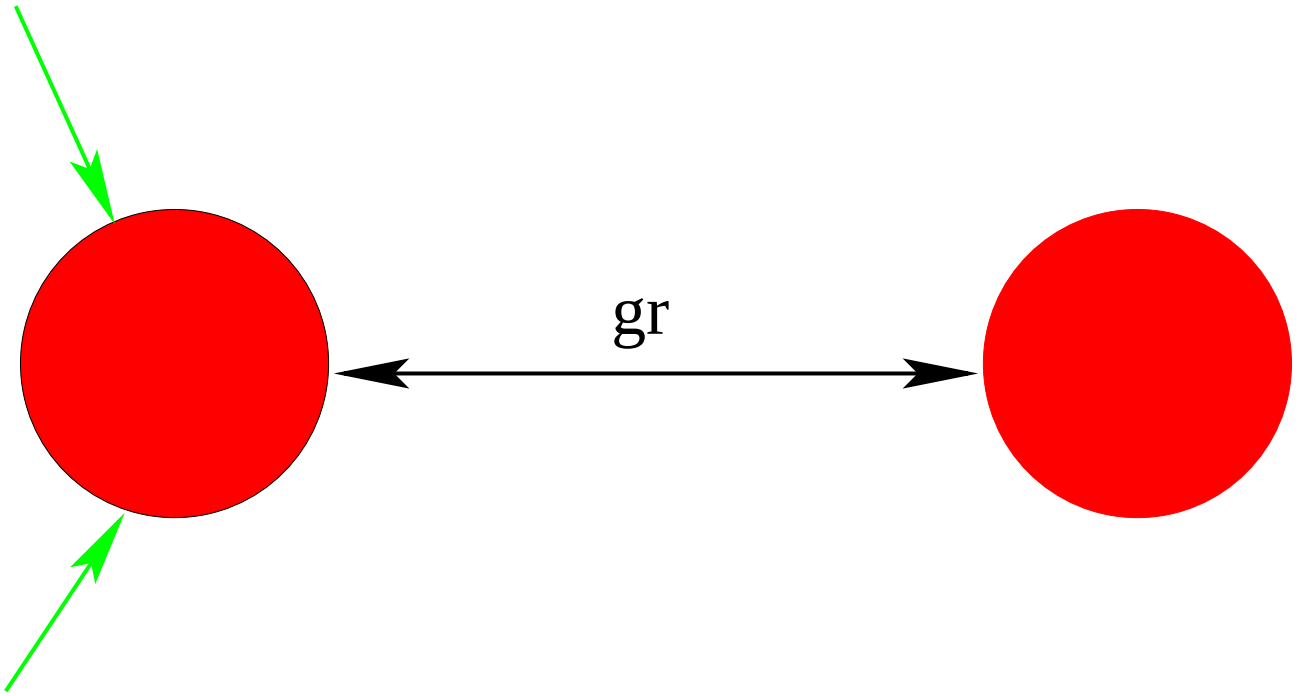
- Parmi lesquelles 5 interactives (une assez difficile !)

Second Raffinement

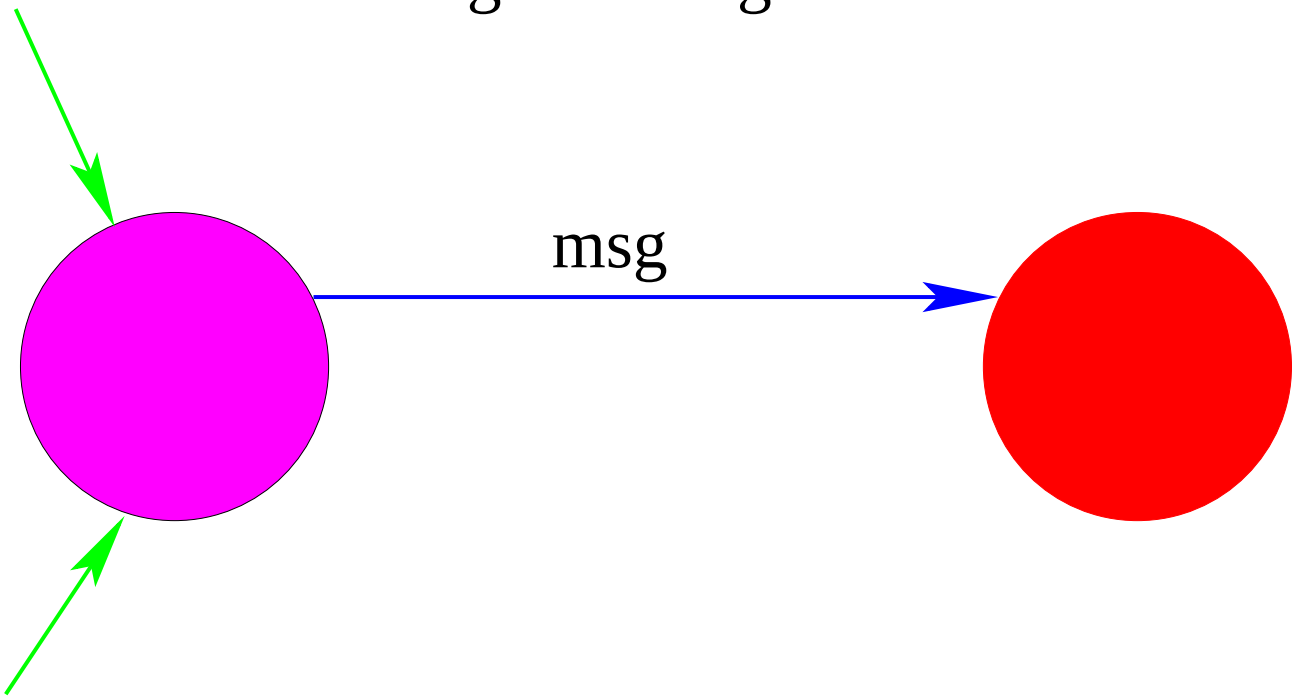
- Les nœuds communiquent avec leurs voisins
- On utilise des messages
- Les messages ont des accusés de réception
- Les accusés de réception sont confirmés
- Ensuite une animation locale


gr


tr

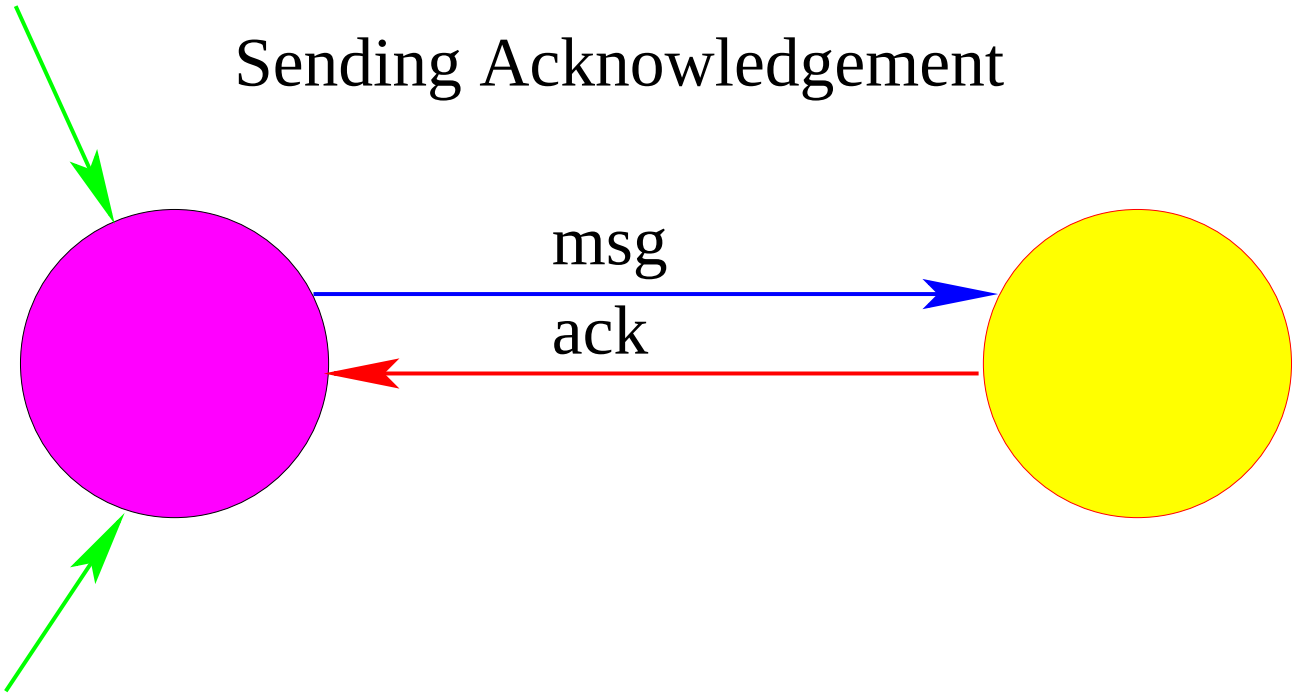

gr

Sending a message



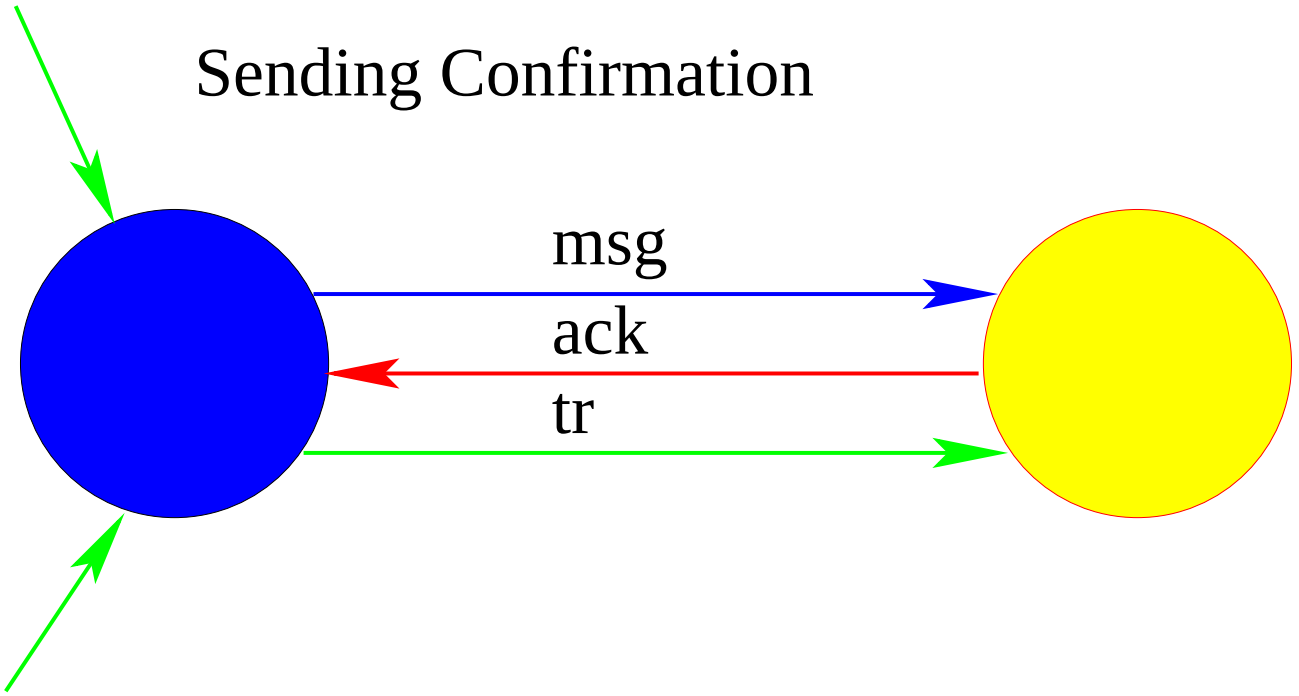
Receiving a message

Sending Acknowledgement

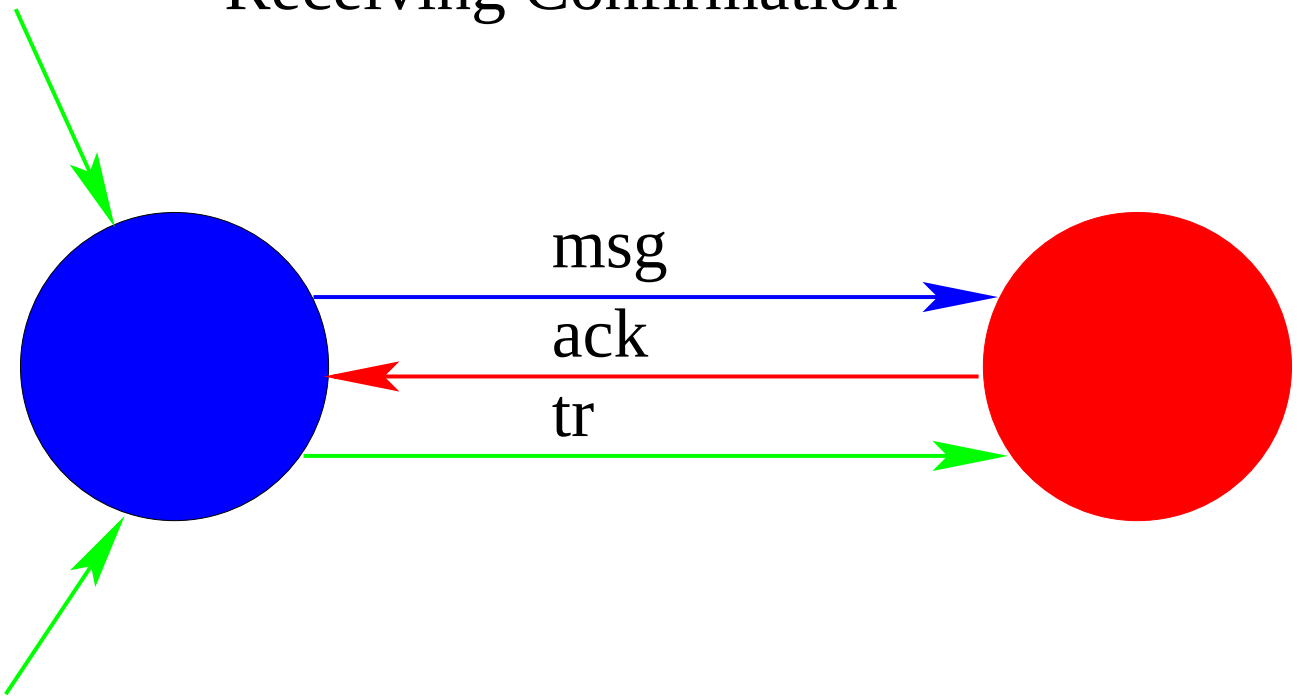


Receiving Acknowledgement

Sending Confirmation



Receiving Confirmation



Invariant (1)

$$msg \in ND \nrightarrow ND$$

$$ack \in ND \nrightarrow ND$$

$$tr \subseteq ack \subseteq msg \subseteq gr$$

Nœud x envoie un message au nœud y

$$
\begin{array}{l}
\text{EVENT send_msg} \;\; \widehat{=} \\
\quad \textbf{ANY } x, y \textbf{ WHERE} \\
\qquad x, y \in gr \;\; \wedge \\
\qquad gr[\{x\}] = tr^{-1}[\{x\}] \cup \{y\} \;\; \wedge \\
\qquad y, x \notin ack \;\; \wedge \\
\qquad x \notin \mathsf{dom}\,(msg) \\
\quad \textbf{THEN} \\
\qquad msg := msg \cup \{x \mapsto y\} \\
\quad \textbf{END}
\end{array}
$$

Nœud y envoie un acknowledgement au nœud x

$$
\begin{array}{l}
\text{EVENT send_ack} \;\widehat{=}\\
\quad \textbf{ANY } x, y \textbf{ WHERE}\\
\qquad x, y \in msg{-}ack \;\wedge\\
\qquad y \notin \mathsf{dom}\,(msg)\\
\quad \textbf{THEN}\\
\qquad ack := ack \cup \{x \mapsto y\}\\
\quad \textbf{END}
\end{array}
$$

Nœud x envoie une confirmation au nœud y

$$
\begin{array}{l}
\text{EVENT progress} \;\widehat{=} \\
\quad \textbf{ANY } x, y \textbf{ WHERE} \\
\qquad x, y \in ack \;\wedge \\
\qquad x \notin \mathsf{dom}\,(tr) \\
\quad \textbf{THEN} \\
\qquad tr := tr \cup \{x \mapsto y\} \\
\quad \textbf{END}
\end{array}
$$

Invariant (2)

$$\forall\,(x,y)\cdot\left(\begin{array}{l} x,y\in msg{-}ack \\ \Longrightarrow \\ x,y\in gr \quad\wedge \\ x\notin \mathsf{dom}\,(tr)\ \wedge\ y\notin \mathsf{dom}\,(tr)\ \wedge \\ gr[\{x\}]\ =\ tr^{-1}[\{x\}]\cup\{y\} \end{array}\right)$$

$$\forall\,(x,y)\cdot\left(\begin{array}{l} x,y\in ack \quad\wedge \\ x\notin \mathsf{dom}\,(tr) \\ \Longrightarrow \\ x,y\in gr \quad\wedge \\ y\notin \mathsf{dom}\,(tr)\ \wedge \\ gr[\{x\}]\ =\ tr^{-1}[\{x\}]\cup\{y\} \end{array}\right)$$

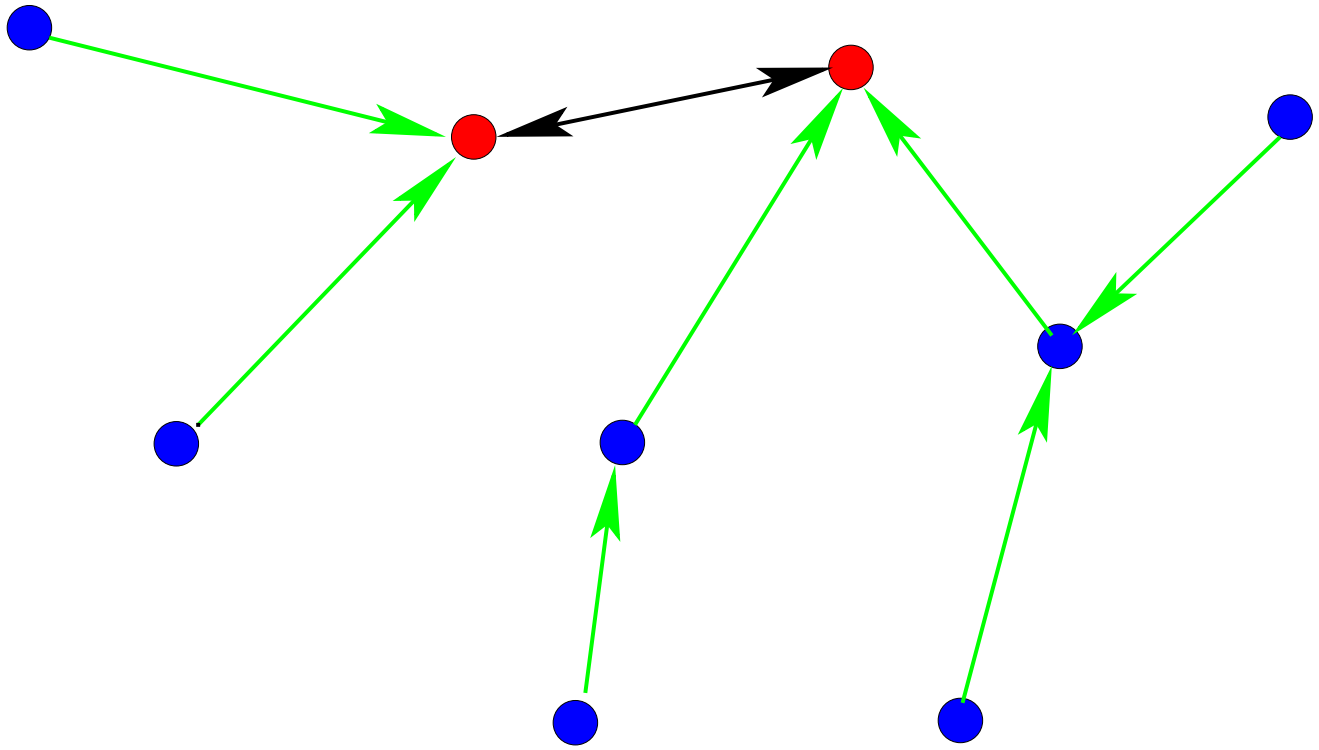
contention

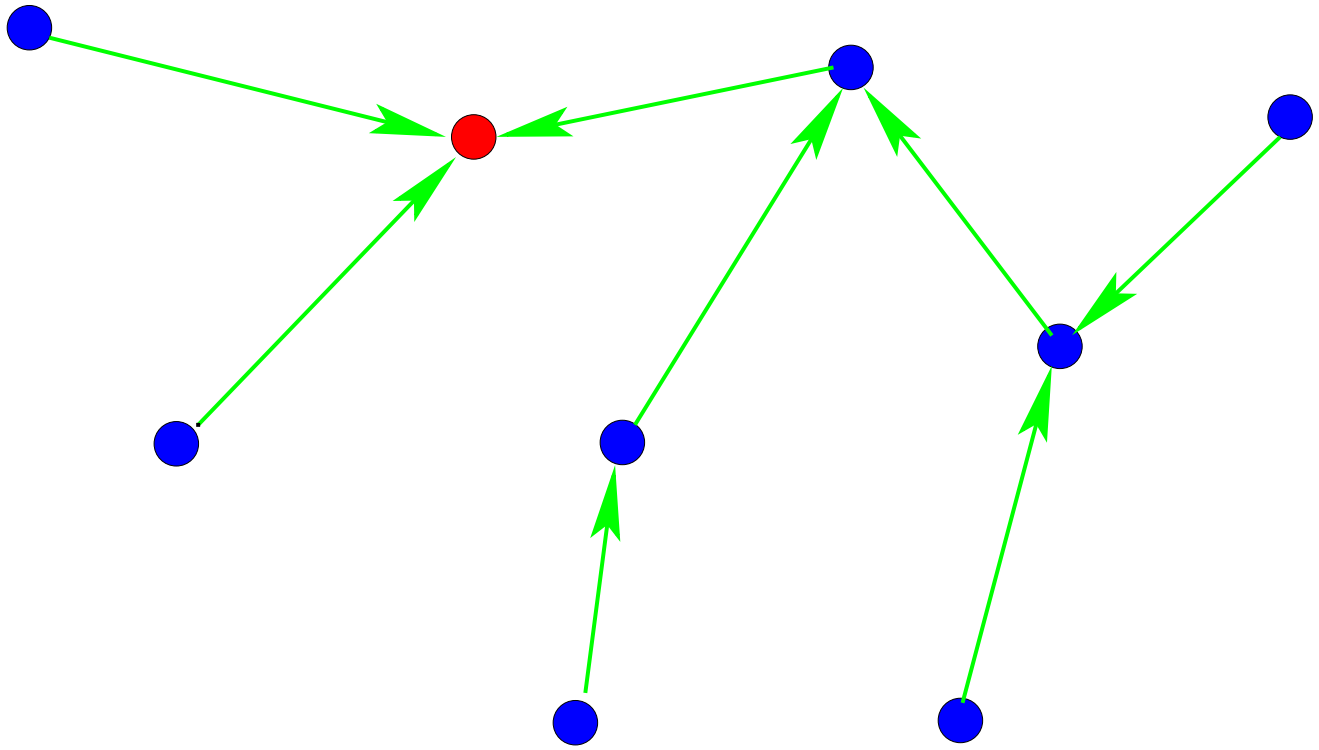
- Expliquer le problème

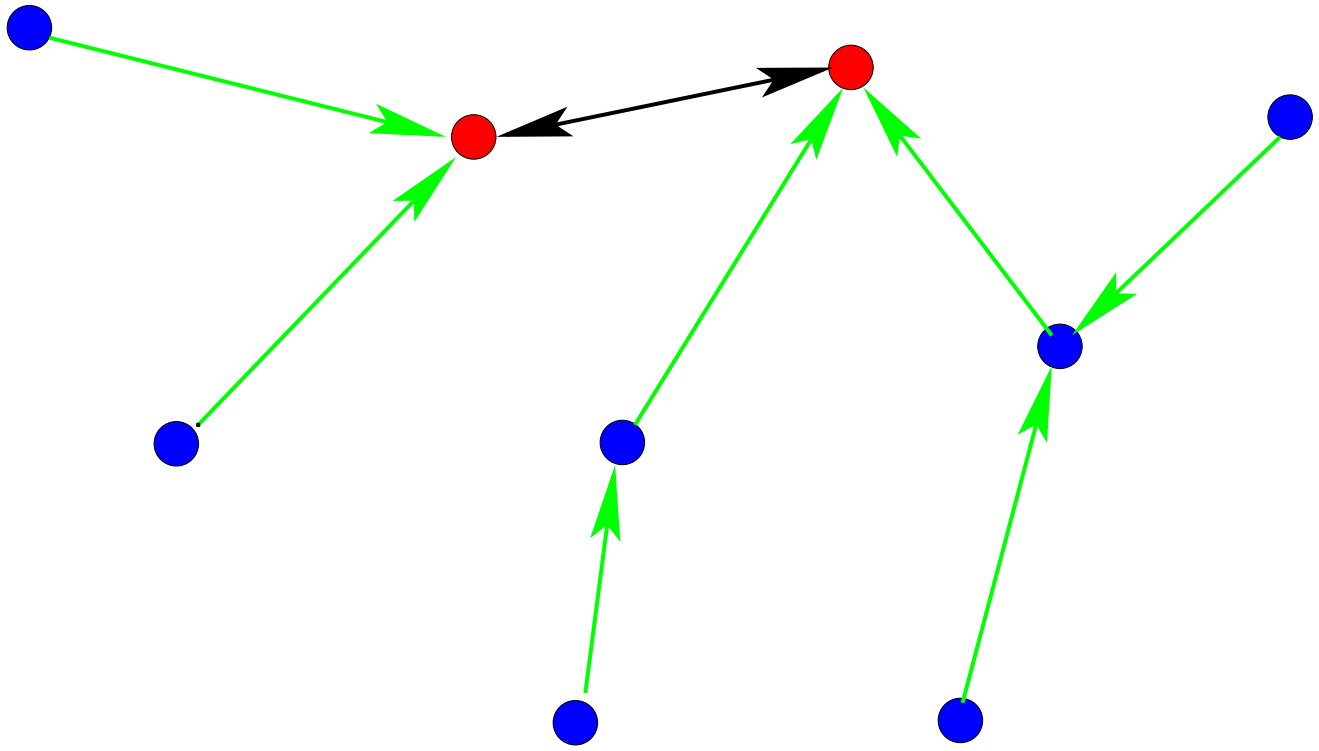
- Proposer une solution partielle.

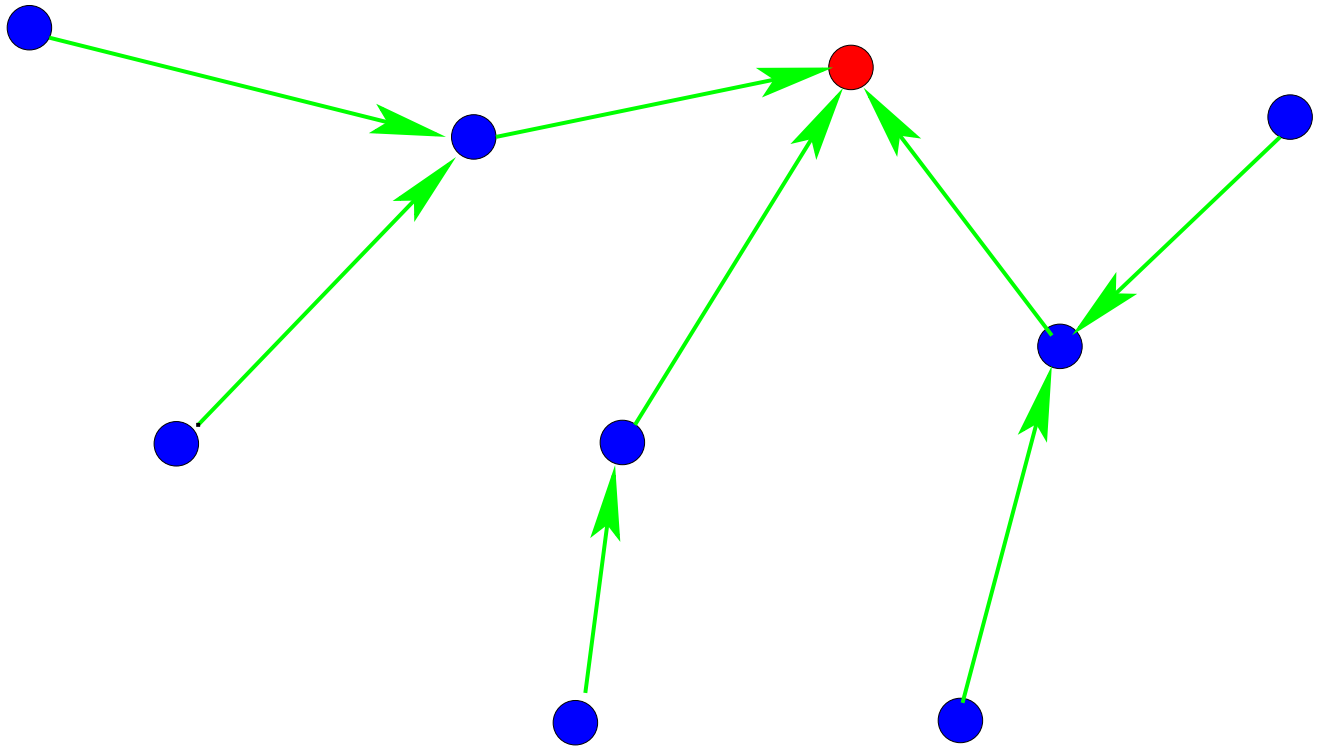
- Vers un meilleur traitement

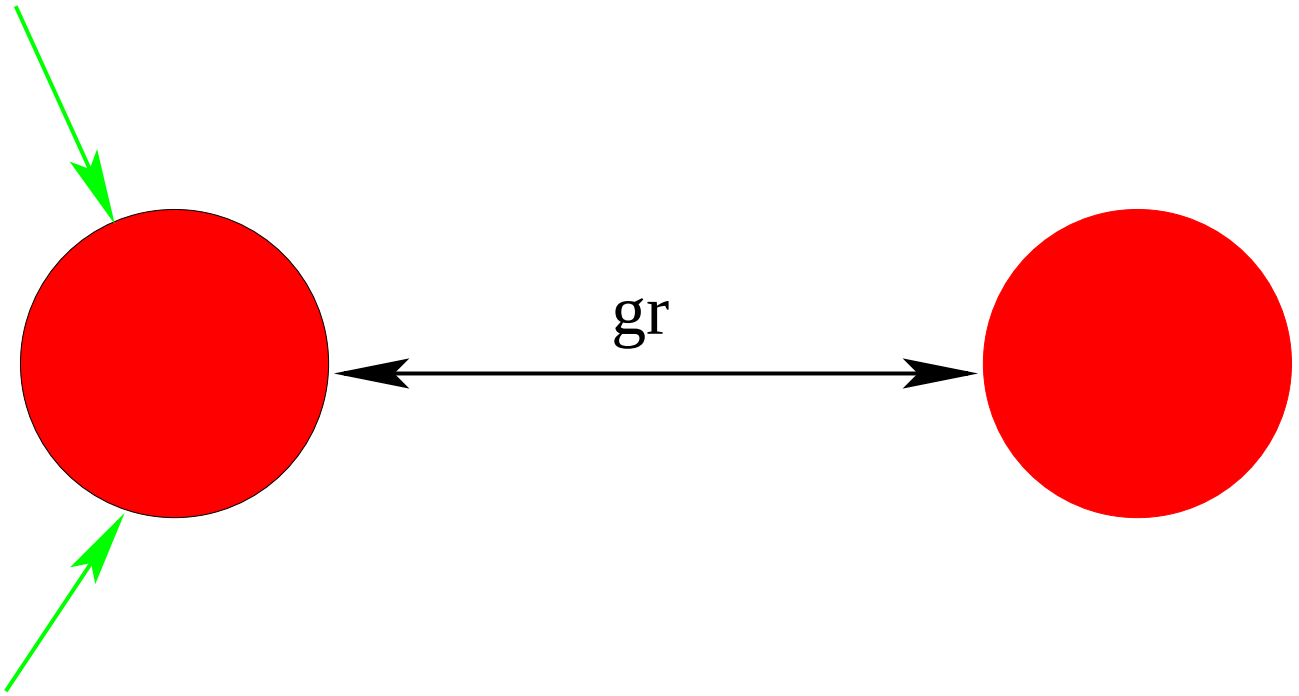
- Retour à l'animation locale.



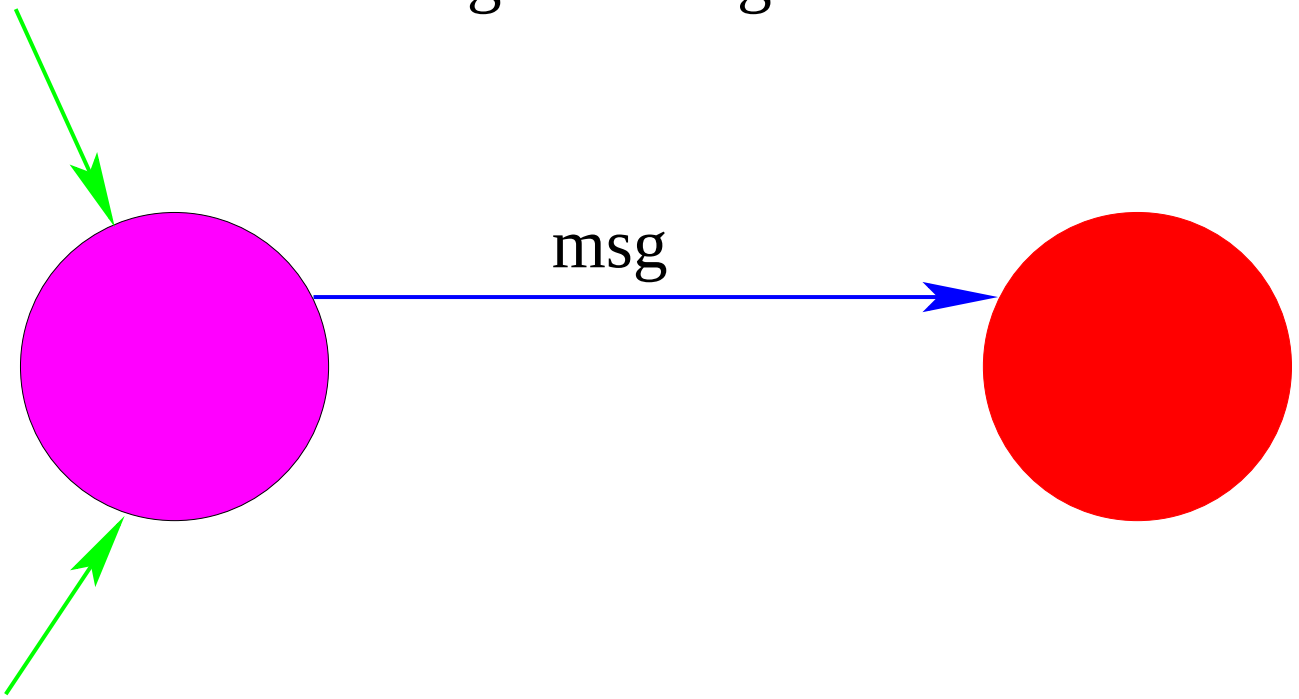




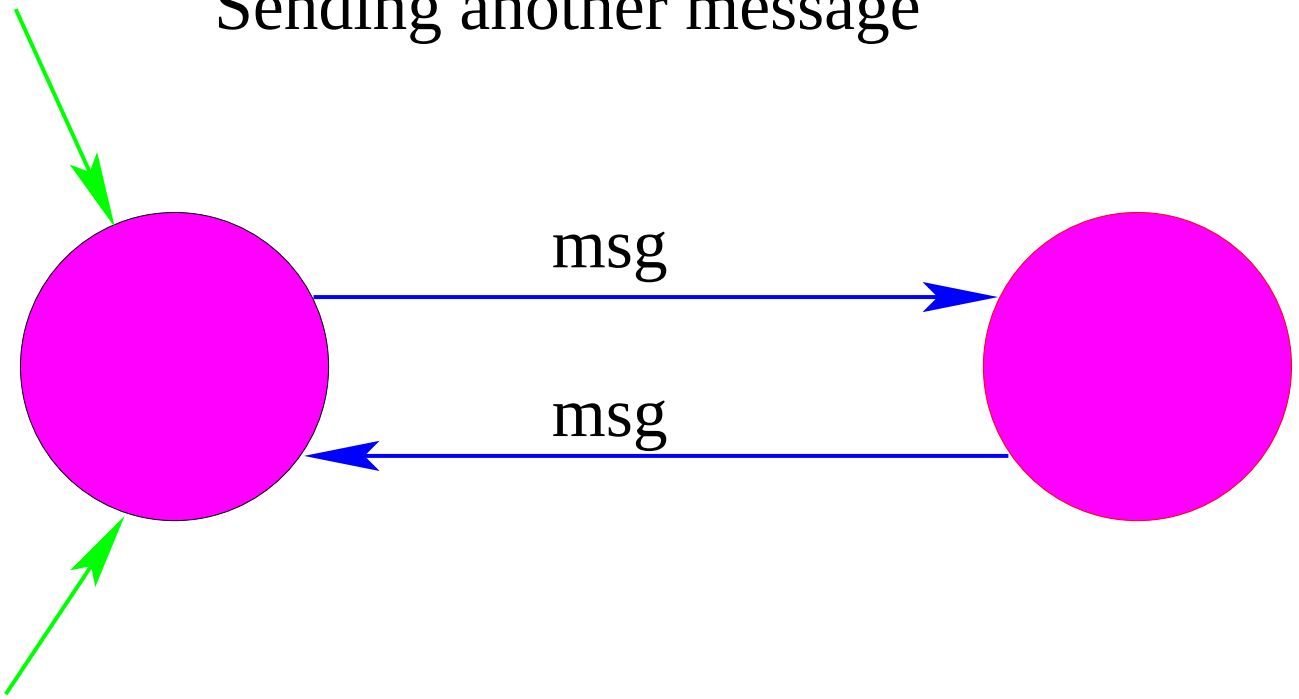



gr

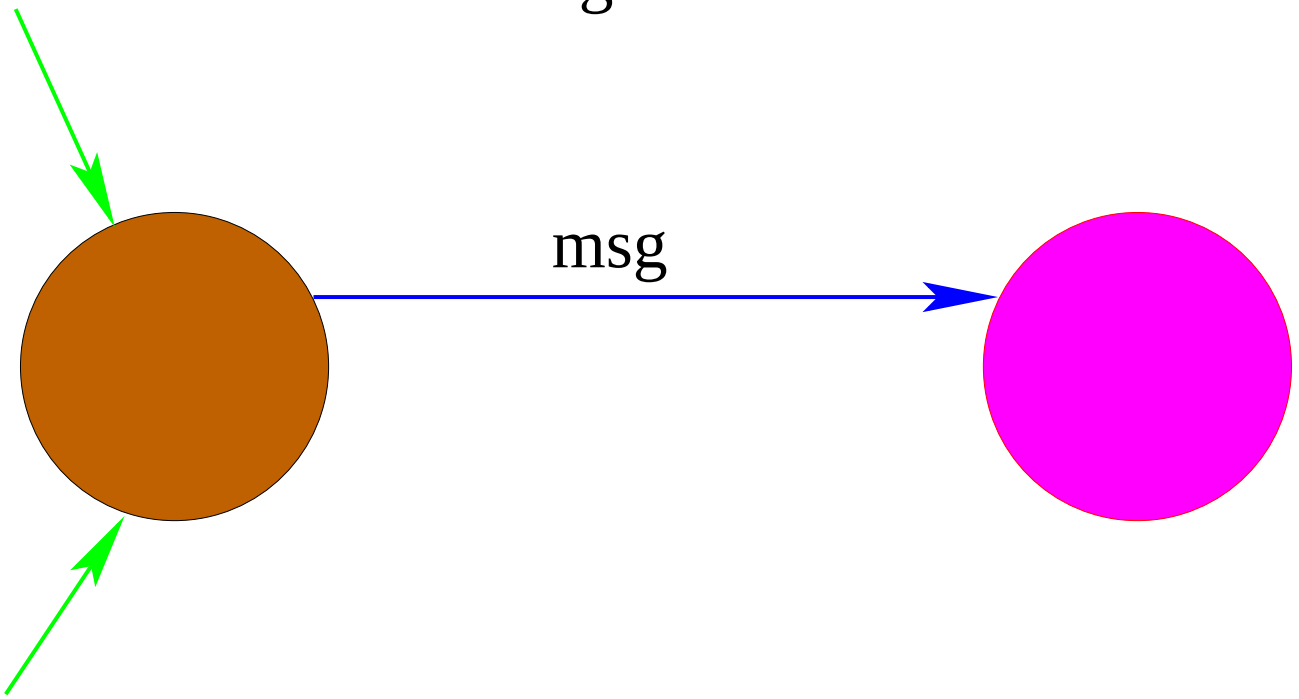
Sending a message



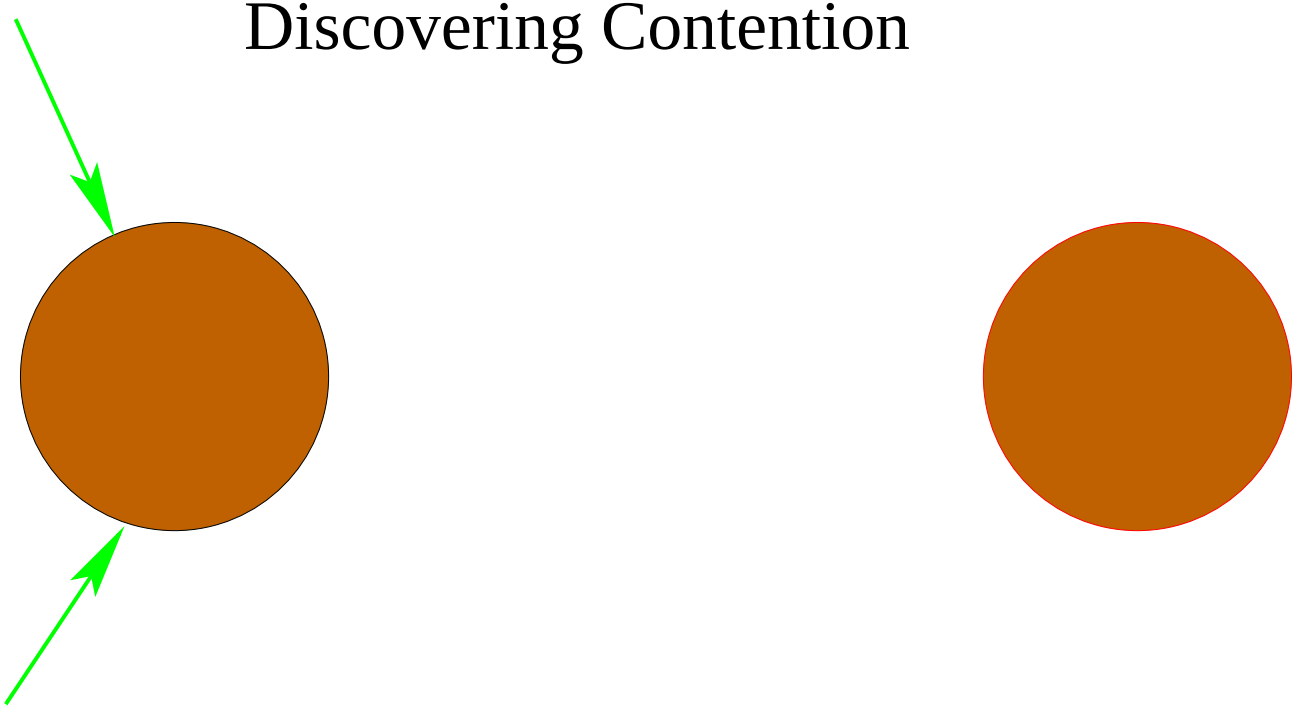
Sending another message



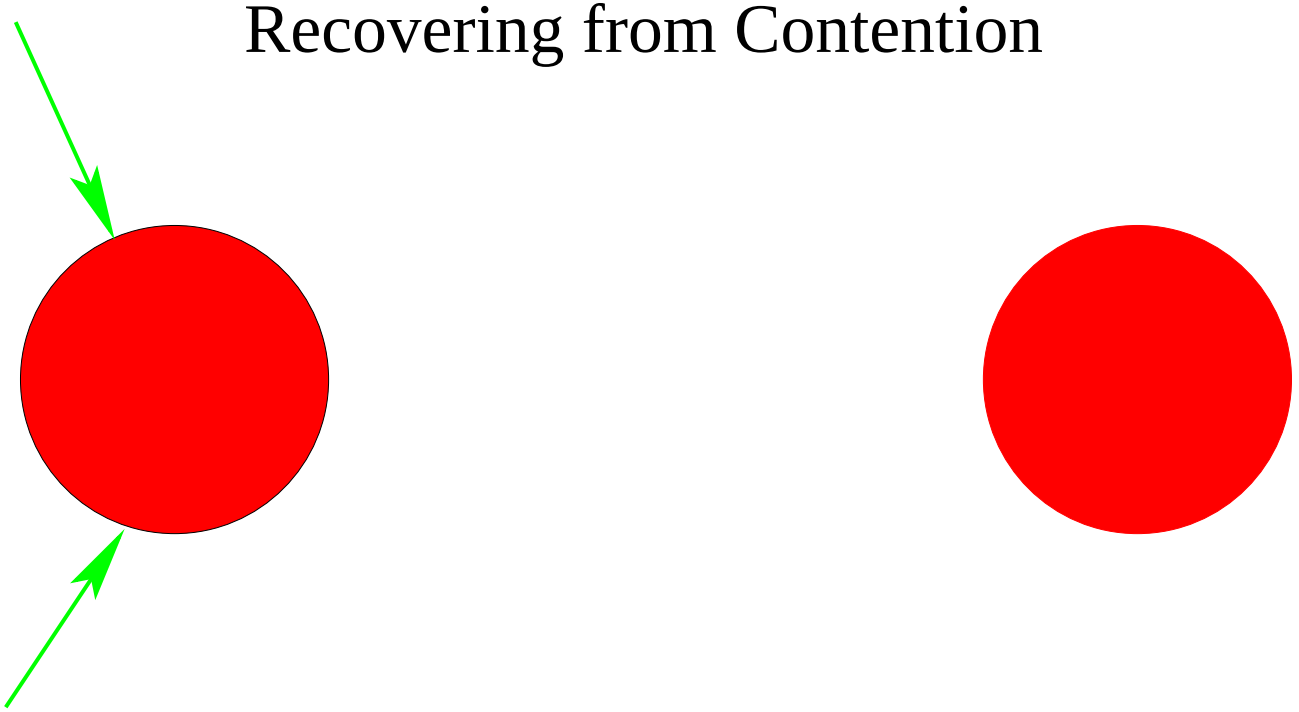
Discovering Contention



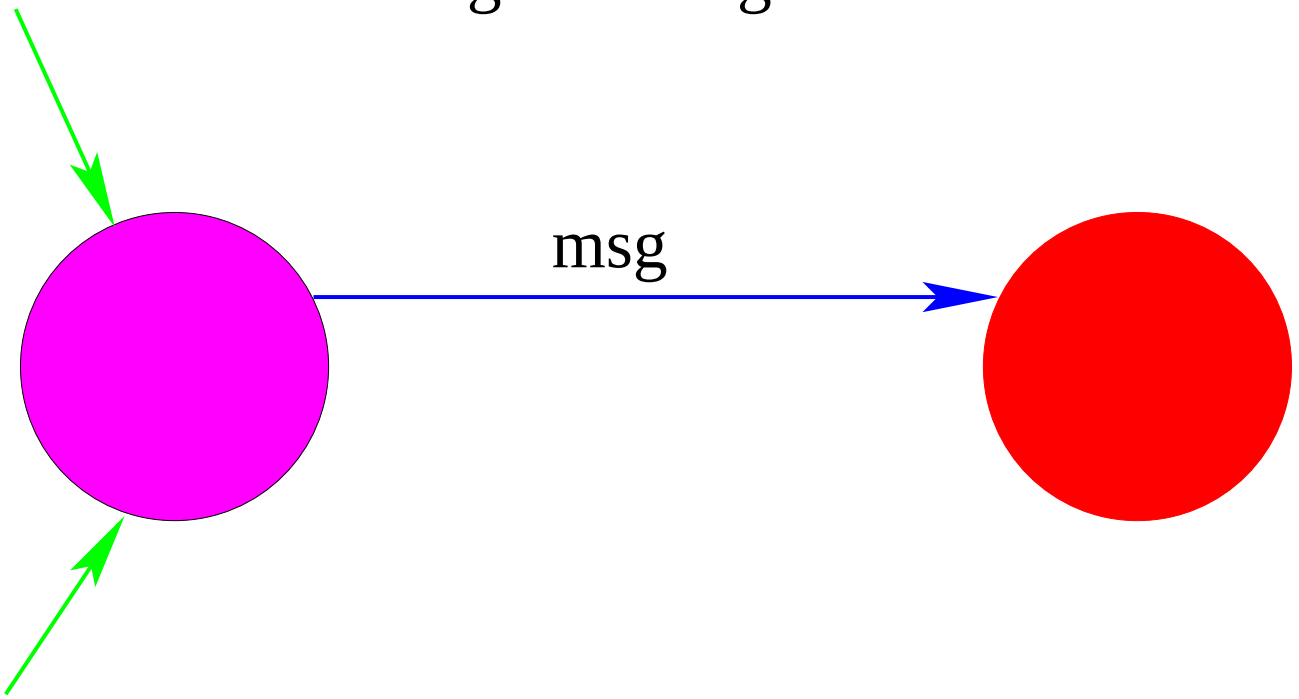
Discovering Contention



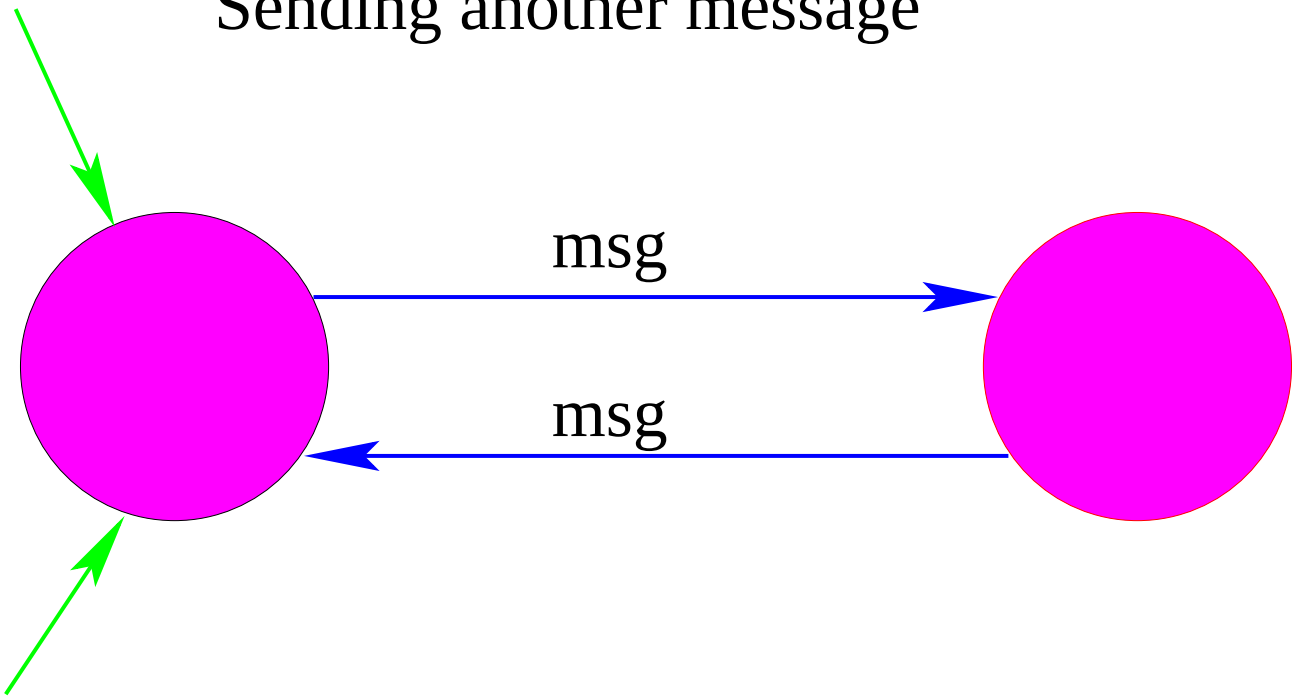
Recovering from Contention



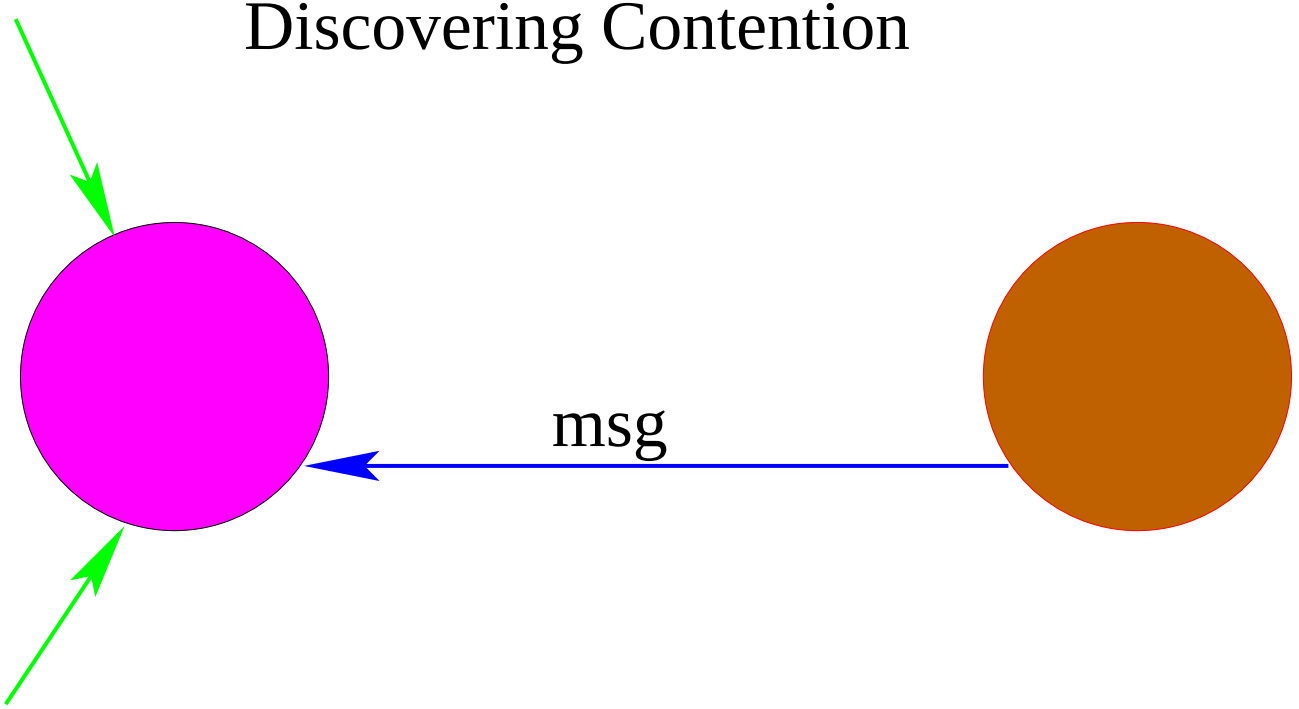
Sending a message



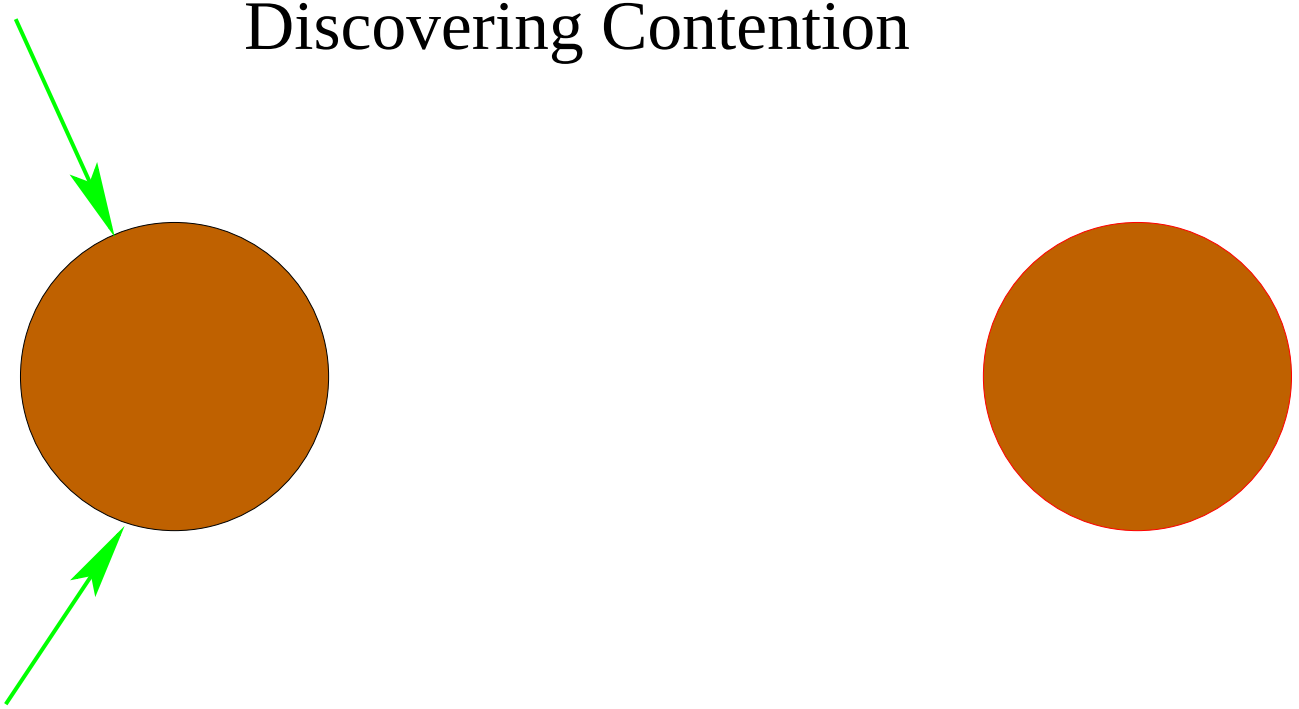
Sending another message



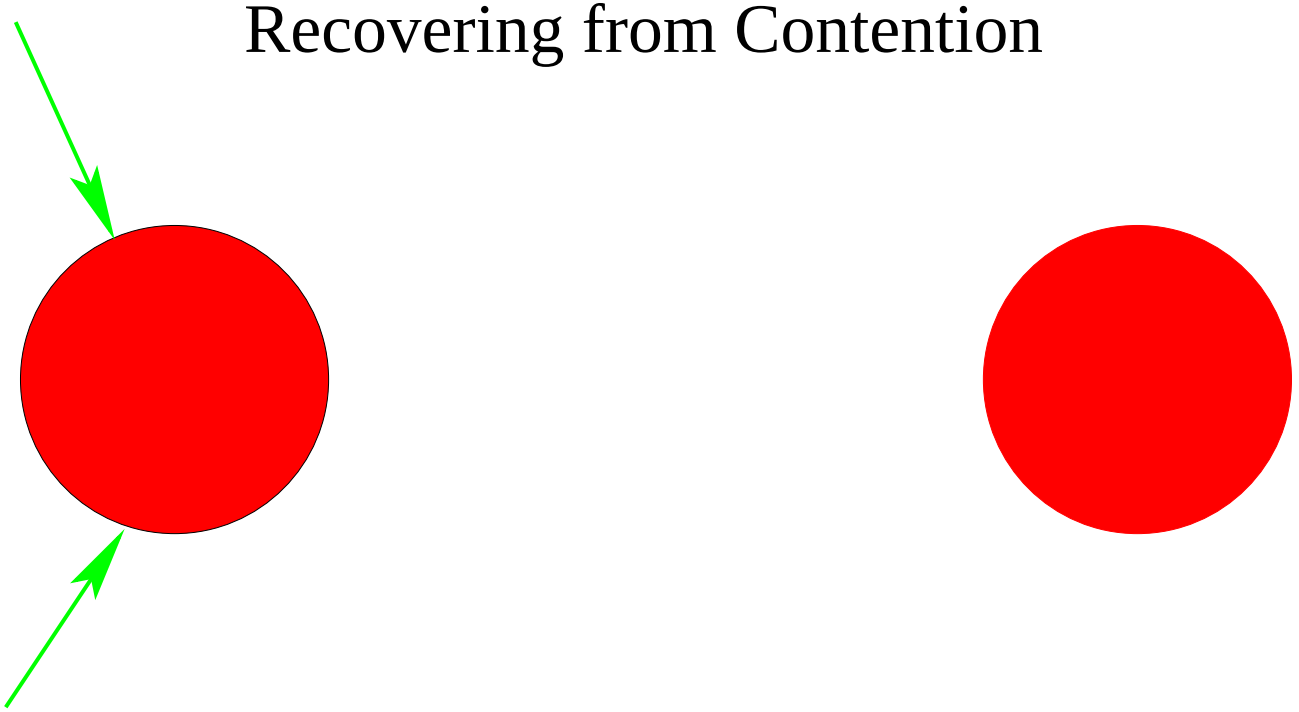
Discovering Contention



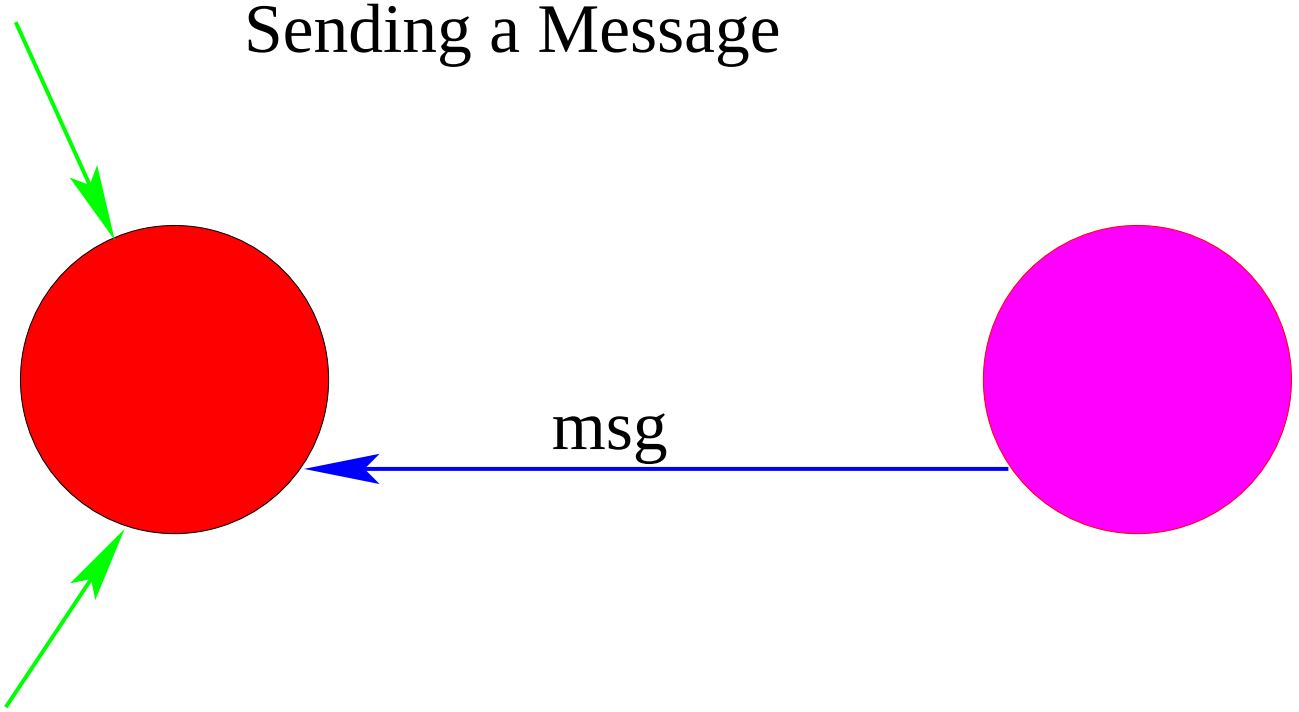
Discovering Contention



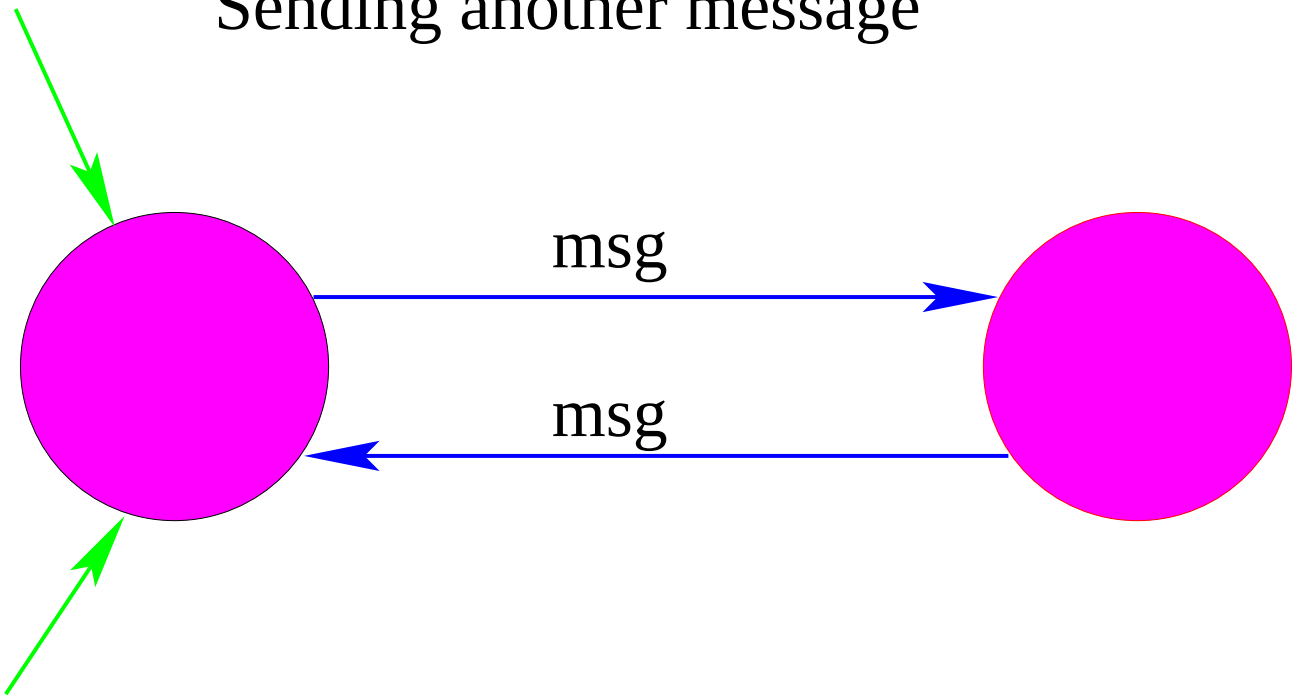
Recovering from Contention



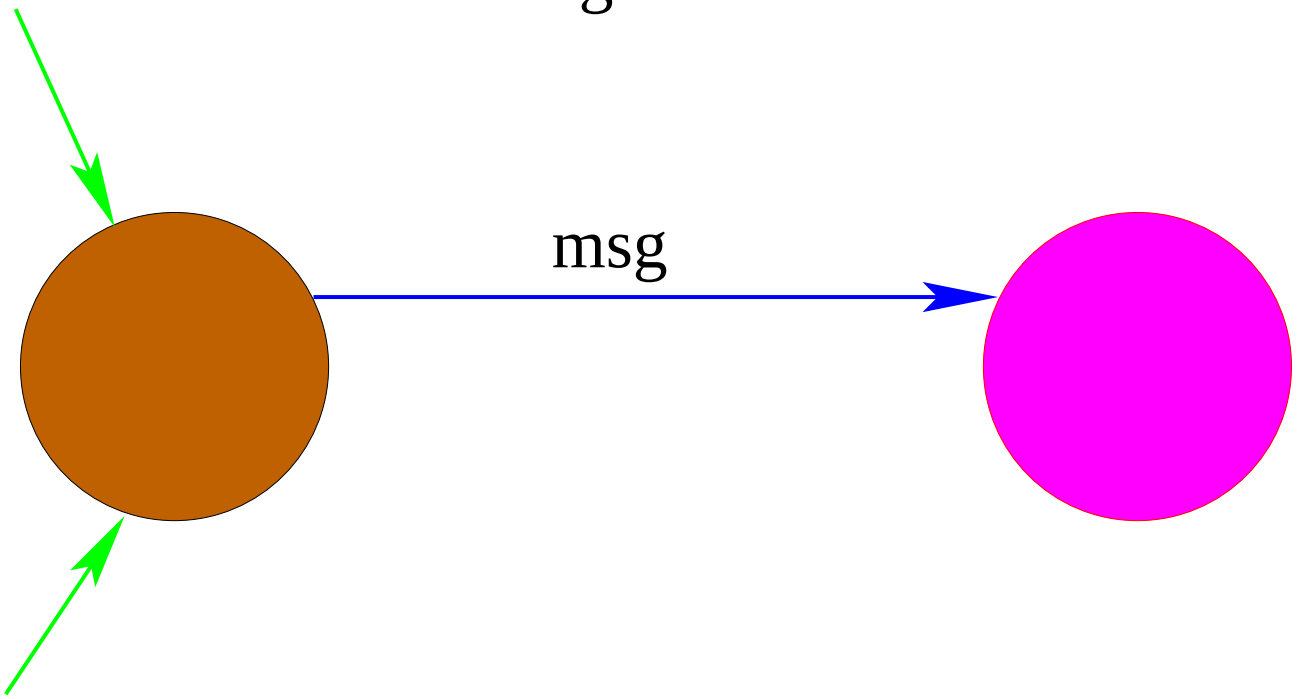
Sending a Message



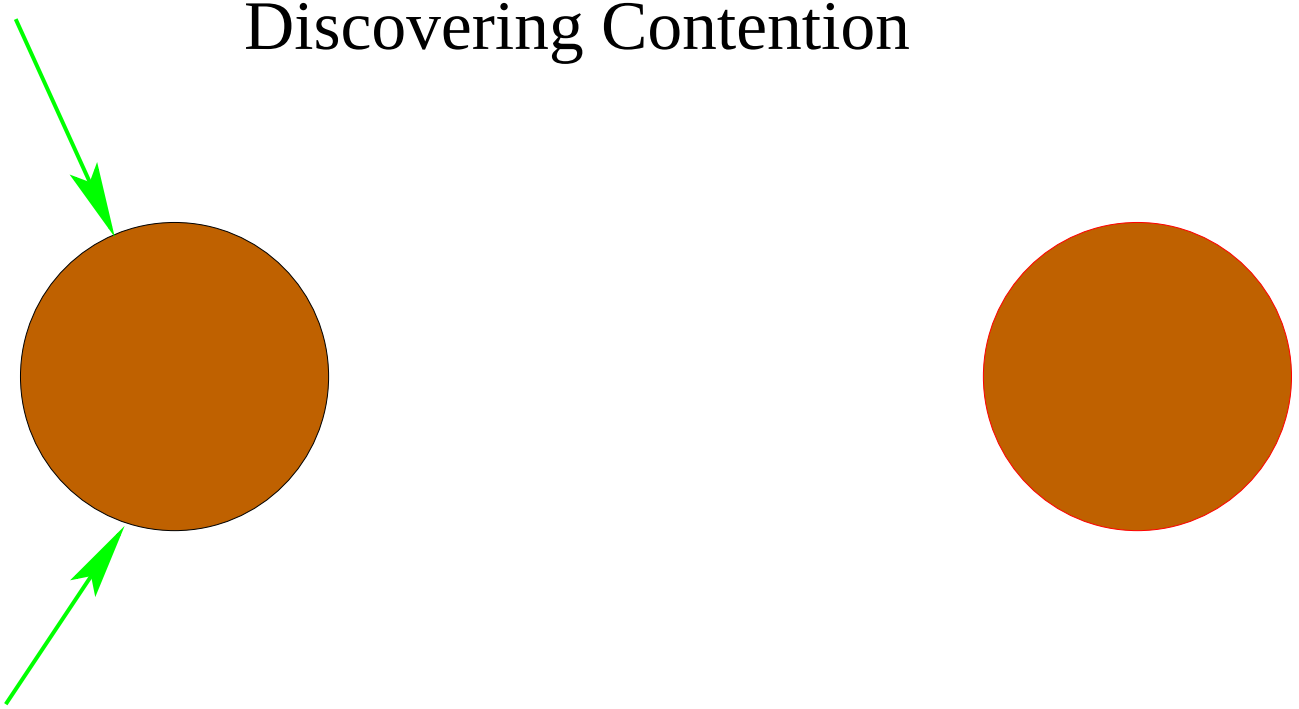
Sending another message



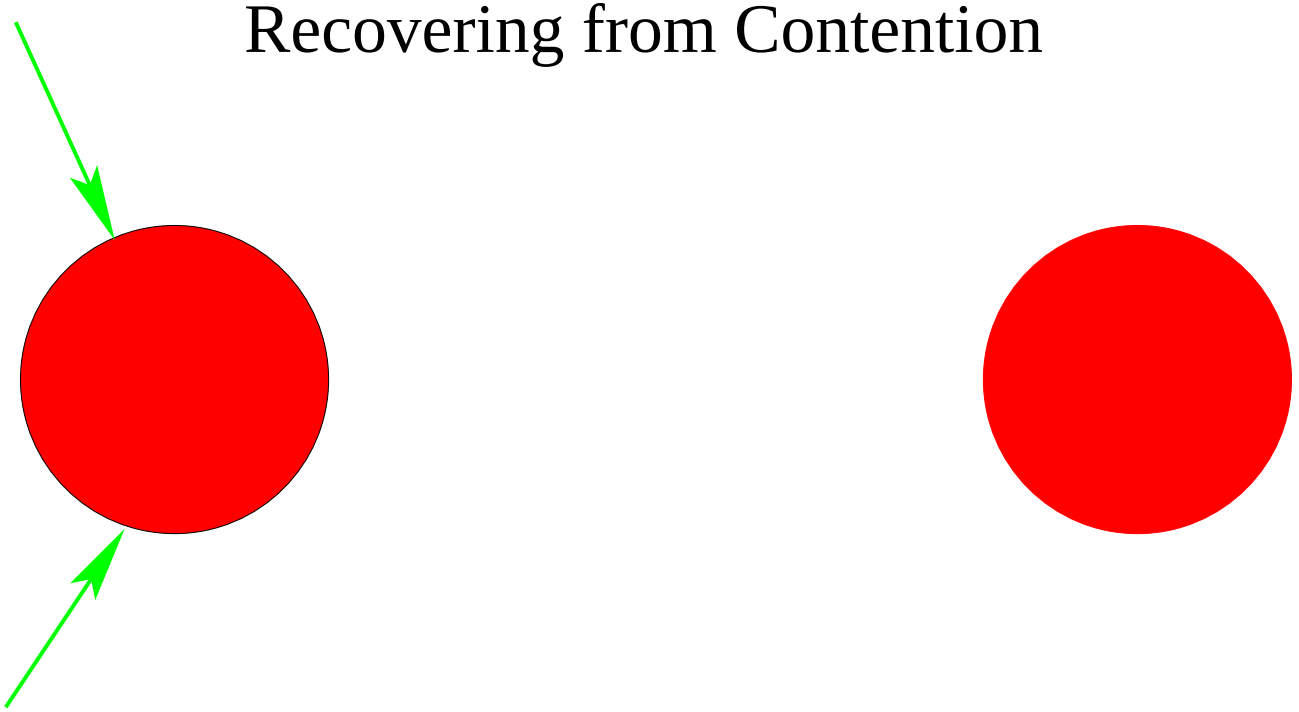
Discovering Contention



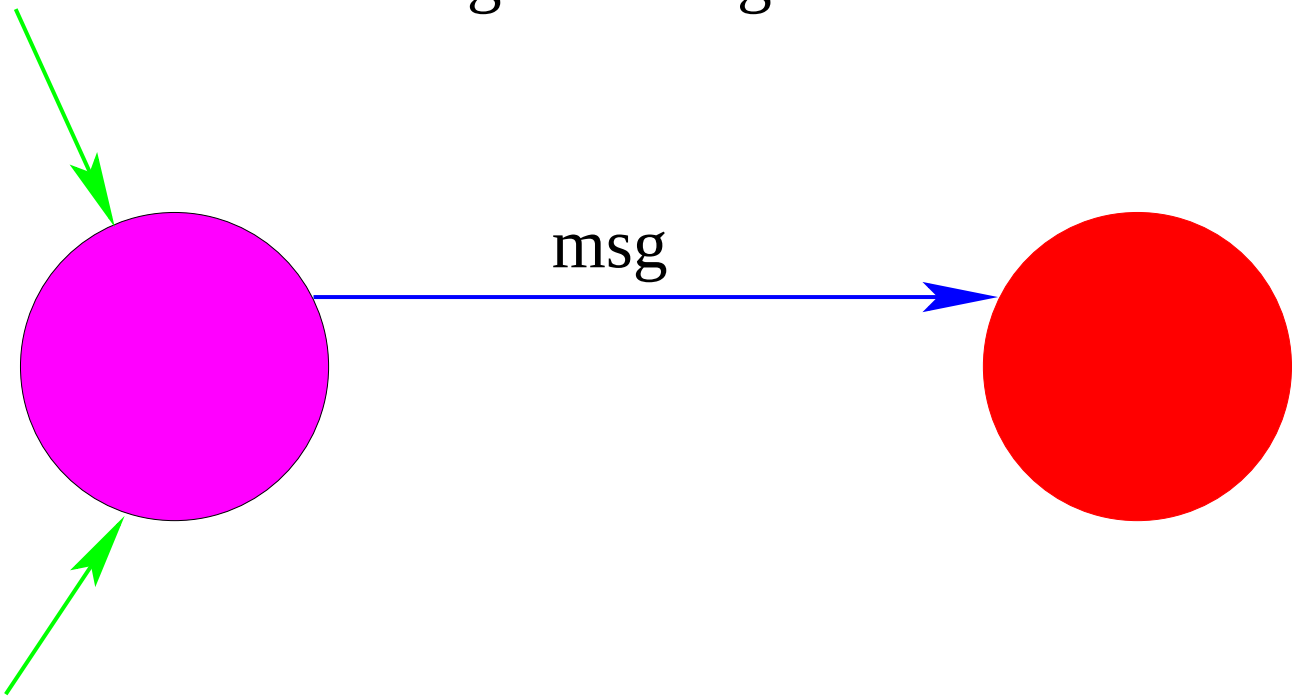
Discovering Contention



Recovering from Contention

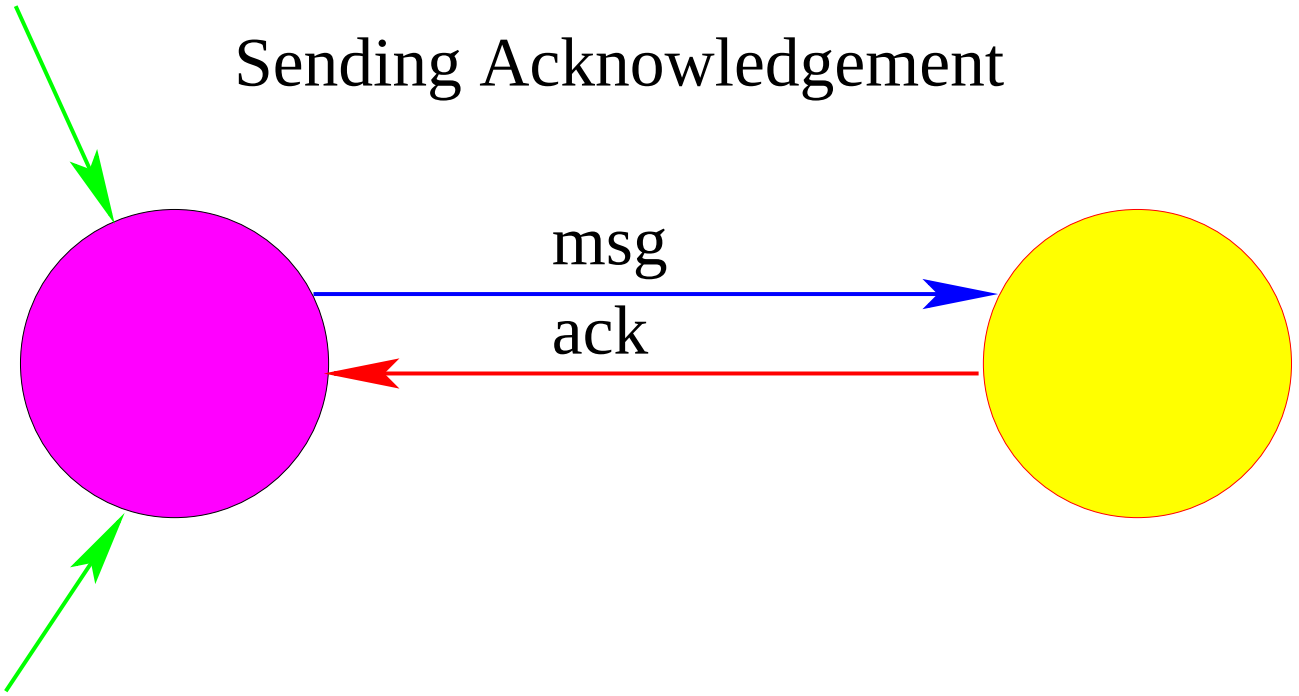


Sending a message



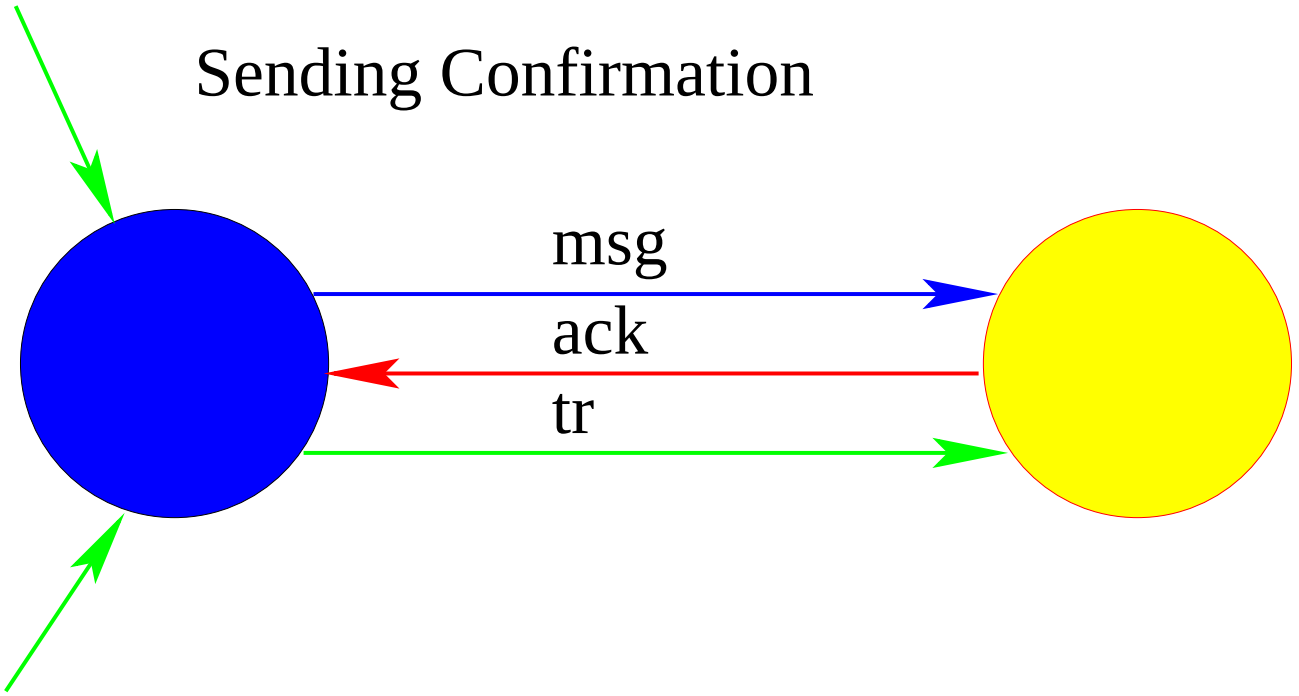
Receiving a message

Sending Acknowledgement

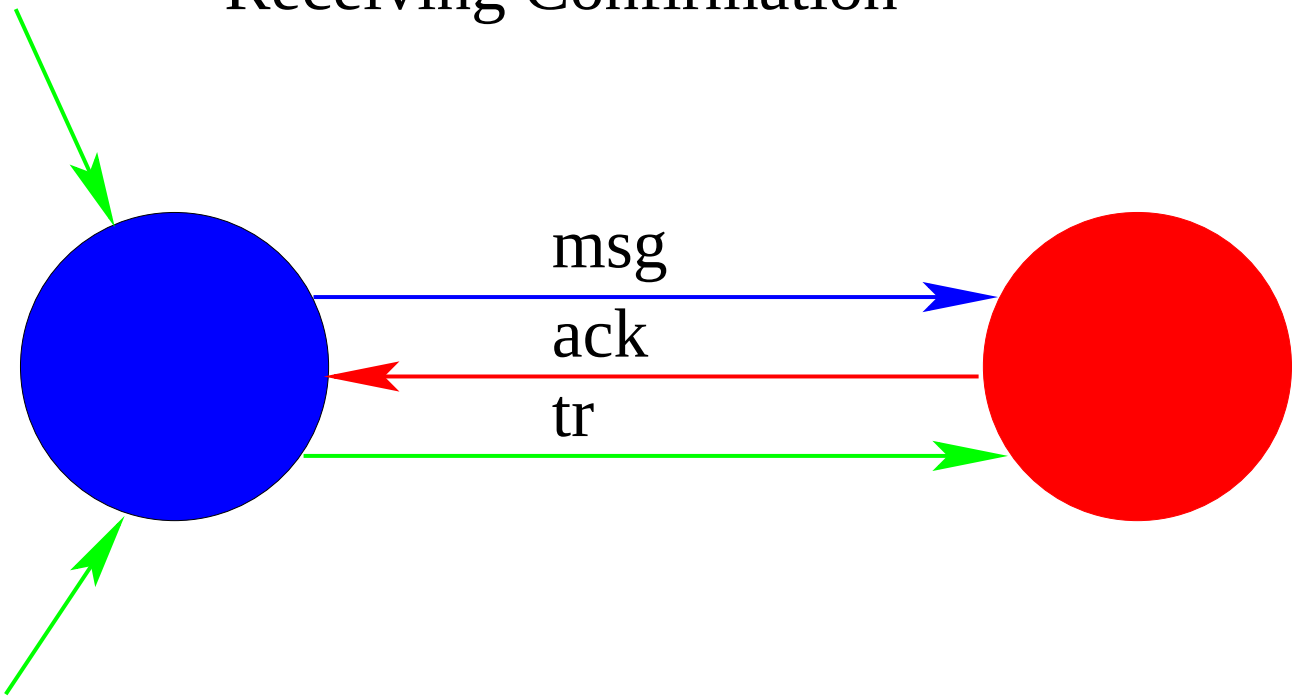


Receiving Acknowledgement

Sending Confirmation



Receiving Confirmation



Découverte de la contention (1)

- Nœud y découvre la contention avec x car :
    - Il a envoyé un message à x
    - Il n'a pas encore reçu une réponse du nœud x
    - Il reçoit à la place un message de x

Découverte de la contention (2)

- x découvre aussi la contention avec y

- Hypothèse : Le temps entre deux découvertes
  EST SUPPOSÉ BORNÉ
  PAR τ ms

- Le temps τ est le temps maximum de transmission entre 2 noeuds connectés

Une solution partielle

- Chaque noeud attend τ ms après sa découverte

- Après cela, chaque noeud sait que l'autre a aussi découvert la contention

- Chaque noeud essaie à nouveau immédiatement

- PROBLEME : Cela peut continuer indéfiniment

Une meilleure solution (1)

- Tout nœud attend τ ms après sa propre découverte

- Chaque nœud choisit avec équiprobabilité :
    - soit d'attendre un délai court
    - soit d'attendre un délai long

- Chaque nœud essaie à nouveau

nœud x

EVENT send_ack $\widehat{=}$
ANY x, y **WHERE**
$x, y \in msg{-}ack \ \wedge$
$y \notin \mathsf{dom}\,(msg)$
THEN
$ack := ack \cup \{x \mapsto y\}$
END

EVENT contention $\widehat{=}$
ANY x, y **WHERE**
$x, y \in msg{-}ack \ \wedge$
$y \in \mathsf{dom}\,(msg)$
THEN
$cnt := cnt \cup \{x \mapsto y\}$
END

- Introduire un canal de contention factice : cnt

Invariant (3)

$$\forall\,(x,y)\cdot\left(\begin{array}{l} x,y\in msg{-}ack\ \wedge \\ y\in \mathsf{dom}\,(msg) \\ \Longrightarrow \\ y,x\in msg{-}ack \end{array}\right)$$

$$\forall\,(x,y,z)\cdot\left(\begin{array}{l} x,y\in msg\quad\wedge \\ z\in gr[\{x\}]\quad\wedge \\ z\neq y \\ \Longrightarrow \\ z,x\in tr \end{array}\right)$$

Invariant (4)

$$\forall (x, y) \cdot \left(\begin{array}{ll} x, y \in msg - ack & \wedge \\ y \notin \mathsf{dom}\,(msg) & \\ \Longrightarrow & \\ x, y \notin cnt & \end{array} \right)$$

$$ack \cap ack^{-1} = \varnothing$$

$$ack \cap cnt = \varnothing$$

Résolution de la contention (simuler le délai τ)

$$
\begin{array}{l}
\text{EVENT solve_contention} \quad \widehat{=} \\
\quad \textbf{ANY } x, y \textbf{ WHERE} \\
\qquad x, y \in cnt \cup cnt^{-1} \\
\quad \textbf{THEN} \\
\qquad msg := msg - cnt \quad \| \\
\qquad cnt := \varnothing \\
\quad \textbf{END}
\end{array}
$$

Récapitulatif du Second Raffinement

- 63 preuves

- Parmi lesquelles 31 interactives

développement

- Etablir le cadre mathématique
- Résoudre le problème mathématique en un coup.
- Résoudre le même problème progressivement.
- Inclure les communications par des messages.
- Localiser les structures de donnée
- Méthodologie fondée sur le raffinement et la preuve
- Autres développements : algorithmique répartie, algorithmique séquentielle, systèmes, SoCs, ...

The predicate invariant (tr)

$$tr \in ND \nrightarrow ND$$

The predicate invariant (tr)

$$tr \in ND \nrightarrow ND$$

$$\mathsf{dom}\,(tr) \lhd (tr \cup tr^{-1}) = \mathsf{dom}\,(tr) \lhd gr$$

Invariant (3)

$$\forall\,(x,y)\cdot\left(\begin{array}{ll} x,y\in msg & \wedge \\ y,x\in msg & \\ \Longrightarrow & \\ y,x\notin ack & \end{array}\right)$$

$$\forall\,(x,y,z)\cdot\left(\begin{array}{ll} x,y\in msg & \wedge \\ z\in gr[\{x\}] & \wedge \\ z\neq y & \\ \Longrightarrow & \\ z,x\in tr & \end{array}\right)$$

Invariant (4)

$$\forall (x, y) \cdot \left(\begin{array}{ll} x, y \in cnt & \\ \Longrightarrow & \\ x, y \in msg{-}ack & \wedge \\ y \in \mathsf{dom}\,(msg) & \end{array} \right)$$

$$ack \cap ack^{-1} = \varnothing$$

$$ack \cap cnt = \varnothing$$

Third Refinement : Localization

- The representation of the graph gr is modified

- The representation of the tree tr is modified

- Other data structures are localized

Localization (1)

The graph gr and the tree tr are now localized

$$nb \in ND \rightarrow \mathbb{P}(ND)$$

$$\forall x \cdot (\, x \in ND \implies nb(x) = gr[\{x\}]\,)$$

$$sn \in ND \rightarrow \mathbb{P}(ND)$$

$$\forall x \cdot (\, x \in ND \implies sn(x) \subseteq tr^{-1}[\{x\}]\,)$$

Localization (2)

$$bm \subseteq ND$$

$$bm = \mathsf{dom}\,(msg)$$

$$bt \subseteq ND$$

$$bt = \mathsf{dom}\,(tr)$$

$$ba \in ND \rightarrow \mathbb{P}(ND)$$

$$\forall x \cdot (\, x \in ND \implies ba(x) = ack^{-1}[\{x\}]\,)$$

- Node x is elected the leader

$$
\begin{array}{l}
\text{EVENT elect} \;\; \widehat{=} \\
\quad \textbf{ANY } x \textbf{ WHERE} \\
\qquad x \in ND \;\wedge \\
\qquad nb(x) = sn(x) \\
\quad \textbf{THEN} \\
\qquad rt := x \\
\quad \textbf{END}
\end{array}
$$

- Node x sends a message to node y (y is unique)

$$
\begin{array}{l}
\text{EVENT send_msg} \ \widehat{=} \\
\quad \textbf{ANY } x, y \textbf{ WHERE} \\
\qquad x \in ND\text{-}bm \ \wedge \\
\qquad y \in ND\text{-}ba(x) \ \wedge \\
\qquad nb(x) = sn(x) \cup \{y\} \\
\quad \textbf{THEN} \\
\qquad msg := msg \cup \{x \mapsto y\} \quad \| \\
\qquad bm := bm \cup \{x\} \\
\quad \textbf{END}
\end{array}
$$

- Node y sends an acknowledgement to node x

$$
\begin{array}{l}
\text{EVENT send_ack} \;\; \widehat{=} \\
\quad \textbf{ANY } x, y \textbf{ WHERE} \\
\qquad x, y \in msg \;\; \wedge \\
\qquad x \notin ba(y) \;\; \wedge \\
\qquad y \notin bm \\
\quad \textbf{THEN} \\
\qquad ack := ack \cup \{x \mapsto y\} \quad \| \\
\qquad ba(y) := ba(y) \cup \{x\} \\
\quad \textbf{END}
\end{array}
$$

- Node x sends a confirmation to node y

$$
\begin{array}{l}
\text{EVENT progress} \;\; \widehat{=} \\
\quad \textbf{ANY } x, y \textbf{ WHERE} \\
\qquad x, y \in ack \;\; \wedge \\
\qquad x \notin bt \\
\quad \textbf{THEN} \\
\qquad tr := tr \cup \{x \mapsto y\} \quad \| \\
\qquad bt := bt \cup \{x\} \\
\quad \textbf{END}
\end{array}
$$

- Node y receives confirmation from node x

$$
\begin{array}{l}
\text{EVENT rcv_cnf} \ \widehat{=} \\
\quad \textbf{ANY } x, y \textbf{ WHERE} \\
\qquad x, y \in tr \ \wedge \\
\qquad x \notin sn(y) \\
\quad \textbf{THEN} \\
\qquad sn(y) := sn(y) \cup \{x\} \\
\quad \textbf{END}
\end{array}
$$

$$
\begin{array}{l}
\text{EVENT contention} \;\; \widehat{=} \\
\quad \textbf{ANY } x, y \textbf{ WHERE} \\
\qquad x, y \in cnt \cup cnt^{-1} \;\; \wedge \\
\qquad x \notin ba(y) \;\; \wedge \\
\qquad y \in bm \\
\quad \textbf{THEN} \\
\qquad cnt := cnt \cup \{x \mapsto y\} \\
\quad \textbf{END}
\end{array}
$$

$$
\begin{array}{l}
\text{EVENT solve_contention} \quad \widehat{=} \\
\quad \textbf{ANY } x, y \textbf{ WHERE} \\
\qquad x, y \in cnt \cup cnt^{-1} \\
\quad \textbf{THEN} \\
\qquad msg := msg - cnt \quad \| \\
\qquad bm := bm - \mathsf{dom}\,(cnt) \quad \| \\
\qquad cnt := \varnothing \\
\quad \textbf{END}
\end{array}
$$

Section Courante

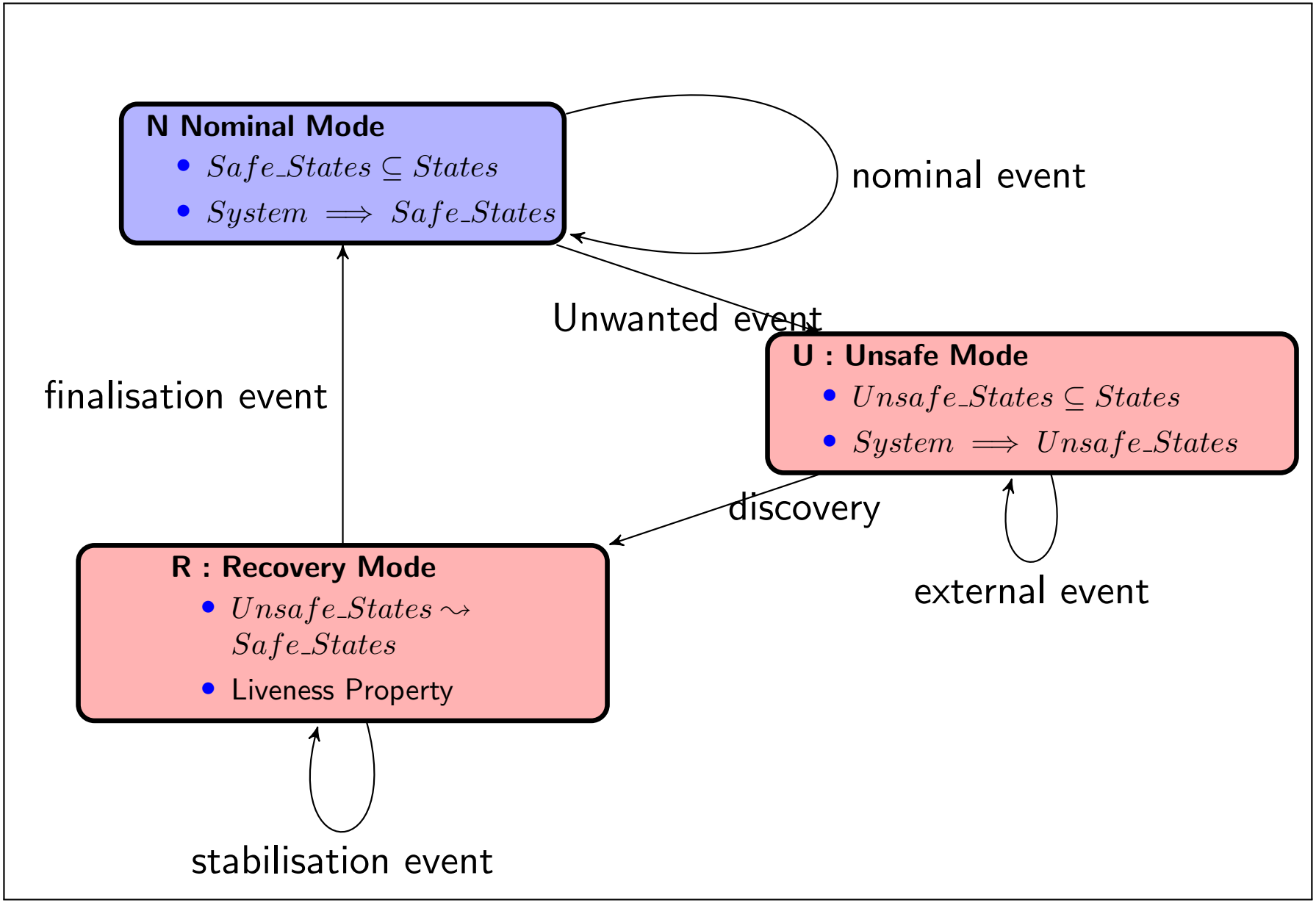
1. Election
   - Problème de l'élection
2. Election
   - Problème de l'élection
   - Election dans un graphe acyclique
3. Auto-stabilisation
   - Généralités
   - Définition
   - Réseaux en anneau
   - Algorithme de Dijkstra
   - Algorithme de coloriage
4. Calcul de l'état global
   - Introduction du problème
   - Etat global consistant
   - Algorithme de Chandy et Lamport

]

Etat courant

1. Election
   - Problème de l'élection
2. Election
   - Problème de l'élection
   - Election dans un graphe acyclique
3. Auto-stabilisation
   - Généralités
   - Définition
   - Réseaux en anneau
   - Algorithme de Dijkstra
   - Algorithme de coloriage
4. Calcul de l'état global
   - Introduction du problème
   - Etat global consistant
   - Algorithme de Chandy et Lamport

Auto-stabilisation dans les systèmes répartis



Stabilisation

- Les systèmes peuvent être victimes de fautes transitoires
- Un certain nombre d'algorithmes permettent d'anticiper de tels problèmes.
- Les algorithmes auto-stabilisants constituent une alternative à la perte de l'état consistant d'un système donné à la suite d'une défaillance.
- Différents phénomènes peuvent apparaître lors de l'exécution d'un algorithme : en particulier peuvent se produire des changements dans le réseau (ajout ou disparition d'un sommet ou d'un lien) ou bien encore des altérations de messages ou de mémoires.
- Un algorithme est dit auto-stabilisant s'il se termine correctement en dépit de l'apparition de ces phénomènes.
- Un algorithme auto-stabilisant ne nécessite pas d'initialisation particulière.

Contexte

- Les systèmes répartis sont exposés à des risques de défaillances transitoires qui peuvent placer un système donné dans un état ou une configuration arbitraire.
- Cette configuration arbitraire peut être une configuration ne satisfaisant plus l'invariant du système.
- Dans ce cas, la question est de concevoir un algorithme réparti permettant de faire converger le système global d'une configuration arbitraire vers une configuration dite stable et satisfaisant l'invariant.

Etat courant

1. Election
   - Problème de l'élection
2. Election
   - Problème de l'élection
   - Election dans un graphe acyclique
3. Auto-stabilisation
   - Généralités
   - Définition
   - Réseaux en anneau
   - Algorithme de Dijkstra
   - Algorithme de coloriage
4. Calcul de l'état global
   - Introduction du problème
   - Etat global consistant
   - Algorithme de Chandy et Lamport

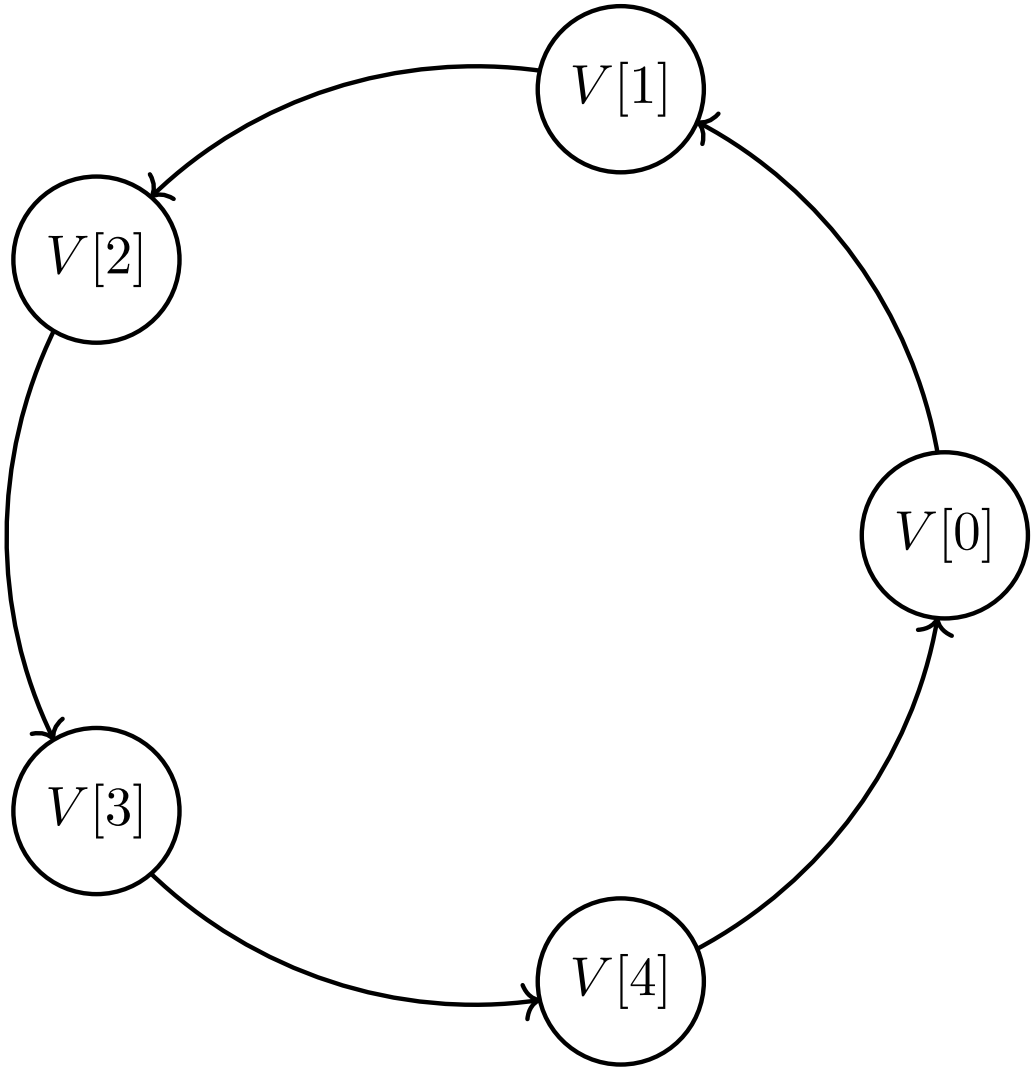
Modélisation d'un système auto-stabilisant

- Soit un système réparti $\mathcal{S}$ modélisé par un système de transition $(States, \longrightarrow)$ où $States$ est un ensemble d'états ou de configurations et où $\longrightarrow$ modélise la relation de transition simulant l'activité du système.
- Une exécution ou un calcul de $\mathcal{S}$ est une suite maximale $(\sigma_0, \sigma_1, \sigma_2, \ldots, \sigma_i, \ldots)$ engendrée par $(States, \longrightarrow)$.
- Tout préfixe d'une telle suite est un calcul puisqu'il n'y a pas d'états initiaux déterminés.
- Tout préfixe d'exécution est donc un calcul ou une exécution de $\mathcal{S}$ (pas de dépendance d'un état initial)

Etat courant

1. Election
   - Problème de l'élection
2. Election
   - Problème de l'élection
   - Election dans un graphe acyclique
3. Auto-stabilisation
   - Généralités
   - Définition
   - Réseaux en anneau
   - Algorithme de Dijkstra
   - Algorithme de coloriage
4. Calcul de l'état global
   - Introduction du problème
   - Etat global consistant
   - Algorithme de Chandy et Lamport

Réseau en anneau



Etat courant

1. Election
   - Problème de l'élection
2. Election
   - Problème de l'élection
   - Election dans un graphe acyclique
3. Auto-stabilisation
   - Généralités
   - Définition
   - Réseaux en anneau
   - Algorithme de Dijkstra
   - Algorithme de coloriage
4. Calcul de l'état global
   - Introduction du problème
   - Etat global consistant
   - Algorithme de Chandy et Lamport

Problème de l'exclusion mutuelle

Description

- Dans toute configuration, au plus un processus a le privilège.
- Chaque processus a le privilège infiniment souvent.

- Dans un réseau avec une topologie en anneau, des processus ont accès à une ressource.
- Le privilège est accordé à un processus à la fois.
- Le problème est que le privilège est géré par un jeton qui peut être perdu et donc être dans une configuration ne satisfaisant plus la propriété.

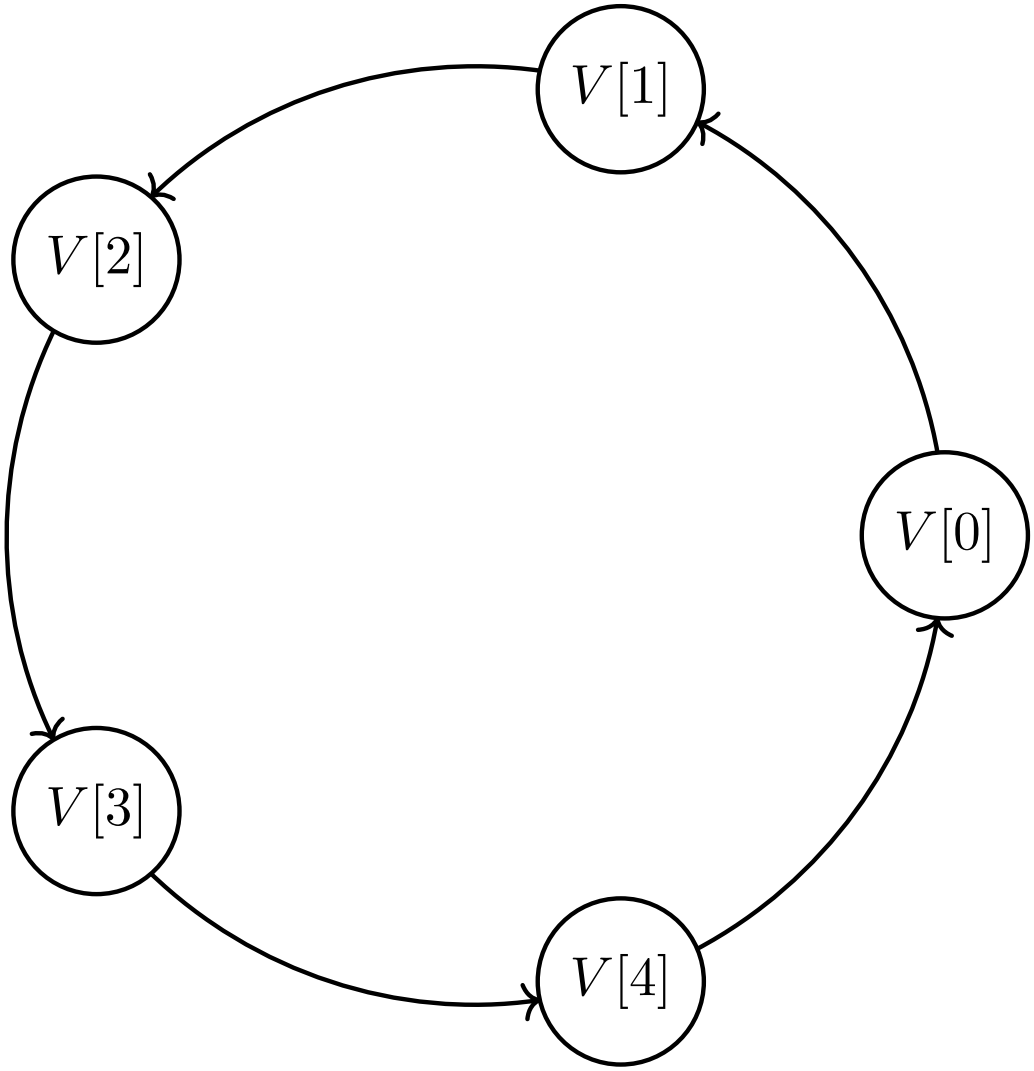
Algorithme de Dijkstra

$$\text{DOMAINE} \triangleq 0..N$$
$$\text{IMAGE} \triangleq 0..M$$

$$\text{NToZero} \triangleq$$
$$\begin{array}{ll} \wedge & V[0] = V[N] \\ \wedge & V' = [V \ \textbf{EXCEPT} \ ![0] = (V[0]+1)\%(M+1)] \end{array}$$

$$\text{Others}(\text{I}) \triangleq$$
$$\begin{array}{ll} \wedge & I \in 0..N \\ \wedge & I \neq N \\ \wedge & V[I+1] \neq V[I] \\ \wedge & V' = [V \ \textbf{EXCEPT} \ ![I+1] = V[I]] \end{array}$$

Réseau en anneau



Propriétés de stabilisation

```
Init == V = [i \in 0..N |-> (IF i # N THEN i  ELSE 0)]
Next == NToZero \/ (\E I \in 0..N-1:Others(I))

Prop1 ==    (V[0] = V[N])
              => (\A i \in 1..N-1: V[i+1] = V[i])

Prop2 ==
        \/  (\E i,j  \in 0..N-1: i # j
              /\ V[i+1] #  V[i] /\ V[j+1] #  V[j])
        \/ (V[0] = V[N] /\
              (\E i  \in 0..N-1:  V[i+1] #  V[i]))
```

Etat courant

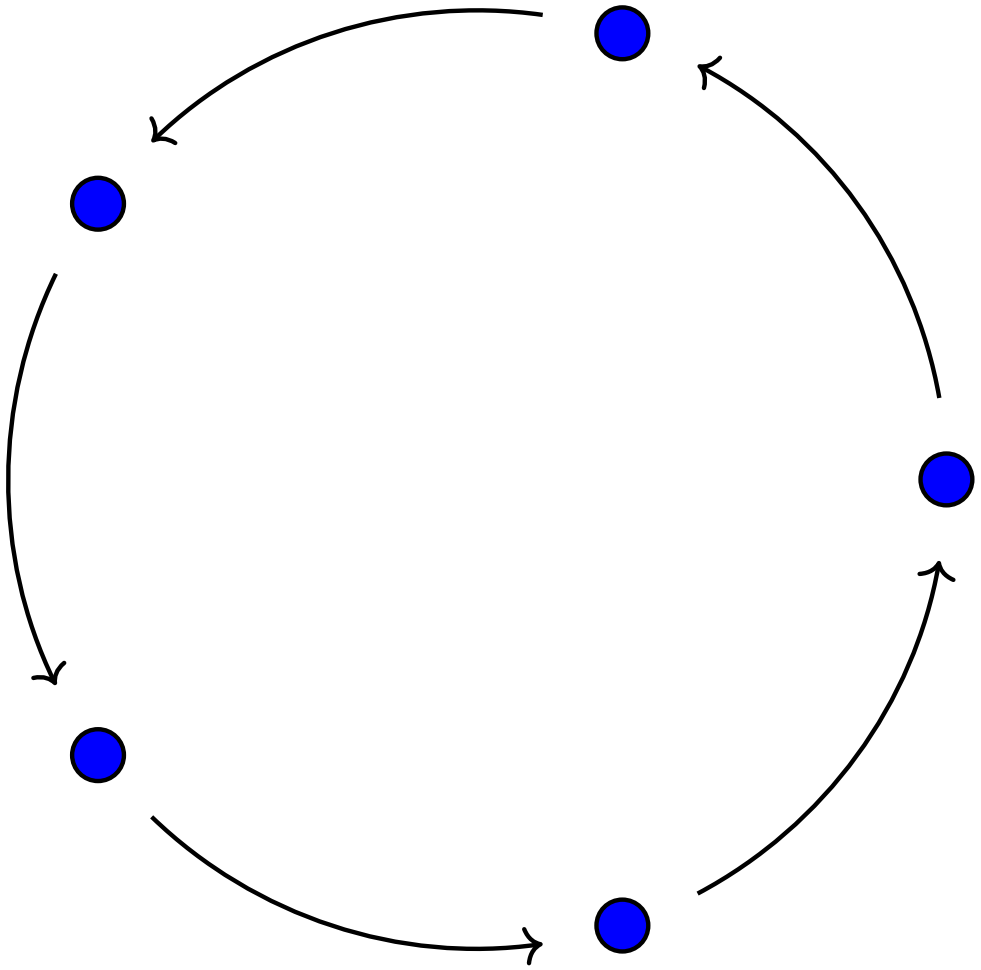
1. Election
   - Problème de l'élection
2. Election
   - Problème de l'élection
   - Election dans un graphe acyclique
3. Auto-stabilisation
   - Généralités
   - Définition
   - Réseaux en anneau
   - Algorithme de Dijkstra
   - Algorithme de coloriage
4. Calcul de l'état global
   - Introduction du problème
   - Etat global consistant
   - Algorithme de Chandy et Lamport

Coloriage d'un anneau

Le problème du coloriage d'un anneau concerne l'existence d'une configuration dans laquelle trois nœuds consécutifs n'ont pas la même couleur et pose la question de définir un algorithme réparti permettant de l'atteindre à parrtir de toute configuration intiale de l'anneau.

- Modéliser un processus de coloration d'un anneau à partir d'une situation quelconque initiale pour atteindre une situation stable caractérisée par le fait que deux nœuds voisins n'ont pas la même couleur.
- La règle de changement de couleur : Si un nœud a la même couleur que l'un de ses vosins, il chosit une autre couleur que celle de ses voisins.
- La règle est applicable tant que deux nœuds ont la même couleur.
- La règle est non-applicable dès que les nœuds adjacents ont tous une couleur différente : le cas d'un anneau à un nœud est exclus en considérant qu'il y a au moins deux nœuds dans l'anneau.

Exemple d'anneau



CONTEXT $ring$
SETS
N
CONSTANTS
n
AXIOMS
$axm1 : n \in N \rightarrowtail\!\!\!\!\twoheadrightarrow N$
$axm2 : \forall s \cdot s \subseteq n[s] \Rightarrow N \subseteq s$
$axm3 : finite(N)$
END
MACHINE $one-shot$
SEES $ring, color$
VARIABLES
col
INVARIANTS
$inv1 : col \in N \rightarrow color$

EVENTS
EVENT INITIALISATION
$act1 : col :\in N \rightarrow color$
EVENT stable
ANY
f
WHERE
$grd1 : f \in N \rightarrow color$
$grd2 : \forall x \cdot f(x) \notin f[(n \cup n^{-1})[\{x\}]]$
THEN
$act1 : col := f$
END

Le modèle définit la solution à trouver par un événement abstrait exprimant le lien entre la configuration initiale et une configuration stable. Les régles données explique comment le calcul est effectué par les différents processus du réseau. Chaque règle suppose que leur application est possible quand trois nœuds consécutifs vérifient une condition donnée.

Chaque règle constitue un élément de calcul réparti et cet élément est appliqué sur la *boule* centrée sur un nœud donné et tient compte des couleurs des nœpériphériques.

MACHINE $rule$ **REFINES** $one{-}shot$
SEES $ring, color$
VARIABLES col, c
INVARIANTS
$inv1 : c \in N \rightarrow color$
THEOREMS
$thm1 :$

$$\left(\begin{array}{l} \forall x \cdot c(x) \neq c(n(x)) \\ \vee \\ \exists x, clr \cdot c(x) = c(n(x)) \wedge clr \neq c(x) \wedge clr \neq c(n(x))) \end{array} \right)$$

EVENTS

EVENT INITIALISATION

$$act1 : col, c : | \left(\begin{array}{l} col' \in N \rightarrow color \\ \wedge \\ c' \in N \rightarrow color \\ \wedge \\ col' = c' \end{array} \right)$$

EVENT stable REFINES stable

WHEN

$grd1 : \forall x \cdot c(x) \neq c(n(x))$

WITNESSES

$f : f = c$

THEN

$act1 : col := c$

END

EVENT rule

STATUS convergent

ANY

x, clr

WHERE

$grd1 : c(x) = c(n(x)) \vee c(x) = c(n^{-1}(x))$

$grd2 : clr \neq c(n(x))$

$grd3 : clr \neq c(n^{-1}(x))$

THEN

$act1 : c(x) := clr$

END

La terminaison de ce processus dépend de la condition appelée variant et qui doit décroître par application des règles.

VARIANT $card(\{x|c(x) = c(n(x))\})$

TLA$^+$

Les graphes VISIDIA sont symétriques ; le graphe g est défini à partir du graphe n et la règle $rule$ est raffinée en $visidia{-}rule$

CONTEXT $ringgr$
EXTENDS $ring1$
CONSTANTS
g
AXIOMS
$axm1 : g = n \cup n^{-1}$
END

EVENT INITIALISATION

$act1 : col, c : |(col' \in N \rightarrow color \wedge c' \in N \rightarrow color \wedge col' = c')$

EVENT stable REFINES stable

WHEN

$grd1 : \forall x \cdot c(x) \notin c[g[\{x\}]]$

THEN

$act1 : col := c$

END

EVENT rule

STATUS convergent

REFINES rule

ANY

x, clr

WHERE

$grd1 : c(x) \in c[g[\{x\}]]$

$grd2 : clr \notin c[g[\{x\}]]$

THEN

$act1 : c(x) := clr$

END

Dérivation d'un programme VISIDIA

Rule : $\overset{cl1}{\bullet}\!\!-\!\!\!-\!\!\!-\!\!\!-\!\!\overset{cl1}{\bullet}$ et $\overset{cl1}{\bullet}\!\!-\!\!\!-\!\!\!-\!\!\!-\!\!\overset{cl2}{\bullet}$ se réduit en $\overset{cl1}{\bullet}\!\!-\!\!\!-\!\!\!-\!\!\!-\!\!\overset{cl3}{\bullet}$ et $\overset{cl3}{\bullet}\!\!-\!\!\!-\!\!\!-\!\!\!-\!\!\overset{cl2}{\bullet}$

$\overset{cl1}{\bullet}\!\!-\!\!\!-\!\!\!-\!\!\!-\!\!\overset{cl1}{\bullet}$ et $\overset{cl1}{\bullet}\!\!-\!\!\!-\!\!\!-\!\!\!-\!\!\overset{cl1}{\bullet}$ se réduit en $\overset{cl1}{\bullet}\!\!-\!\!\!-\!\!\!-\!\!\!-\!\!\overset{cl3}{\bullet}$ et $\overset{cl3}{\bullet}\!\!-\!\!\!-\!\!\!-\!\!\!-\!\!\overset{cl1}{\bullet}$

Sommaire sur l'autostabilisation

- Définition d'algorithmes indépendants des états initiaux
- Algorithmes très complexes à vérifier en particulier *la terminaison*
- Version autostabilisante de beaucoup d'algorithmes

Terminaison de l'algorithme

- stabilité : un seul processus a la main ou un seul $i \in 0..N \wedge v[i] = v[i +_N 1]$.
- propriété 1 : il y a au moins un processus avec une garde franchissable :
  - tous les processus de 1 à N ont la même valeur (hypothèse)
  - nécessairement 0 et N ont la même valeur et 0 a la main.
- propriété 2 : toute configuration légale satisfait la propriété de clôture :
  - les processus de 0 à i-1 ont la même valeur
  - les processus de i à N ont la même valeur
  - $v[i] \neq v[i-1]$
- propriété 3 : à partir de toute configuration illégale, l'anneau atteint une configuration légale (le nombre de processus ayant la main ne peut pas croître et au mieux il peut stagner)
  - $0 \ldots i-1$: non franchissable
  - chaque processus maintient le nombre de processus franchissables : on construit à partir de p, une suite w1 ... wN différent deux à deux
  - cette suite ne peut pas rester indéfiniment stable pr propagation de la valeur et respect de la constnace.

Conclusion

- Algorithmes répartis : problème de l'expression locale de la globalité
- Algorithmes très complexes à vérifier
- Prise en compte de nombreux aspects de l'environnement comme les fautes, les erreurs, ...

Section Courante

1. Election
   - Problème de l'élection
2. Election
   - Problème de l'élection
   - Election dans un graphe acyclique
3. Auto-stabilisation
   - Généralités
   - Définition
   - Réseaux en anneau
   - Algorithme de Dijkstra
   - Algorithme de coloriage
4. Calcul de l'état global
   - Introduction du problème
   - Etat global consistant
   - Algorithme de Chandy et Lamport

]

Etat courant

1. Election
   - Problème de l'élection
2. Election
   - Problème de l'élection
   - Election dans un graphe acyclique
3. Auto-stabilisation
   - Généralités
   - Définition
   - Réseaux en anneau
   - Algorithme de Dijkstra
   - Algorithme de coloriage
4. Calcul de l'état global
   - Introduction du problème
   - Etat global consistant
   - Algorithme de Chandy et Lamport

système réparti

- un programme réparti est constitué d'un ensemble de processus s'exécutant en parallèle et communiquant seulement par envoi de messages via des canaux de communication.
- Il n'y a pas d'horloge globale commune pour ces processus
- Les temps de transfert des messages ne sont pas bornés : sous hypothèse de communication fiable, ces temps sont finis. n a
- **Conséquence** : pas d'observation *simultanée* globale possible de l'état
- **Problème** : déterminer un état global consistant
- Le snapshot est une copie ou une photographie instantanée de l'état d'un système.

Exemples

- Calcul de la topologie d'un réseau
- Compter le nombre de processus dans un système réparti
- Détecter la terminaison
- Détecter le blocage
- Détecter la perte de coordination

Reconstituer l'état global

- Les algorithmes de snapshots permettent de donner des clichés répartis de l'état global
- Comment rassembler ces *clichés répartis* ou *snapshots* afin de donner une vue consistante et globale à chaque processus du système ?
- Proposer des algorithmes rassemblant les *fragments* en un état global.

Applications de ce problème

- *détection du blocage* : tout processus bloqué momentanément souhaite connare l'état global du système, afin de savoir s'il a atteint une configuration de blocage.
- *détection de la terminaison* : dans le cas de calculs répartis, les processus exécutent des actions au cours de phases de calculs mais le passage à une phase suivante peut exiger que le processus en cours obtienne la connaisance de l'état des autres processus : nécessité de connaître l'état global.
- *détection de la conservation de jetons dans un réseau*
- *réinitialisation du réseau* : un système réparti peut entrer dans une configuration nécessitant de relancer ou de réinitilaiser l'état global cournat et cette relance peut être faite à partir des snapshots conservés qui vont donc constituer des points de reprise (*checkpoints*)

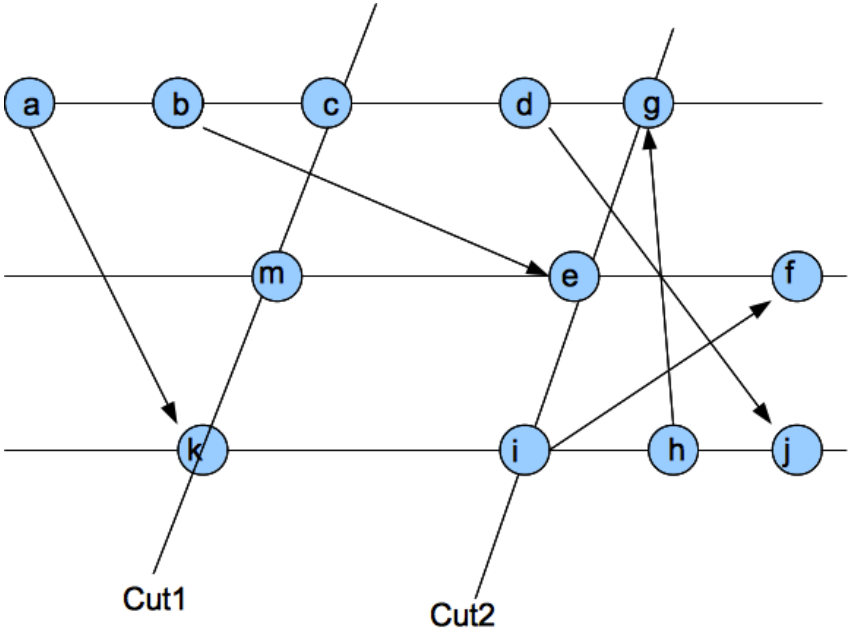
Etat courant

1. Election
   - Problème de l'élection
2. Election
   - Problème de l'élection
   - Election dans un graphe acyclique
3. Auto-stabilisation
   - Généralités
   - Définition
   - Réseaux en anneau
   - Algorithme de Dijkstra
   - Algorithme de coloriage
4. Calcul de l'état global
   - Introduction du problème
   - Etat global consistant
   - Algorithme de Chandy et Lamport

Modélisation d'un système réparti

- Un système réparti est modélisé comme un ensemble fini de processus séquentiels
- Chaque processus i est caractérisé par un état local noté LS_i
- Chaque canal de communication C_{ij} a un état noté SC_{ij}.
- Chaque processus séquentiel peut réaliser trois types d'actions :
  - Une action interne modifiant l'état local
  - Une envoi sur un canal
  - Une réception sur un canal

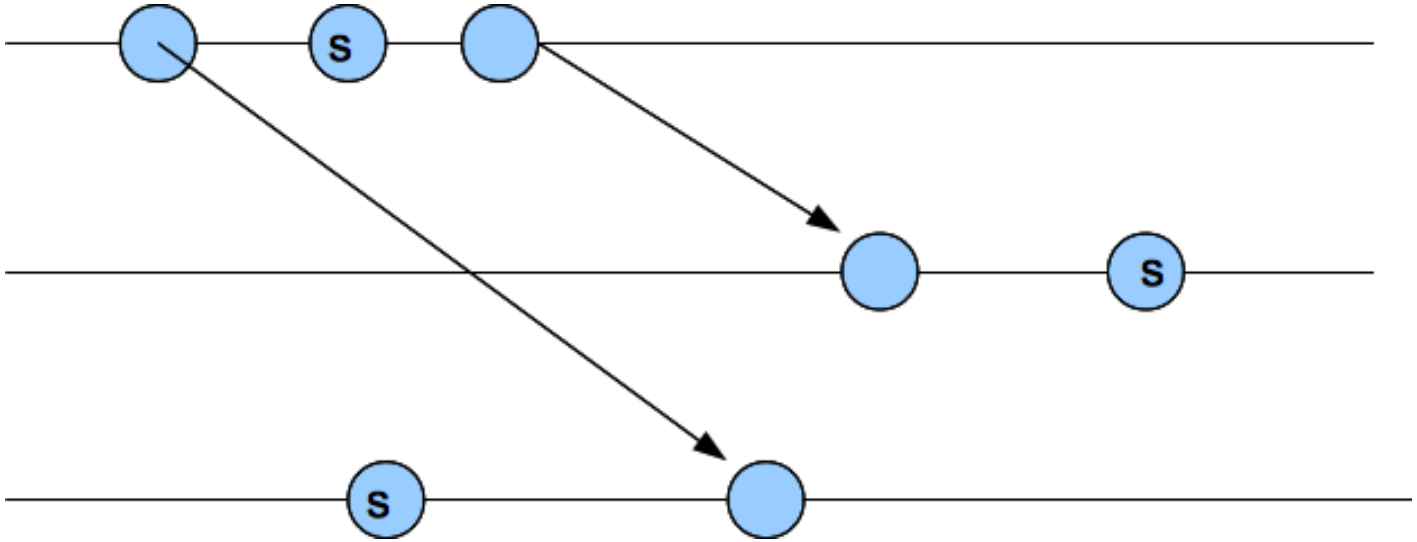
Coupures



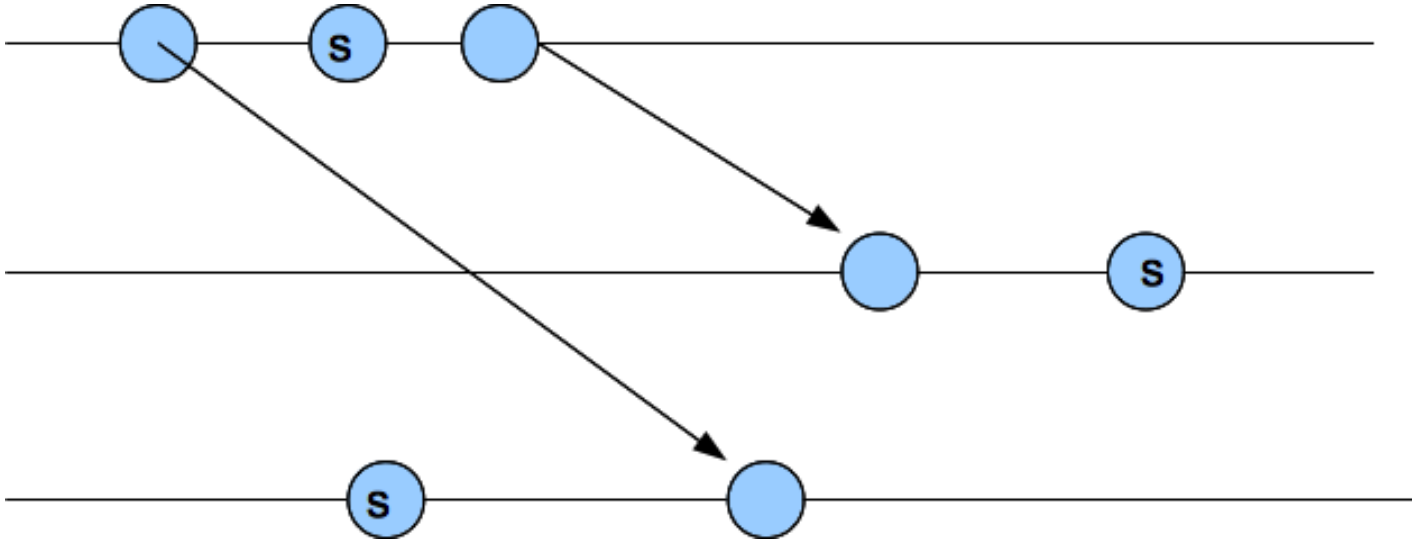
- $Cut1 = \{a, b, c, m, k\}$
- $Cut2 = \{a, b, c, d, g, m, e, k, i\}$

- Une coupure C est un ensemble d'événements
- Une coupure consistante C est une coupure satisfaisant la propriétét suivante : $\forall a, b. a \in C \wedge b < a \Rightarrow b \in C$
- $Cut1$ est consistante mais $Cut2$ n'est pas consistante ($h < g$ mais $h \notin Cut2$).

Cas pathologique

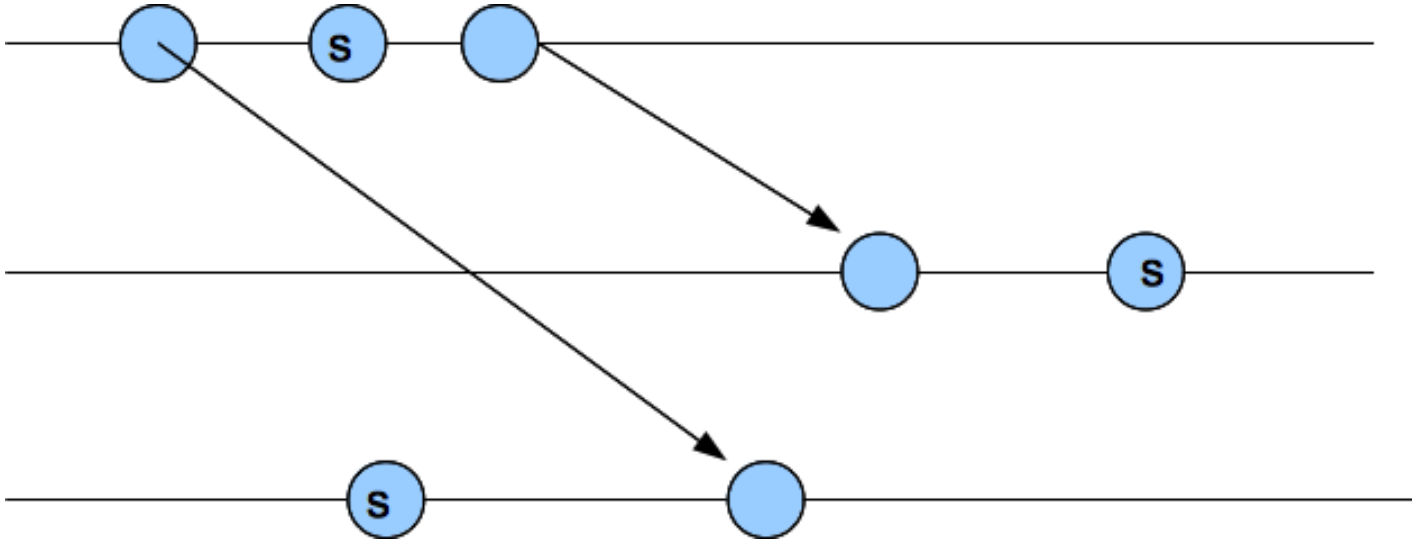


Cas pathologique



- Un message est reçu parce qu'il a été envoyé

Cas pathologique



- Un message est reçu parce qu'il a été envoyé
- *Pour tout canal de communication, tout message reçu a été envoyé*

Des coupures aux snapshots

- Deux coupures peuvent être équivalentes dans la mesure où l'échange de deux actions indépendantes c'est-à-dire causalement indépendantes.
- Pour une coupure donnée, on peut définir l'ensemble des événements les plus récents : ces événements constituent une frontière du calcul en cours.
- Une coupure induit donc un snapshot : une photographie du système et de son état global.
- Toute coupure consistante induit un snapshot consistant et le snapshot courant est l'ensemble des états locaux suivants les événements récemment enregistrés.
- Si $C1$ et $C2$ sont deux coupures telles que $C1 \subseteq C2$, alors $S2$ est plus récent que $S1$

Calculs répartis

- Une exécution (ou un calcul) C est un ordre total des événements
- Une exécution C est *consistante*, si elle est construite en satisfaisant la condition suivante :

  Pour toute action a et b de C, si $a < b$, alors $a\ precedes\ b$.
- Deux exécutions sont équivalentes modulu la relation d'indépendance : deux actions a et b sont indépendantes si ni $a\ precedes\ b$ et ni $b\ precedes\ a$
- Les algorithmes vont construire des snapshots consistants mais vont faire des choix selon les techniques utilisées.

Etat courant

1. Election
   - Problème de l'élection
2. Election
   - Problème de l'élection
   - Election dans un graphe acyclique
3. Auto-stabilisation
   - Généralités
   - Définition
   - Réseaux en anneau
   - Algorithme de Dijkstra
   - Algorithme de coloriage
4. Calcul de l'état global
   - Introduction du problème
   - Etat global consistant
   - Algorithme de Chandy et Lamport

Hypothèses de l'algorithme

- P est un ensemble de processus connectés via un ensemble de canaux C
- Pas d'horloge commune ni de mémoire partagée
- Toute processus peut enregistrer son état local, envoyer des messages ou recevoir des messages
- Les canaux sont
  - unidirectionnels
  - gérés selon la politique FIFO
  - de capacité infinie
  - sans erreur
  - équitables
- *Construire un snapshot consistant*

Principe de cet algorithme

- On se donne deux ensembles : P est l'ensemble des processus et M est l'ensemble des messages
- On définit un message particulier appelé marqueur et noté $\bowtie$
- Un ensemble de processus P connectés par des canaux caractérisés par deux variables
  - $sent \in P \times P \nrightarrow (Nodes \nrightarrow M)$
  - $rec \in P \times P \nrightarrow (Nodes \nrightarrow M)$
  - $dom(sent) = dom(rec)$
- On se donne un initiateur du calcul $i \in P$.
- On se donne une variable s qui contient tous les processus au début de l'algorithme
- Deux actions sont possibles :
  - *CL1* : le processus i réalise les actions suivantes en un pas de calcul, si $i \in s$: soustraire i de s ; enregistrer l'état local ; envoyer le marqueur $\bowtie$ à tous ses canaux.
  - *CL2* : le processus $j \in P$ reçoit le marqueur $\bowtie$ sur un canal incident et réalise les actions suivantes : soustraire j de s ; enregistrer l'état local ; envoyer le marqueur $\bowtie$ à tous ses canaux.

Algorithme de Chandy et Lamport CL

Initialisation de CL

Initialisation du processus de calcul par un processus p tel que $taken_p = false$.

BEGIN
  $\mathbf{record(state(p))}$
  $taken_p := true$
  send m **to** $Neighbors(p)$
  For each port i of p, record messages arriving on i
END

Algorithme de Chandy et Lamport CL

Initialisation de CL

Initialisation du processus de calcul par un processus p tel que $taken_p = false$.

BEGIN
$\mathbf{record(state(p))}$
$taken_p := true$
send m **to** $Neighbors(p)$
For each port i of p, record messages arriving on i
END

- Applicable par n'importe quel nœud qui souhaite commencer

Algorithme de Chandy et Lamport CL

Initialisation de CL

Initialisation du processus de calcul par un processus p tel que $taken_p = false$.

BEGIN
$\mathbf{record(state(p))}$
$taken_p := true$
send m **to** $Neighbors(p)$
For each port i of p, record messages arriving on i
END

- Applicable par n'importe quel nœud qui souhaite commencer
- Action effectuée localement

Algorithme de Chandy et Lamport

R-CL

Une action est activée si un marqueur m arrive sur un port j de p.

BEGIN

receive m **on** j **of** p

mark$(port\ j)$

IF $not\ taken_p$ **THEN**

$taken_p := true$

$\mathbf{record(state(p))}$

send m **to** $Neighbors(p)$

For each port $i \neq j$ **of p, record messages arriving on i via** $M_{p,i}$

ELSE

Stop recording messages incoming from port j of p

$\mathbf{record}(M_{p,j})$

FI

IF p has received a marker via all incoming channels THEN

$local_snapshot_p := true$

FI

END

Propriétés de l'algorithme

- Pourquoi l'algorithme termine :
  - les canaux sont supposés fiables
  - les messages sont transmis FIFO
  - s est vide à la terminaison
  - Quand s est vide, tous les processus ont reçu le marqueur sur leurs canaux incidents.

Applications de l'algorithme

- Calculer le nombre de jetons dans un réseau
- Détermination de point de reprise de calcul

un réseau

- On considère un réseau de type anneau de 1 à 3 dans ce sens

un réseau

- On considère un réseau de type anneau de 1 à 3 dans ce sens
- CL1 : le processus 1 envoie le jeton à 2 ; le processus devient local ($n(1) = 0$) ; il positionne les variables associées aux canaux : $sent(1, 2) = 1$ et $rec(3, 1) = 0$) ; il envoie le marqueur sur le canal $(1, 2)$.

un réseau

- On considère un réseau de type anneau de 1 à 3 dans ce sens
- CL1 : le processus 1 envoie le jeton à 2 ; le processus devient local ($n(1) = 0$) ; il positionne les variables associées aux canaux : $sent(1, 2) = 1$ et $rec(3, 1) = 0$) ; il envoie le marqueur sur le canal $(1, 2)$.
- E : le processus i reçoit le jeton et le renvoie à son voisin.

un réseau

- On considère un réseau de type anneau de 1 à 3 dans ce sens
- CL1 : le processus 1 envoie le jeton à 2 ; le processus devient local $(n(1) = 0)$; il positionne les variables associées aux canaux : $sent(1, 2) = 1$ et $rec(3, 1) = 0)$; il envoie le marqueur sur le canal $(1, 2)$.
- E : le processus i reçoit le jeton et le renvoie à son voisin.
- CL2 : le processus i reçoit le marqueur ; le processus i devient local$(n(i) = 0)$; il positionne $rec(i{-}1, i) = 1$ et $sent(i, i{+}1) = 1$; il envoie le marqueur sur le canal $(i, i{+}1)$.

un réseau

- On considère un réseau de type anneau de 1 à 3 dans ce sens
- CL1 : le processus 1 envoie le jeton à 2 ; le processus devient local $(n(1) = 0)$; il positionne les variables associées aux canaux : $sent(1, 2) = 1$ et $rec(3, 1) = 0)$; il envoie le marqueur sur le canal $(1, 2)$.
- E : le processus i reçoit le jeton et le renvoie à son voisin.
- CL2 : le processus i reçoit le marqueur ; le processus i devient local$(n(i) = 0)$; il positionne $rec(i{-}1, i) = 1$ et $sent(i, i{+}1) = 1$; il envoie le marqueur sur le canal $(i, i{+}1)$.
- L'algorithme termine et le nombre de jetons enregistrés est : $n_1{+}n_2{+}n_3{+}sent(1, 2){-}rec(1, 2){+}sent(2, 3){-}rec(2, 3){+}sent(3, 1){-}rec(3,$ 1

Algorithme de Lay et Yang

Algorithme de Lay et Yang

- LY1 : L'initialisateur enregistre son état et quand il doit envoyer un message à un autre processus, il envoie le message (m,r) où r désigne une couleur.

Algorithme de Lay et Yang

- LY1 : L'initialisateur enregistre son état et quand il doit envoyer un message à un autre processus, il envoie le message (m,r) où r désigne une couleur.B
- LY2 : Quand un processus reçoit un message (m,r), il enregistre son état, si ce n'est pas déjà fait, et accepe le message m.

roa

Algorithme de Lay et Yang

- LY1 : L'initialisateur enregistre son état et quand il doit envoyer un message à un autre processus, il envoie le message (m,r) où r désigne une couleur.B
- LY2 : Quand un processus reçoit un message (m,r), il enregistre son état, si ce n'est pas déjà fait, et accepe le message m.

roa

- L'accessibilité de l'état global n'est pas garanti